

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA DE EVALUACIÓN 5 – UNIDAD 5
INTEGRACIÓN, MÉTRICAS Y DEFENSA DEL PROYECTO INTEGRADOR
SISTEMA MUNDIPETS

MATERIA:

INGENIERÍA DE REQUISITOS (ISR-401)

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

INTEGRANTES:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, efuertes2@uteq.edu.ec
Rol: Analista líder

GUTIÉRREZ ORTEGA GÉNESIS ADRIANA, CI: 1250274477, ggutierrez@uteq.edu.ec
Rol: Verificador

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amedozap9@uteq.edu.ec
Rol: Modelador

MORALES SÁNCHEZ GARY ALEJANDRO, CI: 1251154173, gmoraless2@uteq.edu.ec
Rol: Documentador

NIEVES SÁNCHEZ JIMMY SAMUEL, CI: 1250907878, jnievess@uteq.edu.ec
Rol: Verificador

CURSO:

CUARTO SEMESTRE PARALELO “B”

Versión del documento: 1.0

Repositorio GitHub (PE5):

<https://github.com/MiloSaurio4kHD/MundiPets-PE5-IngReq-Fuertes-Gutierrez-Mendoza-Morales-Nieves.git>

Repositorio GitHub (PFC base – ERS, UML, PE1–PE4):

<https://github.com/andymendoza03/MundiPets-PFC-IngReq.git>

QUEVEDO – LOS RÍOS – ECUADOR
2026 – 2027 PPA

Resumen

Index Terms

Ingeniería de requisitos, ISO/IEC/IEEE 29148:2018, ISO/IEC 25010:2023, métricas de calidad, trazabilidad end-to-end, requisitos de inteligencia artificial, MundiPets.

Índice

Fundamento Teórico	5
Marco de calidad y especificación (Fuertes)	5
Fundamentos de especificación (Gutiérrez)	5
Fundamentos de trazabilidad (Morales)	7
Fundamentos de requisitos de IA (Mendoza)	8
Fundamentos de gestión (Nieves)	9
1. Introducción	10
2. Metodología de Ingeniería de Requisitos	11
2.1. PE1: fundamentos y contexto del sistema	11
2.2. PE2: elicitación, análisis y priorización	11
2.3. PE3: especificación y modelado	12
2.4. PE4: validación mediante inspección formal y gestión de cambios	13
2.5. PE5: integración, cierre de cambios pendientes y defensa	13
2.6. Tabla resumen del proceso	14
2.7. Fundamento normativo del proceso seguido	14
3. ERS/SRS Final del Sistema MundiPets	15
3.1. Estructura conforme a ISO/IEC/IEEE 29148:2018	15
3.2. Requisitos clave y cambios incorporados en PE1–PE5	16
3.3. Control de versión, identificadores y línea base	17
3.3.1. Identificación de la versión final	17
3.3.2. Verificación de unicidad de identificadores	17
4. Modelos UML del Sistema	18
4.1. Diagrama de Casos de Uso	18
4.2. Diagrama de Contexto	19
4.3. Diagrama de Componentes	20
4.4. Diagrama de Despliegue	21
4.5. Diagramas Estratégicos (i*)	22
4.6. Diagrama de Flujo de Datos (Nivel 1)	23
4.7. Diagramas de Máquinas de Estado	24
4.8. Diagramas de Secuencia	27
4.8.1. Registrar mascota (CU-01)	27
4.8.2. Registrar historial médico y antecedentes genéticos (CU-02)	28
4.8.3. Cargar documento veterinario y carnet de vacunación (CU-03)	28
4.8.4. Publicar mascota en adopción (CU-04)	29
4.8.5. Publicar mascota para cruce y gestionar solicitud (CU-05)	30
4.8.6. Configurar privacidad médica (CU-06)	31
4.8.7. Gestionar proceso de adopción (CU-07)	32
4.8.8. Gestionar comunicación entre usuarios (CU-08)	33
4.8.9. Validar información médica (CU-09)	34
4.8.10. Consultar perfil médico (CU-10)	35
4.8.11. Buscar mascota disponible (CU-11)	36
4.8.12. Consultar recomendaciones de cuidado (CU-12)	37
4.8.13. Evaluar compatibilidad de cruce (CU-13)	37
4.9. Diagramas de Actividad	38
4.9.1. Registrar mascota (CU-01)	38
4.9.2. Registrar historial médico y antecedentes genéticos (CU-02)	39
4.9.3. Cargar documento veterinario y carnet de vacunación (CU-03)	40
4.9.4. Publicar mascota en adopción (CU-04)	41
4.9.5. Publicar mascota para cruce y gestionar solicitud (CU-05)	42
4.9.6. Configurar privacidad médica (CU-06)	43
4.9.7. Gestionar proceso de adopción (CU-07)	44
4.9.8. Gestionar comunicación entre usuarios (CU-08)	45
4.9.9. Validar información médica (CU-09)	46

4.9.10.	Consultar perfil médico (CU-10)	47
4.9.11.	Buscar mascota disponible (CU-11)	48
4.9.12.	Consultar recomendaciones de cuidado (CU-12)	49
4.9.13.	Evaluar compatibilidad de cruce (CU-13)	50
4.10.	Diagrama de Clases	51
4.10.1.	Usuarios y Roles	51
4.10.2.	Mascotas y Salud	52
4.10.3.	Comunicación y Publicaciones	53
4.10.4.	Solicitudes y Evaluaciones	54
4.10.5.	Vista Refinada General	55
5.	Validación: Inspección, Defectos y Re-inspección	56
5.1.	Resultados de la inspección de la PE4	56
5.2.	Correcciones aplicadas	56
5.3.	Re-inspección de la PE5 y defectos residuales	56
6.	Gestión y Trazabilidad	57
6.1.	Línea base y control de cambios	57
6.2.	Matriz de trazabilidad final	57
6.3.	Diagnóstico de cobertura: huérfanos y cadenas rotas	57
6.4.	Sincronización entre el <i>product backlog</i> y el ERS	57
7.	Requisitos de los Componentes de Inteligencia Artificial	58
7.1.	Identificación y justificación de los componentes	58
7.2.	Ficha del componente IA-01	58
7.3.	Requisitos funcionales y no funcionales de IA	58
7.4.	Ética, privacidad y clasificación de riesgo	58
7.5.	Plan de monitoreo y trazado de los requisitos de IA	58
8.	Métricas de Calidad del ERS	59
8.1.	Instrumento de auditoría	59
8.2.	Cálculo de las seis métricas con aritmética visible	59
8.3.	Tabla de auditoría de calidad	59
8.4.	Comparación antes/después de las mejoras aplicadas	59
9.	Retrospectiva del Equipo	60
10.	Conclusiones	61
	Conclusión 1 — Unidad 1	61
	Conclusión 2 — Unidad 2	61
	Conclusión 3 — Unidad 3	61
	Conclusión 4 — Unidad 4	61
	Conclusión 5 — Unidad 5	61
	Apéndice	65

Índice de figuras

1.	Diagrama de casos de uso general del sistema MundiPets.	18
2.	Diagrama de contexto del sistema MundiPets.	19
3.	Diagrama de componentes del sistema MundiPets.	20
4.	Diagrama de despliegue del sistema MundiPets.	21
5.	Diagrama de dependencia estratégica (SD) del sistema MundiPets.	22
6.	Diagrama de razón estratégica (SR) del sistema MundiPets.	22
7.	Diagrama de flujo de datos, Nivel 1, notación Yourdon–DeMarco.	23
8.	Diagrama de máquinas de estado – Perfil Mascota.	24
9.	Diagrama de máquinas de estado – Solicitudes.	25
10.	Diagrama de máquinas de estado – Verificación de Documentos.	26
11.	Diagrama de secuencia – Registrar mascota (CU-01).	27

12.	Diagrama de secuencia – Registrar historial médico y antecedentes genéticos (CU-02).	28
13.	Diagrama de secuencia – Cargar documento veterinario y carnet de vacunación (CU-03).	28
14.	Diagrama de secuencia – Publicar mascota en adopción (CU-04).	29
15.	Diagrama de secuencia – Publicar mascota para cruce y gestionar solicitud (CU-05).	30
16.	Diagrama de secuencia – Configurar privacidad médica (CU-06).	31
17.	Diagrama de secuencia – Gestionar proceso de adopción (CU-07).	32
18.	Diagrama de secuencia – Gestionar comunicación entre usuarios (CU-08).	33
19.	Diagrama de secuencia – Validar información médica (CU-09).	34
20.	Diagrama de secuencia – Consultar perfil médico (CU-10).	35
21.	Diagrama de secuencia – Buscar mascota disponible (CU-11).	36
22.	Diagrama de secuencia – Consultar recomendaciones de cuidado (CU-12).	37
23.	Diagrama de secuencia – Evaluar compatibilidad de cruce (CU-13).	37
24.	Diagrama de actividades – Registrar mascota (CU-01).	38
25.	Diagrama de actividades – Registrar historial médico y antecedentes genéticos (CU-02).	39
26.	Diagrama de actividades – Cargar documento veterinario y carnet de vacunación (CU-03).	40
27.	Diagrama de actividades – Publicar mascota en adopción (CU-04).	41
28.	Diagrama de actividades – Publicar mascota para cruce y gestionar solicitud (CU-05).	42
29.	Diagrama de actividades – Configurar privacidad médica (CU-06).	43
30.	Diagrama de actividades – Gestionar proceso de adopción (CU-07).	44
31.	Diagrama de actividades – Gestionar comunicación entre usuarios (CU-08).	45
32.	Diagrama de actividades – Validar información médica (CU-09).	46
33.	Diagrama de actividades – Consultar perfil médico (CU-10).	47
34.	Diagrama de actividades – Buscar mascota disponible (CU-11).	48
35.	Diagrama de actividades – Consultar recomendaciones de cuidado (CU-12).	49
36.	Diagrama de actividades – Evaluar compatibilidad de cruce (CU-13).	50
37.	Diagrama de clases – Usuarios y Roles.	51
38.	Diagrama de clases – Mascotas y Salud.	52
39.	Diagrama de clases – Comunicación y Publicaciones.	53
40.	Diagrama de clases – Solicitudes y Evaluaciones.	54
41.	Diagrama de clases – Vista refinada general (integración de las cuatro particiones).	55
42.	Vista general de la matriz de trazabilidad final de MundiPets.	57
43.	Historial de <i>commits</i> del repositorio al corte de la PE5.	65

Índice de tablas

1.	Resumen del proceso de Ingeniería de Requisitos aplicado a MundiPets, PE1–PE5.	14
2.	Correspondencia entre las cláusulas de ISO/IEC/IEEE 29148:2018 y las secciones del ERS de MundiPets.	15
3.	Requisitos con trazabilidad de cambio completa (redacción original, defecto, corrección) incorporados durante PE1–PE5.	16
4.	Identificación de la versión final del ERS.	17
5.	Verificación de unicidad y continuidad de identificadores del ERS.	17
6.	Requisitos huérfanos, artefactos huérfanos y cadenas rotas.	57
7.	Sincronización entre el <i>product backlog</i> y el ERS.	57
8.	Ficha del componente de inteligencia artificial IA-01.	58
9.	Requisitos funcionales y no funcionales de los componentes de IA.	58
10.	Tabla de auditoría de calidad del ERS de MundiPets.	59
11.	Comparación antes/después de las mejoras aplicadas al ERS.	59
12.	Retrospectiva Start–Stop–Continue del equipo.	60
13.	Reparto de la exposición y los tiempos de la defensa.	62
14.	Preguntas del tribunal, respuesta preparada y artefacto de respaldo.	65
15.	Declaración individual de aporte al proyecto integrador.	65
16.	Declaración de uso de inteligencia artificial por sección.	65
17.	Matriz de trazabilidad final del sistema MundiPets.	66

Fundamento Teórico

Marco de calidad y especificación de requisitos

El sustento teórico de esta auditoría descansa en tres cuerpos normativos y metodológicos que se aplican directamente en las Secciones 5 y 8 de este informe.

En primer lugar, la norma ISO/IEC 25010:2023 [1] define el modelo de calidad de producto de software mediante ocho características —adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad—, de las cuales se derivan las seis métricas M1–M6 empleadas en la Sección 8: completitud y consistencia como propiedades estructurales de la especificación, verificabilidad y trazabilidad como propiedades de comprobabilidad, y modificabilidad y corrección como propiedades de mantenimiento del artefacto documental. El propio ERS de MundiPets declara explícitamente su cobertura de las nueve características del modelo (Sección 3.3, tabla de cobertura ISO/IEC 25010), lo que permite anclar cada RNF a una característica normativa concreta en lugar de a un criterio subjetivo de calidad.

En segundo lugar, los principios de medición y experimentación en ingeniería de software de Wohlin et al. [2] establecen que toda métrica de proceso o de producto debe apoyarse en un conteo verificable y reproducible, no en una estimación cualitativa del equipo. Este principio es el que exige, en la Sección 8.2, mostrar la división aritmética completa de cada métrica —numerador, denominador y fuente exacta del dato— en lugar de reportar directamente un porcentaje. Un umbral pedagógico sin conteo detrás no constituye una métrica: es una afirmación no verificable, exactamente la falla que este marco busca prevenir.

En tercer lugar, ISO/IEC/IEEE 29148:2018 [3] fija las características de calidad exigibles a un requisito individual y al conjunto de requisitos de un SRS: necesario, no ambiguo, completo, singular, factible, verificable, correcto, consistente con otros requisitos, y trazable hacia atrás y hacia adelante. La métrica M3 (verificabilidad) de la Sección 8 opera directamente esta característica: un requisito solo se cuenta como verificable si su criterio de aceptación permite diseñar una prueba objetiva, sin depender de adjetivos no medibles (“rápido”, “adecuado”, “amigable”) que no fijan umbral, unidad ni método de comprobación.

Estos tres marcos —calidad de producto (25010), medición reproducible (Wohlin) y calidad de la especificación (29148)— no se aplican de forma aislada: la auditoría de la Sección 8 es, en esencia, la operación conjunta de los tres sobre el ERS final de MundiPets. Se eligieron por ser las fuentes normativas que la propia asignatura y el propio ERS del equipo ya adoptan como marco de referencia, evitando introducir un estándar de calidad ajeno al resto del proyecto.

Fundamentos de la especificación y el ciclo de vida

Ingeniería de requisitos y fundamentos de los requisitos

La ingeniería de requisitos comprende el conjunto de actividades orientadas a identificar, analizar, documentar, validar y gestionar las necesidades que un sistema debe satisfacer, de modo que dichas necesidades puedan servir de base verificable para el resto de las actividades de desarrollo [3, 4]. Esta disciplina no se limita a recoger enunciados de los interesados: los organiza mediante un marco de trabajo que distingue el contexto en que opera el sistema, el propio proceso de obtención y negociación, y los artefactos documentados que resultan de ese proceso [4]. En el caso de MundiPets, este marco es lo que permite tratar los requisitos no como una lista informal recogida de los futuros usuarios de la plataforma, sino como un conjunto estructurado que puede auditarse, corregirse y cerrarse con criterios objetivos.

Un requisito individual, para ser útil al proceso de desarrollo, debe reunir un conjunto de características de calidad reconocidas: debe ser necesario, no ambiguo, completo, singular, factible, verificable y trazable, entre otras [3]. Estas características no son exigencias puramente formales: son la condición que hace posible que, más adelante, un requisito pueda auditarse con una métrica, enlazarse a un caso de uso y a un caso de prueba, y defenderse ante un tribunal con un artefacto concreto que lo respalde. Por ello, el fundamento de la especificación de MundiPets no puede separarse del fundamento de la calidad del requisito individual.

Elicitación, análisis y validación de requisitos

Antes de que un requisito pueda especificarse con calidad, debe elicitar, analizarse y negociarse con los interesados del sistema. Pohl describe estas actividades como parte de un ciclo continuo en el que la elicitación aporta el conocimiento disperso entre los interesados, el análisis lo estructura y depura, y la negociación resuelve los conflictos de intereses que surgen entre distintos puntos de vista sobre el mismo sistema [4]. El SWEBOK

sitúa estas mismas actividades dentro del área de conocimiento de Requisitos de Software, subrayando que la elicitación y el análisis no son un paso previo desechable, sino una fuente de retroalimentación continua sobre la propia especificación [5]. El temario del IREB para el nivel Foundation coincide en este punto: la obtención, la documentación y la validación de requisitos se presentan como prácticas interdependientes, no como fases estancas de un proceso lineal [6].

La validación cierra este ciclo al comprobar que los requisitos documentados representan realmente lo que los interesados necesitan, y no únicamente lo que fue posible transcribir en una primera iteración [3, 4]. En el proyecto integrador, esta actividad se corresponde con la inspección formal ejecutada en la PE4 y con la re-inspección de la PE5 sobre el ERS ya corregido (Sección 5): ambas son, en términos de este marco teórico, instancias concretas de validación de requisitos aplicadas al caso de MundiPets.

Especificación de requisitos y estructura del ERS

La especificación traduce el resultado de la elicitación, el análisis y la validación en una representación documentada, estructurada y verificable de las necesidades y características que debe satisfacer el sistema [3]. La ISO/IEC/IEEE 29148:2018 define el contenido y la estructura exigibles a una Especificación de Requisitos de Software (SRS), estableciendo las cláusulas que un documento de este tipo debe contemplar y las características de calidad aplicables tanto al conjunto de requisitos como a cada requisito individual [3]. El SWEBOK, por su parte, ubica la especificación como una de las prácticas centrales del área de Requisitos de Software y la vincula explícitamente con la gestión de requisitos y con la trazabilidad, entendida esta última como la capacidad de seguir la vida de un requisito en ambas direcciones a lo largo del documento y del resto de artefactos del proyecto [5]. Pohl aporta a esta misma idea el fundamento conceptual de los tipos de artefactos de requisitos objetivos, escenarios y requisitos orientados a la solución que la especificación debe documentar de forma coherente entre sí [4].

Desde esta perspectiva, el ERS de MundiPets no constituye únicamente un documento descriptivo del sistema, sino un artefacto que fija una referencia verificable para el desarrollo posterior: cada requisito documentado en él debe poder auditarse contra los criterios de completitud, consistencia y verificabilidad que se aplican en la Sección 8, y debe poder trazarse hacia los casos de uso, los modelos y los casos de prueba que lo realizan, tal como exige la cadena de trazabilidad de la Sección 6.

Requisitos dentro del ciclo de vida del sistema y del software

Los requisitos no se especifican de manera aislada: se insertan dentro de un ciclo de vida que abarca, según el nivel de abstracción, al sistema completo o específicamente al software. La ISO/IEC/IEEE 15288:2023 define un conjunto de procesos de ciclo de vida aplicables a sistemas creados por el ser humano y sitúa la definición de las necesidades y requisitos de las partes interesadas, así como el análisis de los requisitos del sistema, dentro de los procesos técnicos del ciclo de vida [7]. Esta norma proporciona el encuadre bajo el cual MundiPets se concibe como un sistema con procesos de definición, desarrollo, verificación, validación y evolución propios, de los cuales la especificación de requisitos es una entrada temprana y determinante.

Cuando el interés se concentra específicamente en el software, la ISO/IEC/IEEE 12207:2017 en su edición de 2017, vigente al momento de redactar este informe define el marco de procesos de ciclo de vida del software, incluidos los relacionados con su desarrollo, su mantenimiento y su operación, y comparte con la ISO/IEC/IEEE 15288:2023 una estructura de procesos armonizada para facilitar su uso conjunto [8]. Para MundiPets, esto significa que los requisitos del ERS no se cierran en la PE5 de forma definitiva: quedan insertados en un ciclo de vida de software que contempla su evolución futura, su mantenimiento y su eventual modificación, razón por la cual la trazabilidad y la modificabilidad auditadas en la Sección 8 no son un ejercicio académico aislado, sino una condición para que ese ciclo de vida posterior sea manejable.

Prácticas reconocidas de ingeniería de requisitos

Más allá de las normas de proceso, el equipo se apoya en dos cuerpos de conocimiento que consolidan las prácticas reconocidas de la disciplina. El SWEBOK Guide, en su versión 4.0, dedica un área de conocimiento completa a los Requisitos de Software y organiza sus prácticas en torno a los fundamentos del requisito, la elicitación, el análisis, la especificación, la validación y la gestión de requisitos, incluida la trazabilidad [5]. El IREB, por su parte, sostiene el temario CPRE Foundation Level en su versión 3.3.0, que estructura el conocimiento básico exigible a un ingeniero de requisitos en torno a los interesados, las técnicas de elicitación,

la documentación de requisitos, su validación, su gestión y su comunicación [6]. Ambos cuerpos de conocimiento coinciden con Pohl y con la propia ISO/IEC/IEEE 29148:2018 en un punto central para MundiPets: ninguna de estas prácticas se sostiene si el requisito documentado no puede verificarse objetivamente, lo que en esta práctica se traduce en la exigencia de un criterio de aceptación en formato Dado–Cuando–Entonces para cada requisito del sistema (Sección 3.2).

Integración de los fundamentos para el cierre del ERS de MundiPets

Las seis fuentes utilizadas en este bloque cumplen funciones complementarias y no redundantes dentro de la argumentación que sustenta el cierre del ERS de MundiPets. La ISO/IEC/IEEE 29148:2018 aporta el contenido y la estructura exigibles a la especificación misma: es la norma contra la cual se verifica que el documento entregado sea, formalmente, un SRS y no una colección de notas de reunión [3]. La ISO/IEC/IEEE 15288:2023 sitúa esa especificación dentro del ciclo de vida del sistema MundiPets en su conjunto, mientras que la ISO/IEC/IEEE 12207:2017 hace lo propio para el ciclo de vida específico del software que se construirá a partir de ese ERS [7, 8]; ambas normas comparten estructura de procesos, de modo que no compiten entre sí, sino que delimitan el nivel de abstracción (sistema o software) desde el que se lee la misma cadena de requisitos. Pohl aporta los fundamentos y las actividades de la ingeniería de requisitos que preceden a la especificación formal (elicitación, análisis, negociación, validación) y que explican por qué el contenido del ERS es defendible y no arbitrario [4]. El SWEBOK aporta el cuerpo de conocimiento y las prácticas reconocidas de la disciplina que enmarcan estas actividades dentro de la ingeniería de software en general [5], y el temario CPRE Foundation Level del IREB aporta las prácticas estructuradas y verificables de ingeniería de requisitos que un profesional certificable debe poder aplicar [6].

La integración de estos seis fundamentos permite sostener que el cierre del ERS de MundiPets no es un acto administrativo de fin de semestre, sino la culminación fundamentada de un proceso: los requisitos fueron elicitados y analizados conforme a las prácticas descritas por Pohl, el SWEBOK y el IREB [4-6]; fueron especificados con la estructura y las características de calidad que exige la ISO/IEC/IEEE 29148:2018 [3]; fueron auditados, corregidos y trazados de extremo a extremo conforme a esas mismas características (Secciones 6 y 8); y quedan insertados en un ciclo de vida de sistema y de software reconocido por la ISO/IEC/IEEE 15288:2023 y la ISO/IEC/IEEE 12207:2017 [7, 8], que sostiene su evolución más allá de esta práctica. Sobre esa base, y no sobre una declaración de intenciones, el equipo considera que el ERS final de MundiPets reúne el fundamento teórico y normativo necesario para su cierre.

Fundamentos de la trazabilidad y el modelado

El planteamiento seminal de Gotel y Finkelstein [9] identificó que buena parte de los problemas prácticos de trazabilidad no surgen de una técnica insuficiente para vincular artefactos, sino de una falta de acuerdo previo sobre qué tipo de trazabilidad se está exigiendo. Los autores separan el problema en dos direcciones complementarias: la trazabilidad previa a la especificación, que vincula un requisito hacia atrás con la evidencia que lo originó, ya sea una entrevista, una observación o una norma legal; y la trazabilidad posterior a la especificación, que lo vincula hacia adelante con aquello en lo que ese requisito se transforma durante el desarrollo, como un caso de uso, una clase o un caso de prueba. Esta distinción sustenta directamente la columna Fuente y la columna Estado de la traza de la matriz del punto 6.2 de este informe: la primera resuelve la pregunta de dónde nació este requisito, y la segunda, la pregunta de en qué se convirtió; sin esa separación conceptual, ambas preguntas tienden a mezclarse en una sola verificación superficial que no detecta en cuál de las dos direcciones falla realmente la cadena.

La revisión sistemática más reciente de Mucha, Kaufmann y Riehle [10], basada en el análisis de setenta y siete estudios publicados entre 1992 y 2022, retoma específicamente la mitad de ese marco correspondiente a la trazabilidad previa a la especificación, señalada por los propios autores como la menos estudiada de las dos, y documenta de forma sistemática los problemas recurrentes que enfrenta un equipo al intentar mantenerla vigente durante todo el ciclo de vida del proyecto. Entre esos problemas se encuentran precisamente las dos patologías que este informe debió diagnosticar y cerrar en la Sección 6.3: la cadena rota, cuando un artefacto de diseño, como un caso de uso o una clase, se construye sin que ningún requisito lo respalde realmente, señal de alcance no solicitado; y el requisito huérfano, cuando un requisito carece de fuente identificable o de cualquier artefacto posterior que lo realice, señal de que fue documentado sin evidencia suficiente o abandonado tras su especificación. Que estas patologías aparezcan documentadas de forma independiente en la literatura de trazabilidad, y no solo como una ocurrencia particular de este proyecto, respalda que su tratamiento en la

Sección 6.3 no fue un ejercicio cosmético, sino la aplicación de un problema ya caracterizado formalmente en la disciplina.

La cadena mínima que exige la PE5, compuesta por Fuente, RF, CU, Clase, Estado, Criterio BDD y Caso de Prueba, es en consecuencia la aplicación directa de este marco al proyecto MundiPets: sus primeros dos eslabones, Fuente y RF, instancian la trazabilidad previa a la especificación de Gotel y Finkelstein, y los eslabones restantes, desde RF hasta el Caso de Prueba pasando por CU, Clase, Estado y Criterio BDD, instancian su trazabilidad posterior a la especificación. Verificar ambas direcciones en una sola matriz, en lugar de documentar solamente el origen de los requisitos o solamente su realización en los modelos de la Sección 4, es lo que permite detectar tanto las cadenas rotas como los huérfanos que describen Mucha et al., y es exactamente el criterio bajo el cual se construyeron y depuraron la lectura interpretativa de los modelos UML de la Sección 4 y la matriz final del punto 6.2.

Fundamentos de los requisitos de inteligencia artificial

Especificar los componentes de inteligencia artificial (IA) de MundiPets –moderación de mensajes, verificación de imagen y recomendación de compatibilidad de cruza– exige un marco distinto del que basta para un requisito determinista, porque el comportamiento del componente no está fijado en el código sino aprendido a partir de datos. Vogelsang y Borg documentaron este desplazamiento de paradigma, de programar a entrenar, a partir de un estudio con científicos de datos: sus hallazgos muestran que la ingeniería de requisitos para sistemas de aprendizaje automático (AA) obliga al equipo a comprender las métricas de rendimiento del modelo para redactar requisitos funcionales verificables, y a incorporar atributos de calidad nuevos –explicabilidad, ausencia de discriminación y requisitos legales específicos– que no tienen equivalente directo en el desarrollo de software tradicional [11]. Esta distinción es la base de la Sección 7: un RF-IA no describe una regla fija sino el comportamiento esperado de un modelo bajo incertidumbre, y un RNF-IA debe fijar umbral, unidad y método de comprobación precisamente porque, a diferencia de un RNF tradicional, el valor obtenido puede variar según los datos con los que se entrenó o se está evaluando el componente.

Esta observación puntual se confirma a escala mediante el mapeo sistemático de Ahmad, Abdelrazek, Arora, Bano y Grundy, que revisó 43 estudios primarios sobre ingeniería de requisitos para sistemas de IA: los autores concluyen que las herramientas y plantillas de especificación heredadas del desarrollo de software tradicional no capturan de forma adecuada requisitos propios de la IA como los datos de entrenamiento, la ética o la explicabilidad, y señalan que la evaluación empírica de estos atributos –particularmente ética, explicabilidad y confianza– sigue siendo escasa en la literatura [12]. Umm-e-Habiba, Haug, Bogner y Wagner llegan a una conclusión convergente desde un mapeo posterior centrado en la práctica profesional: la disciplina de RE para sistemas de IA es todavía inmadura frente a la RE tradicional, con vacíos particularmente marcados en la gestión de requisitos de datos y en la trazabilidad entre requisito y componente entrenado [13]. Esta brecha documentada desde dos ángulos distintos –el mapeo de la literatura académica y el análisis de la práctica profesional– es el motivo por el cual la ficha de componente de la Sección 7 obliga a completar explícitamente los campos de datos de entrenamiento, equidad y explicabilidad, y por el cual la Sección 7 exige trazar cada RF-IA y RNF-IA hacia los requisitos preexistentes del sistema, en vez de dejarlos como una sección narrativa aparte del resto de la especificación.

Un problema relacionado, y no solo de especificación sino de proceso, es el que documentan Amershi et al. a partir de un estudio de campo con equipos de Microsoft: incluso en una organización con procesos ágiles maduros, la integración de componentes de aprendizaje automático introdujo dificultades adicionales de gestión –como el acoplamiento entre datos, modelo y código, y la dificultad de fijar criterios de aceptación estables cuando el desempeño del modelo cambia con cada reentrenamiento– que los flujos de trabajo Scrum convencionales no anticipaban [14]. Este hallazgo justifica que el plan de monitoreo de la Sección 7 no se trate como una tarea de una sola vez, sino como una traza continua que debe revisarse en cada iteración del componente.

Sobre esa base, Habibullah, Gay y Horkoff aportan evidencia empírica de que los requisitos no funcionales de aprendizaje automático reciben en la práctica profesional un tratamiento heterogéneo: los equipos rara vez documentan explícitamente atributos como equidad, explicabilidad o robustez frente a datos fuera de distribución, y cuando lo hacen carecen de un método sistemático de medición [15]. Este hallazgo es la justificación directa de exigir, en la Sección 7, que cada RNF-IA de MundiPets declare umbral numérico, unidad y método de comprobación explícitos: sin esa disciplina, la moderación de mensajes o la verificación de imagen quedarían documentadas como intención (“el modelo debe ser preciso”) y no como un requisito auditable.

El componente legal del marco proviene de dos fuentes complementarias. El Reglamento (UE) 2024/1689, conocido como Ley de IA, adopta un enfoque basado en niveles de riesgo y establece obligaciones de transparencia, supervisión humana y documentación técnica proporcionales al riesgo que introduce cada sistema

de IA [16]; aunque MundiPets opera en Ecuador y no está sujeto directamente a su jurisdicción, el equipo lo adopta como referencia de buenas prácticas para clasificar el riesgo de sus tres componentes y fijar las obligaciones de supervisión humana descritas en la Sección 7.4. La base legal aplicable dentro de Ecuador es la Ley Orgánica de Protección de Datos Personales, vigente desde su publicación en el Registro Oficial en 2021, que exige finalidad determinada, minimización de datos y consentimiento del titular para el tratamiento de datos personales [17]; esa misma ley fue desarrollada operativamente dos años después por su Reglamento General, que precisa los mecanismos concretos de seudonimización, plazos y conservación de la información que la ley de 2021 dejó enunciados en términos generales [18]. Se citan ambas normas, y no solo la más reciente, porque no son versiones sucesivas de un mismo texto sino dos niveles distintos y vigentes de la misma obligación: la ley fija qué se debe garantizar y el reglamento fija cómo se implementa; omitir la ley de 2021 dejaría sin base normativa directa a la obligación misma que el reglamento de 2023 desarrolla.

En conjunto, estas fuentes explican por qué un componente probabilístico exige medidas que un requisito determinista no necesita: un RF tradicional de MundiPets, como registrar el perfil médico de una mascota, se comprueba una sola vez contra una regla fija y no cambia de comportamiento si cambian los datos de entrada. Un componente de IA, en cambio, puede degradar su desempeño con el tiempo (deriva de datos), producir resultados distintos para grupos de usuarios distintos (inequidad) y tomar decisiones cuyo razonamiento interno no es directamente observable (opacidad); por eso la Sección 7 exige, además de los RF y RNF de rendimiento habituales, RNF de equidad, un RNF de explicabilidad y un plan de monitoreo en producción que ningún requisito determinista de MundiPets necesita.

Fundamentos de gestión de proyecto y trabajo en equipo

La gestión del proyecto integrador PE1–PE5 de MundiPets se organiza siguiendo los ocho dominios de desempeño de la 7.^a edición del PMBOK Guide: partes interesadas, equipo, enfoque de desarrollo y ciclo de vida, planificación, trabajo del proyecto, entrega, medición e incertidumbre [19]. A diferencia de ediciones anteriores centradas en procesos secuenciales, la 7.^a edición adopta principios orientados a valor y desempeño adaptable, lo que resulta coherente con el proceso realmente seguido por el equipo: iteraciones cortas por práctica (PE1 a PE5), entregables incrementales del ERS y retroalimentación continua del docente, en lugar de una planificación cerrada de una sola vez [19].

Esta forma de trabajar exige, además, una fundamentación propia de la ingeniería de requisitos ágil. La revisión sistemática de Hoy y Xu identifica los retos recurrentes de conciliar la especificación rigurosa de requisitos con ciclos de entrega cortos: comunicación insuficiente entre roles, priorización inestable y trazabilidad que se degrada cuando el equipo no documenta explícitamente sus decisiones [20]. El marco conceptual que proponen centrado en comunicación estructurada, priorización basada en valor y gestión activa del cambio es directamente aplicable al reparto de responsabilidades y a las tres dependencias críticas del cronograma: un retraso en la entrega de la matriz de trazabilidad propaga directamente el riesgo hacia las tareas dependientes de auditoría de métricas y de requisitos de IA, exactamente el tipo de fragilidad de coordinación que documentan Hoy y Xu [20].

Bajo este marco, la evidencia de Git (commits distribuidos por integrante, ramas por criterio de la rúbrica, mensajes descriptivos y frecuencia mínima semanal) no es un simple registro técnico de cambios en el código fuente: es un artefacto de gestión en el sentido que le da el PMBOK al dominio de desempeño de medición [19], porque permite verificar objetivamente sin depender de declaraciones subjetivas el cumplimiento del plan de trabajo, la distribución real de la carga entre los cinco integrantes y el factor de ajuste individual (FAI) que exige la rúbrica de la PE5.

1 Introducción

Problema. Los procesos de adopción y cruce responsable de mascotas se desarrollan hoy, en su mayoría, de manera informal: mediante publicaciones dispersas en redes sociales, grupos de mensajería o acuerdos verbales entre particulares, sin un canal centralizado que garantice la veracidad de la información publicada ni la trazabilidad del proceso. La elicitación realizada en PE1–PE2 documentó esta problemática de forma directa: los propietarios y adoptantes entrevistados señalaron reiteradamente la falta de transparencia de algunos refugios sobre el estado sanitario de las mascotas muchas veces por saturación y falta de financiamiento fijo, casos de mascotas que no correspondían a lo anunciado, y estafas en adopciones en línea mediante cobros sucesivos por concepto de envío o implementos. A esto se suma la ausencia de mecanismos de verificación de la identidad del propietario y de la calidad de las fotografías publicadas, lo que facilita la proliferación de perfiles fraudulentos o irrelevantes. MundiPets responde a este problema como una plataforma digital que centraliza la información de las mascotas, valida su publicación y ofrece mecanismos de apoyo incluyendo componentes basados en inteligencia artificial para moderar contenido y sugerir compatibilidad, sin sustituir en ningún caso la evaluación profesional veterinaria.

Contexto. El dominio de MundiPets involucra a propietarios de mascotas, adoptantes, interesados en procesos de cruce, refugios y veterinarias aliadas, cada uno con necesidades y niveles de acceso distintos dentro de la plataforma. El entorno organizacional es el de un proyecto académico integrador desarrollado durante las prácticas PE1 a PE5 de la asignatura Ingeniería de Requisitos (ISR-401), en el que el equipo actuó simultáneamente como analista de requisitos y como validador de su propio trabajo mediante inspecciones formales. La evidencia recogida en PE1–PE2 (entrevistas y cuestionarios a propietarios, adoptantes, interesados en cruce y refugios) reveló, además, posturas heterogéneas frente a la privacidad de la información: mientras una parte de los usuarios entrevistados prefiere compartir abiertamente el historial médico de sus mascotas porque considera que beneficia al animal ante una emergencia, otra parte delimita como sensible únicamente el tipo de sangre y las enfermedades, reservando ese dato al veterinario. Esta heterogeneidad condicionó directamente las decisiones de especificación registradas en el ERS y sustenta, en particular, los requisitos de base legal y clasificación de riesgo de los componentes de IA descritos en la Sección 7.

Objetivos. El objetivo general de la PE5 es consolidar e integrar el trabajo de las cuatro prácticas previas en un informe final único que demuestre, con evidencia auditable, que el ERS de MundiPets cumple los atributos de calidad exigidos por [3]. De este objetivo general se desprenden cinco objetivos específicos, correspondientes a las cinco secciones centrales del informe: (1) auditar la calidad del ERS mediante las seis métricas M1–M6 con aritmética visible y trazable a los conteos base; (2) cerrar la trazabilidad end-to-end del sistema, resolviendo requisitos huérfanos, artefactos huérfanos y cadenas rotas; (3) especificar formalmente los componentes de inteligencia artificial del sistema, con requisitos funcionales y no funcionales medibles; (4) consolidar una versión final única y coherente del ERS/SRS, con control de versión e identificadores verificados; y (5) preparar y sustentar la defensa técnica individual ante el tribunal, evidenciando dominio del proyecto completo.

Estructura del documento. Tras este Fundamento Teórico, que reúne las bases conceptuales y normativas aplicadas por cada integrante siguiendo los principios de elicitación y validación de [4], el informe se organiza en diez secciones: la Sección 2 describe la metodología de ingeniería de requisitos efectivamente seguida en PE1–PE5; la Sección 3 presenta el ERS/SRS final conforme a la estructura de [3]; la Sección 4 ofrece una lectura interpretativa de los modelos UML vigentes; la Sección 5 documenta la inspección formal y la reinspección de defectos; la Sección 6 desarrolla la gestión y la trazabilidad, incluida la matriz final; la Sección 7 especifica los requisitos de los componentes de inteligencia artificial; la Sección 8 presenta el instrumento y los resultados de la auditoría de calidad del ERS; la Sección 9 recoge la retrospectiva del equipo; y la Sección 10 cierra con cinco conclusiones integradoras, una por cada unidad del proyecto. El Apéndice reúne el instrumento de auditoría completo, el banco de preguntas del tribunal, la declaración individual de aporte y la matriz de trazabilidad final en su versión legible.

2 Metodología de Ingeniería de Requisitos

El equipo de la PE5 no se conformó desde la primera práctica. Durante PE1–PE4, cada integrante trabajó el mismo sistema del PFC — MundiPets — en un subgrupo distinto del curso, y solo a partir de la PE5 el docente reorganizó a los cinco integrantes en un único equipo. Esta circunstancia no resta validez al proceso: los cuatro subgrupos aplicaron, de manera independiente, el mismo marco metodológico sobre el mismo dominio de problema, lo que permite ahora contrastar y consolidar cuatro ejecuciones reales de un mismo proceso de Ingeniería de Requisitos en lugar de una sola. La presente sección describe ese proceso tal como se ejecutó, práctica por práctica, y no un proceso ideal tomado de un manual: cada actividad que se describe a continuación está respaldada por un artefacto verificable en los repositorios de PE1–PE4 del equipo.

2.1 PE1: fundamentos y contexto del sistema

La primera práctica no consistió en elicitar requisitos de campo, sino en construir el marco conceptual necesario para hacerlo con criterio en la práctica siguiente. La actividad central fue doble: por un lado, la caracterización de MundiPets como sistema su tipo según la clasificación de sistemas reactivos, transformacionales e interactivos que propone Pohl [4], y la identificación de sus stakeholders mediante una matriz de poder e interés fundamentada en el PMBOK 7.^a edición [19]; por otro, la aplicación de las ocho perspectivas de contexto de Pohl (dominio, uso, organización, tiempo, sujeto, desarrollo, intención y tecnología) junto con una declaración de visión en el formato propuesto por Moore [4].

Se optó por esta técnica y no por una elicitación directa porque, en ausencia de un marco de análisis previo, una primera sesión con usuarios corre el riesgo de producir una lista de deseos sin estructura ni cobertura demostrable de las dimensiones que un sistema sociotécnico como MundiPets una plataforma de adopción, cruza responsable y bienestar animal efectivamente involucra. Los cuatro subgrupos coincidieron en clasificar a MundiPets como un sistema de información colaborativo con un núcleo interactivo, componentes reactivos (notificaciones) y un componente transformacional (el cálculo de compatibilidad), y en identificar como stakeholders de alto interés o alto poder a los dueños de mascotas, los administradores de la plataforma, las organizaciones de adopción, los veterinarios y los patrocinadores.

El artefacto que produjo la PE1 fue, en consecuencia, un mapa de restricciones y necesidades preliminares no todavía requisitos formales, sino los temas y las preguntas que era necesario llevar a la siguiente práctica: qué información sanitaria es relevante para permitir una interacción entre mascotas, qué datos deben protegerse por ser sensibles, y qué papel debe jugar la validación veterinaria en el proceso de emparejamiento. Este mapa es exactamente lo que la PE2 convirtió en una guía de entrevista y en un cuestionario concretos: sin la perspectiva del dominio y la perspectiva de intención definidas en la PE1, la PE2 no habría tenido un criterio explícito para decidir qué preguntar a los propietarios de mascotas ni por qué.

2.2 PE2: elicitación, análisis y priorización

Con el marco de la PE1 como entrada, la PE2 ejecutó la elicitación de requisitos propiamente dicha. La técnica principal fue la **entrevista estructurada**, complementada con un **cuestionario digital** y, en algunos subgrupos, con una sesión de **brainstorming** grupal para ampliar la cobertura de ideas antes de la consolidación. La selección de estas dos técnicas como núcleo del proceso y no una sola respondió a una limitación reconocida por el propio equipo: la entrevista aporta profundidad cualitativa y permite explorar, en un intercambio directo con el entrevistado, matices que un formulario cerrado no captura (por ejemplo, qué información considera privada un propietario y por qué), pero alcanza a pocos participantes y depende de la disponibilidad real de los stakeholders; el cuestionario, en cambio, permite recoger información de un número mayor de usuarios potenciales y cuantificar preferencias mediante escalas Likert, pero no permite repreguntar ni aclarar una respuesta ambigua.

Usar solo una de las dos habría dejado un vacío: limitarse a la entrevista habría producido una muestra pequeña y difícil de generalizar; limitarse al cuestionario habría impedido descubrir necesidades implícitas que los participantes no saben nombrar hasta que se les pregunta directamente. El IREB CPRE FL sustenta esta selección al describir la entrevista y el cuestionario como técnicas complementarias cuya combinación reduce el riesgo de sesgo de una sola fuente [6].

La aplicación concreta a MundiPets es trazable a un acta real. En la entrevista estructurada aplicada a una propietaria de mascota, ante la pregunta de qué información considera privada o sensible sobre su mascota, la entrevistada señaló la ubicación exacta, los datos personales del dueño y los detalles médicos que no deberían mostrarse públicamente sin autorización. Esta respuesta, tomada de forma aislada, no es un requisito; se convirtió en uno solo después de pasar por el análisis de la PE2, que la clasificó como una necesidad de privacidad y

protección de datos y la tradujo en un requisito no funcional de confidencialidad de la información sensible del perfil. La misma entrevista, ante la pregunta sobre qué información es necesaria antes de permitir que una mascota interactúe con otra, arrojó como respuesta la necesidad de conocer vacunas al día, desparasitación, enfermedades recientes y antecedentes de comportamiento agresivo: esa respuesta es el origen directo de los requisitos de validación sanitaria y de compatibilidad que posteriormente se refinaron en la PE3. La cadena evidencia–necesidad–requisito queda así documentada, no supuesta.

Una vez recogidos, los requisitos crudos se sometieron a un proceso de análisis, clasificación (funcional, no funcional, conforme al modelo de calidad de producto de ISO/IEC 25010 [1]) y priorización. Para priorizar se aplicaron dos técnicas complementarias y no una sola, cada una con un propósito distinto. **MoSCoW** clasificó los requisitos en las categorías Debe tener, Debería tener, Podría tener y No tendrá esta vez, y resolvió la pregunta de qué construir primero dado un alcance acotado de tiempo; esta técnica era necesaria porque el equipo identificó más funcionalidades de las que un MVP inicial podía cubrir, y sin un criterio explícito de priorización la selección del alcance habría quedado a discreción de cada integrante.

El **modelo de Kano**, por su parte, no compite con MoSCoW sino que responde una pregunta distinta: no cuánto urge un requisito, sino qué tipo de satisfacción produce en el usuario si se cumple o si falta básico, de desempeño o atractivo. Esta distinción resultó relevante para MundiPets porque algunas funcionalidades, como la validación veterinaria de la información sanitaria, resultaron básicas (su ausencia genera insatisfacción aunque su presencia no sea percibida como un plus), mientras que otras, como el cálculo de compatibilidad asistido por inteligencia artificial, resultaron atractivas (su ausencia no se nota, pero su presencia sí genera valor diferencial). Aplicar solo MoSCoW habría tratado ambos tipos de requisito de forma indistinguible; aplicar solo Kano no habría resuelto el problema operativo de qué construir primero. El artefacto que produjo la PE2 fue la lista depurada y priorizada de requisitos funcionales, no funcionales y de restricción, con su categoría MoSCoW y su clasificación Kano, que sirvió de entrada directa a la especificación formal de la PE3.

2.3 PE3: especificación y modelado

La PE3 tomó los requisitos priorizados de la PE2 y los llevó a una forma verificable y modelada. La primera actividad fue la corrección de la redacción de los requisitos funcionales y no funcionales mediante la **sintaxis EARS** (Easy Approach to Requirements Syntax) [21], que obliga a expresar cada requisito bajo un patrón fijo (universal, condicional, opcional o de estado) en lugar de en lenguaje natural ambiguo. Se eligió EARS porque los requisitos crudos de la PE2, redactados en el lenguaje de la entrevista y el cuestionario, contenían formulaciones no verificables del tipo “el sistema debe ser fácil de usar” que EARS obliga a reescribir con un disparador, una condición y una respuesta observable del sistema. Esta corrección es la que, más adelante, hace posible calcular la métrica de verificabilidad (M3) en la sección de métricas de calidad de este informe: un requisito redactado en EARS es, por construcción, un requisito verificable.

Sobre los requisitos ya corregidos se construyeron tres modelos complementarios. El **modelo de datos** partió de un diagrama entidad–relación que se transformó en un diagrama de clases UML, siguiendo la práctica de traducir las entidades del dominio (mascota, propietario, veterinario, solicitud de adopción o cruce) en clases con sus atributos y relaciones.

El **modelo funcional** combinó diagramas de flujo de datos (DFD) con diagramas de actividades UML para representar cómo se mueve la información a través de los procesos principales del sistema publicar un perfil, evaluar compatibilidad, gestionar una solicitud.

El **modelo de comportamiento** se resolvió con una máquina de estados UML (por ejemplo, para el ciclo de vida de una solicitud de adopción: creada, en revisión, aprobada o rechazada) y con diagramas de secuencia UML que detallan la interacción temporal entre los actores y el sistema para los casos de uso principales. La razón de emplear tres modelos y no uno solo es que cada uno responde una pregunta distinta que ninguno de los otros dos contesta por sí mismo: el modelo de datos responde qué información existe y cómo se relaciona, el modelo funcional responde cómo fluye esa información entre procesos, y el modelo de comportamiento responde en qué orden ocurren los eventos y cómo cambia de estado un objeto del dominio a lo largo del tiempo.

Finalmente, la PE3 cerró con una **matriz de trazabilidad** que integró los tres modelos entre sí y con los requisitos de origen, verificando que cada requisito funcional tuviera al menos un caso de uso, una clase y un elemento del modelo de comportamiento asociado. Esta matriz preliminar es la base directa de la matriz de trazabilidad final que se cierra en la sección de trazabilidad y matriz final de este informe: la PE3 no inventó una estructura nueva para la PE5, sino que extendió la que ya existía.

2.4 PE4: validación mediante inspección formal y gestión de cambios

Con el ERS modelado en la PE3, la PE4 sometió el documento a una **inspección formal de tipo Fagan** [22], reconocida por ISO/IEC/IEEE 29148:2018 entre las técnicas admisibles del proceso de validación de requisitos [3]. Se seleccionó la inspección formal en lugar de una revisión informal porque el método de Fagan exige preparación individual documentada antes de la reunión, roles diferenciados (Moderador, Lector, Inspector 1 e Inspector 2) y un checklist estructurado por dimensión de calidad ambigüedad, completitud, consistencia, verificabilidad y factibilidad, lo que produce una evidencia auditable de que la validación ocurrió y de qué se encontró, en lugar de una declaración de que “se revisó” el documento sin registro verificable.

En el subgrupo de origen de dos de los cinco integrantes actuales del equipo, la inspección se ejecutó sobre el ERS del PFC con dos inspectoras que trabajaron de forma independiente antes de la reunión de consolidación: una desde la perspectiva de consistencia y otra desde las perspectivas de ambigüedad, verificabilidad, completitud y factibilidad. El resultado fue de 33 defectos reportados en total, de los cuales 27 se acreditaron como únicos tras descartar 6 coincidencias de frontera detectadas por ambas inspectoras. De esos 27 defectos, 22 se corrigieron directamente sobre el ERS y quedaron reflejados en la versión 1.1 del documento, mientras que 5 defectos menores permanecieron abiertos y fueron reconocidos como tales en el informe, en lugar de declararse cerrados sin respaldo.

Los hallazgos que excedían una corrección directa por afectar el alcance, la calidad de un requisito no funcional o una restricción normativa se tramitaron mediante **solicitudes de cambio (RFC)** y se sometieron a un **Comité de Control de Cambios (CCB)** simulado, conforme al proceso de gestión de cambios que describe ISO/IEC/IEEE 12207:2017 [8] y que formaliza el PMBOK 7.^a edición [19]: cada RFC se registró, se analizó su impacto y se sometió a una decisión motivada del comité aprobada, aprobada con condiciones o rechazada, dejando acta. Los otros tres subgrupos del equipo ejecutaron el mismo procedimiento sobre sus propias copias del ERS, con resultados cuantitativos distintos pero metodológicamente equivalentes: entre 18 y 23 defectos únicos según el subgrupo, siempre con roles Fagan diferenciados y siempre con al menos tres RFC tramitadas ante un CCB.

El artefacto que produjo la PE4 fue, en cada subgrupo, una versión corregida y con línea base del ERS, acompañada del registro de defectos, del acta de inspección y de las solicitudes de cambio resueltas. Este es el punto en el que el proceso de las cuatro prácticas individuales se detiene: lo que la PE4 entregó no fue todavía el ERS único del equipo de la PE5, sino cuatro ERS paralelos, cada uno validado de forma independiente sobre el mismo dominio de problema.

2.5 PE5: integración, cierre de cambios pendientes y defensa

La PE5 no repite ninguna de las cuatro prácticas anteriores: las integra. El primer paso metodológico del equipo, ya reunido bajo un mismo repositorio, fue reconciliar las cuatro versiones del ERS surgidas de PE1–PE4 en un documento único y consolidar en él los requisitos, el modelo de datos, el modelo funcional, el modelo de comportamiento y la matriz de trazabilidad de los cuatro subgrupos. Como parte de ese cierre, el equipo revisó las solicitudes de cambio que habían quedado pendientes de las cuatro inspecciones de la PE4 y las resolvió formalmente ante un nuevo CCB: nueve RFC fueron decididas sin dejar ninguna abierta, lo que motivó la corrección de varios requisitos funcionales y no funcionales del ERS consolidado y, en los casos en que la evidencia disponible no permitía aceptar el cambio propuesto, el rechazo justificado de la solicitud con su registro correspondiente como limitación de alcance reconocida.

Sobre esa base consolidada y ya sin cambios pendientes, la PE5 añadió lo que ninguna de las cuatro prácticas anteriores había cubierto: la auditoría de calidad del ERS con las seis métricas de la sección de métricas de calidad, el cierre definitivo de la trazabilidad end-to-end de la sección de trazabilidad y matriz final, la especificación de los componentes de inteligencia artificial de la sección de requisitos de IA, y la preparación de la defensa técnica individual ante el tribunal. En ese sentido, la PE5 es menos una sexta técnica que una disciplina de cierre: aplica a la totalidad del proceso anterior los mismos criterios de verificabilidad que EARS exigió a cada requisito individual en la PE3, pero ahora a la escala del proyecto integrador completo.

2.6 Tabla resumen del proceso

La Tabla 1 sintetiza la actividad, la técnica, el artefacto producido y la norma de referencia de cada práctica, en el mismo orden en que se describieron en esta sección.

Tabla 1: Resumen del proceso de Ingeniería de Requisitos aplicado a MundiPets, PE1–PE5.

Práctica	Actividad	Técnica	Artefacto	Norma
PE1	Caracterización del sistema y su contexto	Matriz poder/interés; ocho perspectivas de contexto; declaración de visión (Moore)	Mapa de restricciones y necesidades preliminares	PMBOK 7. ^a ed.; Pohl
PE2	Elicitación, análisis y priorización	Entrevista estructurada; cuestionario digital; brainstorming; MoSCoW; Kano	Lista de requisitos depurada, clasificada y priorizada	ISO/IEC/IEEE 29148:2018; IREB CPRE FL; ISO/IEC 25010:2023
PE3	Especificación y modelado	Sintaxis EARS; modelo de datos (ER/clases UML); modelo funcional (DFD/actividades); modelo de comportamiento (estados/secuencia)	ERS corregido, modelado y con matriz de trazabilidad preliminar	ISO/IEC/IEEE 29148:2018; SWEBOK v4.0
PE4	Validación y gestión de cambios	Inspección formal Fagan; RFC; CCB simulado	ERS corregido, con línea base y registro de defectos	ISO/IEC/IEEE 29148:2018; ISO/IEC/IEEE 12207:2017; PMBOK 7. ^a ed.
PE5	Integración, auditoría y defensa	Consolidación de las cuatro versiones; cierre de RFC pendientes; métricas ISO/IEC 25010; trazabilidad end-to-end; requisitos de IA	ERS final único, matriz de trazabilidad final, informe integrador	ISO/IEC/IEEE 29148:2018; ISO/IEC 25010:2023; Reglamento (UE) 2024/1689

2.7 Fundamento normativo del proceso seguido

El proceso descrito no se apoya en una sola fuente sino en un conjunto de normas y marcos que se complementan sin superponerse. La ISO/IEC/IEEE 29148:2018 encuadra el proceso de punta a punta (elicitación, análisis, especificación, validación y gestión) y es la norma contra la cual se verifica que cada artefacto producido en PE1–PE5 sea, formalmente, parte de un proceso reconocido de Ingeniería de Requisitos y no una secuencia de tareas de curso sin relación entre sí [3]. La ISO/IEC/IEEE 15288:2023 y la ISO/IEC/IEEE 12207:2017 sitúan ese proceso dentro del ciclo de vida del sistema y del software respectivamente, y son las que sustentan formalmente la actividad de gestión de cambios ejecutada en la PE4 y cerrada en la PE5 [7, 8]. Pohl aporta el fundamento conceptual de las actividades que preceden a la especificación formal (elicitación, análisis, negociación) y es la fuente directa de las técnicas aplicadas en la PE1 y en la priorización de la PE2 [4]. El SWEBOK v4.0 enmarca estas actividades dentro del cuerpo de conocimiento reconocido de la ingeniería de software y respalda en particular la práctica de modelado de la PE3 [5]. El IREB CPRE FL v3.3.0 aporta el catálogo verificado de técnicas de elicitación (entrevista, cuestionario, brainstorming) que sustenta la selección hecha en la PE2 [6]. Ninguna de estas fuentes sustituye a las demás: cada una responde a una etapa distinta del mismo proceso, y es esa complementariedad, y no la acumulación de citas, la que permite sostener que las decisiones metodológicas del equipo en PE1–PE5 no fueron arbitrarias.

3 ERS/SRS Final del Sistema MundiPets

3.1 Estructura conforme a ISO/IEC/IEEE 29148:2018

El ERS final de MundiPets se organizó siguiendo la estructura de contenido que ISO/IEC/IEEE 29148:2018 establece para una Especificación de Requisitos de Software [3]. La Tabla 2 muestra la correspondencia explícita entre las cláusulas de la norma y las secciones efectivamente desarrolladas en el documento del equipo; no se transcribe aquí el contenido de cada sección, que se encuentra íntegro en el ERS consolidado del repositorio, sino que se documenta dónde localizarlo y bajo qué cláusula normativa queda encuadrado.

Tabla 2: Correspondencia entre las cláusulas de ISO/IEC/IEEE 29148:2018 y las secciones del ERS de MundiPets.

Cláusula ISO/IEC/IEEE 29148:2018	Sección del ERS de MundiPets
1. Introducción	Sección 1 — Introducción
1.1 Propósito	1.1 Propósito del documento
1.2 Alcance	1.2 Alcance del producto
1.3 Definiciones, acrónimos y abreviaturas	1.3 Definiciones, acrónimos y abreviaturas (glosario)
1.4 Referencias	1.4 Referencias
1.5 Visión general del documento	1.5 Visión general del documento
2. Descripción general	Sección 2 — Descripción general
2.1 Perspectiva del producto	2.1 Perspectiva del producto y límite del sistema (incluye el entorno operativo)
2.2 Funciones del producto	2.2 Funciones del producto
2.3 Características de los usuarios	2.3 Clases y características de usuarios; 2.4 Mapa de stakeholders (matriz poder/interés y conflictos potenciales)
2.4 Restricciones generales	2.5 Entorno operativo (aspectos no cubiertos en 2.1)
2.5 Suposiciones y dependencias	2.6 Suposiciones y dependencias
3. Requisitos específicos	Sección 3 — Requisitos específicos completos
3.1 Requisitos de interfaces externas	3.1 Interfaces externas (usuario, hardware, software)
3.2 Requisitos funcionales	3.2 Requisitos funcionales (RF-01 a RF-27)
3.3 Requisitos de rendimiento	Incorporados dentro de 3.3 Requisitos no funcionales (RNF-02, RNF-03, RNF-12)
3.4 Restricciones de diseño	3.5 Restricciones de diseño (RD-01 a RD-09)
3.5 Atributos del sistema software (calidad)	3.3 Requisitos no funcionales (RNF-01 a RNF-16)
3.6 Otros requisitos (legales, de negocio)	3.4 Requisitos legales
4. Información de apoyo	3.6 Historias de usuario y criterios de aceptación (verificabilidad); Sección 4 — Modelado del sistema con UML; Sección 5 — Priorización y trazabilidad extendida (matriz de trazabilidad)

La correspondencia es completa: ninguna cláusula de contenido que la norma exige para un SRS quedó sin sección equivalente en el ERS de MundiPets. La distinción entre requisitos de rendimiento (cláusula 3.3) y atributos de calidad del software (cláusula 3.5) — que la norma trata como categorías separadas— se resolvió integrando ambas dentro de la sección única de Requisitos no funcionales del documento del equipo, clasificando internamente cada RNF según corresponda a rendimiento, seguridad, usabilidad, mantenibilidad u otro atributo de calidad; esta es una decisión de organización del equipo, no una omisión, y se declara aquí de forma explícita por la misma razón que exige la verificabilidad de un requisito individual: que la trazabilidad entre lo declarado y lo exigido sea comprobable por cualquier lector, incluido el tribunal.

El ERS del equipo contiene, además de las cláusulas normativas anteriores, dos bloques de contenido que exceden el mínimo que exige ISO/IEC/IEEE 29148:2018: la Sección 6, Producto Mínimo Viable (alcance del MVP, arquitectura de despliegue y evidencia de funcionamiento), y la Sección 7, Componente empírico del proyecto (protocolo de validación experimental, preguntas de investigación en formato PICOC, hipótesis, variables, participantes, instrumentos, plan de análisis estadístico y registro previo del protocolo). Ninguno de estos dos bloques corresponde a una cláusula de la norma y, por lo tanto, no se fuerza su inclusión en la Tabla 2: son contenido adicional que el equipo decidió mantener como valor agregado del proyecto integrador, más allá de la estructura mínima que un SRS conforme a 29148 exige.

3.2 Requisitos clave y cambios incorporados en PE1–PE5

La Tabla 3 documenta la trazabilidad de cambios de los requisitos del sistema para los que el equipo cuenta con evidencia completa y verificable del ciclo defecto–corrección: redacción original, defecto detectado en la inspección de la PE4, corrección aplicada y estado final. Estos cinco requisitos fueron objeto de una solicitud de cambio (RFC) formal ante el Comité de Control de Cambios, tramitada durante la PE4 y resuelta de forma definitiva al cierre de la PE5; dos de ellos — RF-06 y RNF-02— corresponden a la categoría *Debe tener* (Must) de la priorización MoSCoW; los tres restantes — RF-07, RF-10 y RNF-03— corresponden a *Podría tener* (Could) y *Debería tener* (Should) respectivamente, pero se incluyen porque son los únicos casos, junto a los dos anteriores, en los que el equipo puede exhibir la redacción exacta antes y después de la corrección, y no solo declarar que un cambio ocurrió.

Tabla 3: Requisitos con trazabilidad de cambio completa (redacción original, defecto, corrección) incorporados durante PE1–PE5.

ID	Cambio aplicado y motivo	Práctica
RF-06 (Must)	<i>Original:</i> la salida del requisito era información comparativa general para evaluar compatibilidad, junto con una advertencia de que el resultado era orientativo. <i>Defecto (RFC-I, PE4):</i> el propio ERS exigía en RNF-06 y RNF-13 una explicación de la recomendación que RF-06 no declaraba producir — el requisito no era verificable contra sus propios atributos de calidad. <i>Corrección (cierre PE5):</i> la salida se redefinió como un veredicto de compatibilidad en tres niveles discretos (compatible, compatible con reservas, no recomendado), acompañado de la explicación exigida por RNF-13. <i>Estado final:</i> corregido y verificado; cubierto en el MVP mediante heurística explicable.	PE2 (elicitación) → PE4 (defecto vía RFC-I) → PE5 (corrección e integración)
RF-07 (Could)	<i>Original:</i> el requisito permitía la comunicación entre propietarios e interesados mediante mensajes, sin mecanismo de reporte de abuso. <i>Defecto (RFC-F, PE4):</i> el propio ERS reconocía el conflicto de “comunicación inadecuada” entre usuarios sin que ningún requisito funcional lo resolviera. <i>Corrección (cierre PE5):</i> se incorporó la posibilidad de reportar un mensaje o interacción como spam, lenguaje ofensivo o intento de acuerdo externo indebido, con registro del motivo asociado a la conversación; esto obligó además a redefinir RNF-15 (RFC-H) para que la explicabilidad de la moderación fuera coherente con este nuevo mecanismo de reporte manual y no con una moderación automática que el sistema no implementa. <i>Estado final:</i> corregido y verificado.	PE2 (elicitación) → PE4 (defecto vía RFC-F) → PE5 (corrección e integración)
RF-10 (Should)	<i>Original:</i> el requisito generaba recordatorios por notificación en la aplicación y WhatsApp, sin definir qué canal prevalecía si el otro fallaba. <i>Defecto (RFC-C, PE4):</i> la restricción de diseño RD-08 exigía un canal de respaldo ante caída del proveedor externo de mensajería, pero RF-10 no lo formalizaba. <i>Corrección (cierre PE5):</i> se redefinió WhatsApp como canal principal y la notificación in-app como respaldo automático cuando el proveedor externo no está disponible, con su correspondiente criterio de verificación de conmutación de canal. <i>Estado final:</i> corregido y verificado; cubierto en el MVP mediante simulación del canal de respaldo.	PE2 (elicitación) → PE4 (defecto vía RFC-C) → PE5 (corrección e integración)
RNF-02 (Must)	<i>Original:</i> exigía una disponibilidad mínima del 99 % mensual sin acotar el entorno de medición. <i>Defecto (RFC-E, PE4):</i> el MVP local del equipo no permite verificar disponibilidad mensual, al no operar en un ambiente de uso continuo. <i>Corrección (cierre PE5):</i> se acotó explícitamente el criterio al entorno de producción con backend desplegado, declarando que no es exigible ni medible sobre el MVP local. <i>Estado final:</i> corregido y verificado.	PE3 (especificación) → PE4 (defecto vía RFC-E) → PE5 (corrección e integración)
RNF-03 (Should)	<i>Original:</i> exigía un tiempo de respuesta máximo de 2 segundos en el percentil 95 con 100 usuarios concurrentes, sin acotar el entorno de medición. <i>Defecto (RFC-G, PE4):</i> el umbral no era verificable con la infraestructura disponible para el equipo; la propuesta original de relajar el umbral fue rechazada por el CCB de cierre de la PE5, que prefirió acotar el alcance en vez de bajar la exigencia. <i>Corrección (cierre PE5):</i> se acotó el criterio al entorno de producción, igual que RNF-02, sin modificar el valor objetivo de 2 segundos. <i>Estado final:</i> corregido y verificado.	PE3 (especificación) → PE4 (defecto vía RFC-G) → PE5 (corrección e integración, rechazo de la propuesta original)

Además de estos cinco, la revisión de calidad de requisitos funcionales realizada en la Entrega 4 (versión 4.0 del ERS) corrigió por ambigüedad, duplicidad o inconsistencia otros seis requisitos de la categoría Must — RF-02, RF-05, RF-12, RF-17, RF-18 y RF-23—, según consta en el historial de versiones del documento. El equipo no reconstruye aquí la redacción original de cada uno de estos seis por no contar con el registro de defecto individual con el que sí cuenta para los cinco de la Tabla 3: declarar un “antes” sin ese respaldo sería exactamente el tipo de evidencia no verificable que la sección de integridad académica de la rúbrica sanciona con mayor severidad. Lo que sí es verificable, y se declara como tal, es que esa corrección de calidad ocurrió y está registrada en el propio ERS como parte del cierre normal de la Entrega 4.

La existencia de esta tabla es en sí misma la evidencia de que el ERS entregado en la PE5 es la versión final integrada del documento y no una versión previa a las correcciones: cada fila muestra un requisito que en algún punto de PE1–PE4 tuvo un defecto real, registrado en un acta de inspección o en una RFC, y que llegó a la PE5 ya corregido y verificado contra la versión vigente del ERS (v4.0).

3.3 Control de versión, identificadores y línea base

3.3.1 Identificación de la versión final

El ERS final del sistema MundiPets corresponde a la **versión 4.0**, congelada el **30 de agosto de 2026**, asociada al commit `8b1970d` del repositorio compartido del PFC — el estado vigente del repositorio al momento de redactar esta subsección—, que incluye tanto el cierre formal de las 9 RFC pendientes de la inspección de la PE4 (Tabla 3) como la actualización más reciente de los modelos UML y de las evidencias de campo del componente empírico (EV-23 y EV-24). Este commit describe el estado del ERS a la fecha, pero no es todavía la línea base etiquetada del proyecto integrador completo: esa etiqueta se reserva, de forma deliberada, para el cierre final de la PE5 (véase la fila correspondiente en la Tabla 4). La Tabla 4 resume esta identificación.

Tabla 4: Identificación de la versión final del ERS.

Campo	Valor
Versión del ERS	4.0
Fecha de congelación	30/08/2026
Commit asociado (hash corto)	8b1970d
Etiqueta de línea base en Git	v5.0-PE5-baseline
Responsable de la consolidación	Gutierrez Ortega Génesis Adriana

3.3.2 Verificación de unicidad de identificadores

Se verificó, sobre el ERS v4.0 vigente, que ninguna de las series de identificadores del documento presente duplicados ni saltos numéricos. La verificación se realizó de forma automatizada sobre el propio archivo fuente del ERS, contrastando cada identificador declarado contra su definición única en una subsección o fila de tabla correspondiente. La Tabla 5 resume el resultado por serie.

Tabla 5: Verificación de unicidad y continuidad de identificadores del ERS.

Serie	Rango verificado	Duplicados	Observación
RF-XX	RF-01 a RF-27	Ninguno	Secuencia continua, sin saltos; 27 requisitos funcionales, cada uno con una única subsección de definición.
RNF-XX	RNF-01 a RNF-16	Ninguno	Secuencia continua, sin saltos; 16 requisitos no funcionales.
RL-XX	RL-01 a RL-09	Ninguno	Secuencia continua, sin saltos; 9 requisitos legales.
RD-XX	RD-01 a RD-09	Ninguno	Secuencia continua, sin saltos; 9 restricciones de diseño, cada una con una única subsección de definición.
CU-XX	CU-01 a CU-13	Ninguno	Secuencia continua, sin saltos; 13 casos de uso. Cada identificador se reutiliza de forma consistente — no duplicada— en sus tres vistas de modelado (especificación textual, diagrama de secuencia y diagrama de actividad), lo que corresponde al mismo caso de uso documentado desde tres perspectivas y no a una asignación repetida del identificador a contenido distinto.
RF-IA-XX	—	No aplica en esta etapa	Esta serie corresponde a los requisitos de los componentes de inteligencia artificial, que se especifican en la sección de Requisitos de IA de este informe y se incorporan a la matriz de trazabilidad final una vez consolidados; a la fecha de cierre de esta subsección, la serie aún no existe en el ERS base y por tanto no aplica una verificación de unicidad.

No se detectó ningún identificador duplicado en las series verificables a la fecha de esta subsección. La integración del ERS a través de PE1–PE5 mantuvo la disciplina de asignación de identificadores exigida por ISO/IEC/IEEE 29148:2018 [3].

4 Modelos UML del Sistema

Esta sección presenta los modelos vigentes de la PE3 ya corregidos, con una lectura interpretativa de la decisión de diseño que refleja cada uno, no una descripción de lo que se observa en el dibujo. El orden sigue el recorrido natural del sistema: primero los modelos generales de alcance y arquitectura (casos de uso, contexto, componentes, despliegue) y los modelos estratégicos i* (SD y SR); luego el comportamiento dinámico (máquinas de estado, secuencia, actividad); y finalmente la estructura estática (clases).

4.1 Diagrama de Casos de Uso

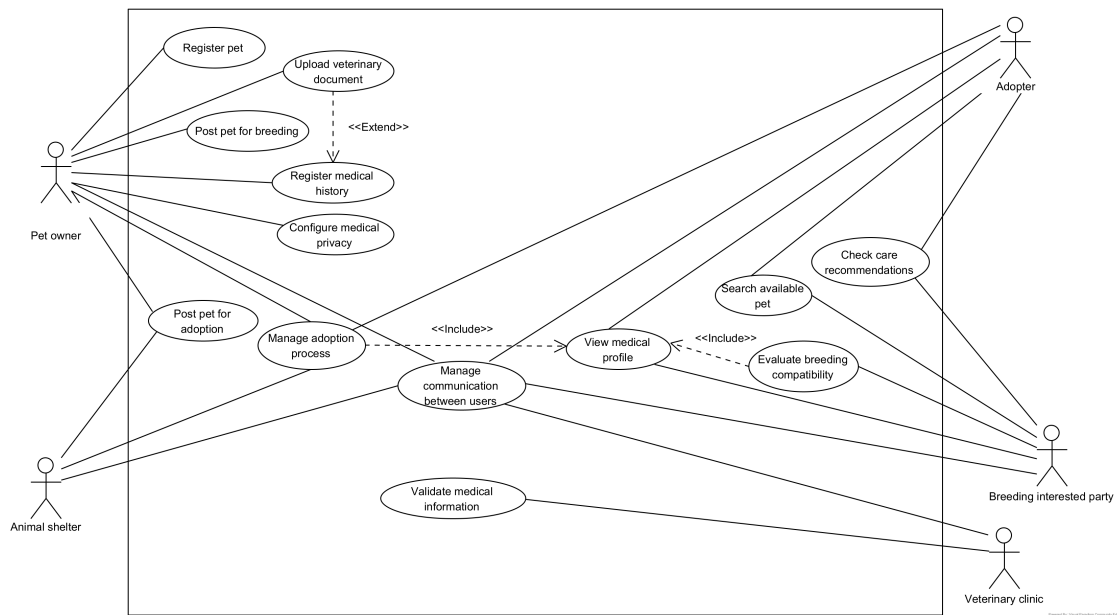


Figura 1: Diagrama de casos de uso general del sistema MundiPets.

En la Figura 1, la inclusión obligatoria de “Manage adoption process” y “Manage communication between users” hacia “View medical profile” refleja que ningún adoptante o interesado en cruce puede avanzar en una negociación sin pasar primero por el historial médico de la mascota: la transparencia sanitaria no es opcional, es un paso obligado del flujo. Por otro lado, “Upload veterinary document” extiende a “Register medical history” (no lo incluye), lo que indica que adjuntar el documento formal es un enriquecimiento posterior y no una condición para registrar el historial básico, permitiendo que el dueño registre datos médicos mínimos, aunque aún no tenga el certificado en mano.

4.2 Diagrama de Contexto

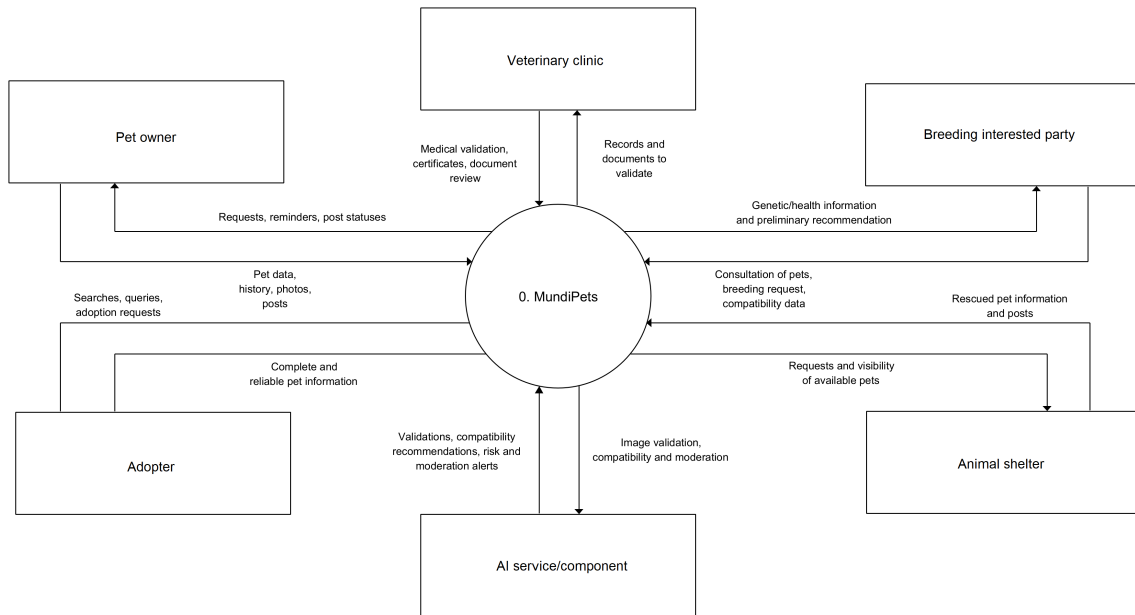


Figura 2: Diagrama de contexto del sistema MundiPets.

Como se observa en la Figura 2, la existencia de un actor externo independiente “AI service/component”, con flujo bidireccional propio hacia el núcleo “0. MundiPets” (envía moderación de imágenes y compatibilidad, recibe validaciones), refleja la decisión de tratar la inteligencia artificial como un límite del sistema desacoplado del resto de actores humanos, y no como una función interna invisible. Esto anticipa que los componentes de IA (Sección ??) tienen contratos de entrada/salida propios y auditables, no una lógica difusa mezclada con el resto del sistema.

4.3 Diagrama de Componentes

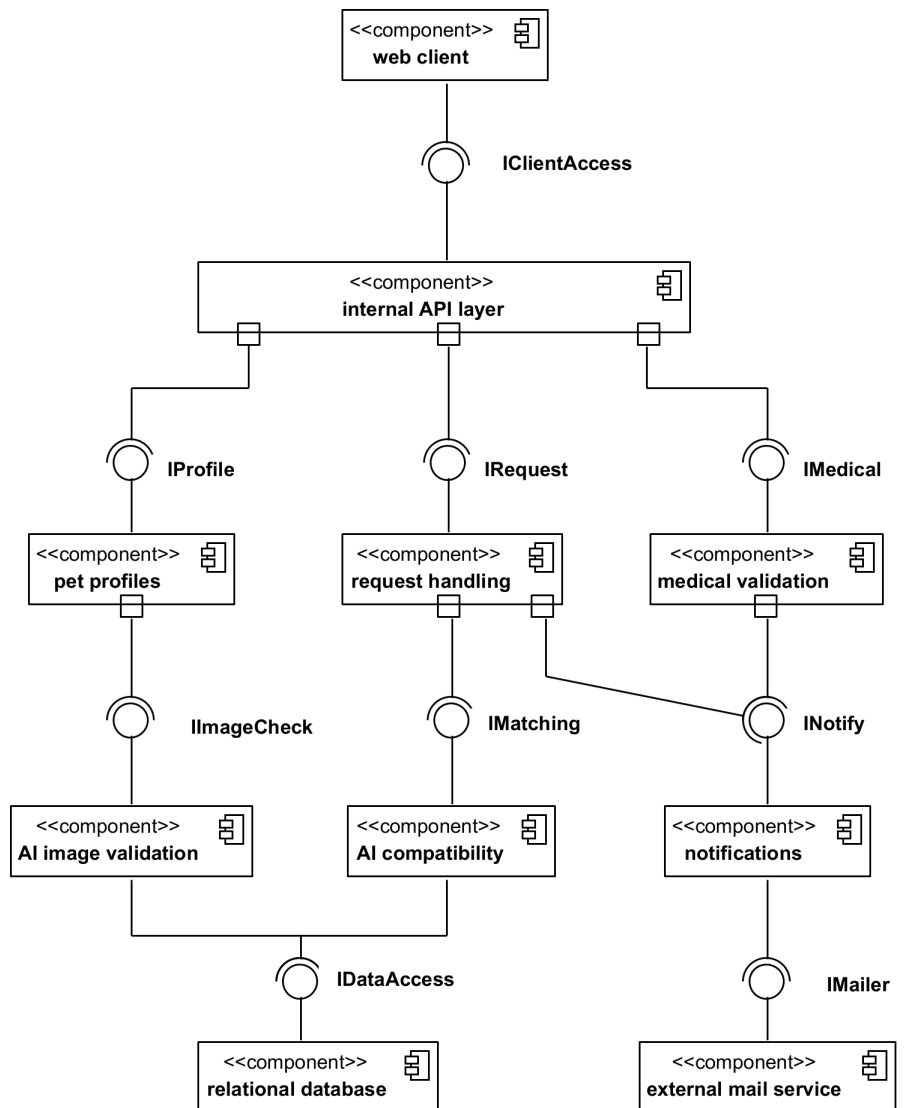


Figura 3: Diagrama de componentes del sistema MundiPets.

La Figura 3 muestra que la separación entre “AI image validation” y “AI compatibility” como componentes independientes que exponen sus propias interfaces (*IImageCheck*, *IMatching*) hacia “request handling” y “pet profiles”, en lugar de integrarse directamente en la lógica de negocio, refleja la decisión de aislar los modelos de IA del resto del sistema para poder reemplazarlos, reentrenarlos o degradarlos (*fallback*) sin tocar la capa de API. Además, que “IRequest” comparta la interfaz “INotify” con “notifications” (en vez de que cada componente notifique por su cuenta) muestra que las notificaciones se centralizaron como un servicio transversal y no como lógica duplicada en cada módulo.

4.4 Diagrama de Despliegue

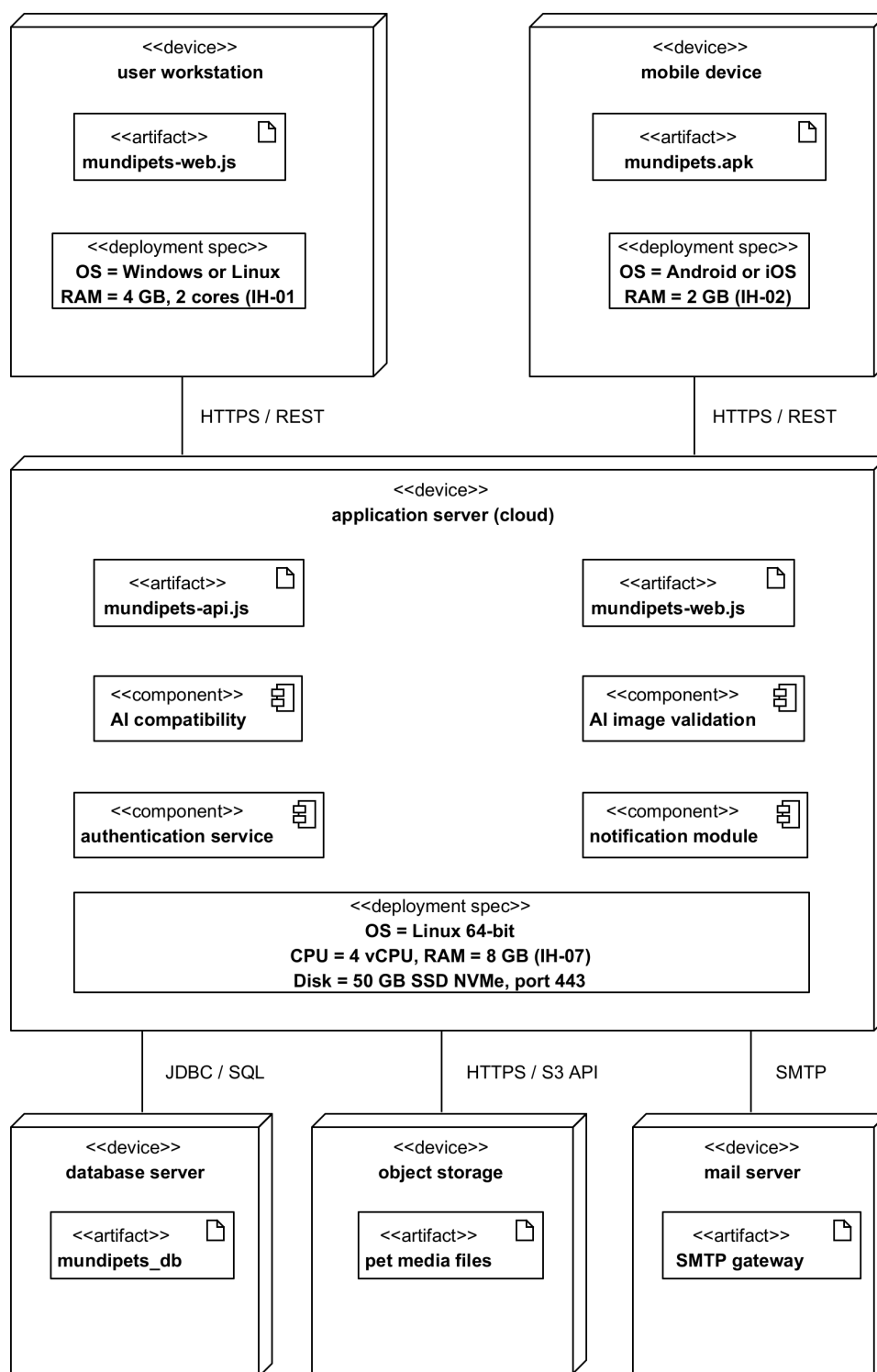


Figura 4: Diagrama de despliegue del sistema MundiPets.

Que los componentes “AI compatibility” y “AI image validation” se desplieguen dentro del mismo nodo “application server (cloud)” junto con la autenticación y las notificaciones (y no en un nodo de cómputo dedicado o servicio externo separado), como se aprecia en la Figura 4, refleja que el equipo optó por una arquitectura monolítica de despliegue para el MVP, priorizando simplicidad operativa sobre escalabilidad independiente de la IA, decisión coherente con el nivel de riesgo y volumen de datos que maneja MundiPets en esta etapa.

4.5 Diagramas Estratégicos (i*)

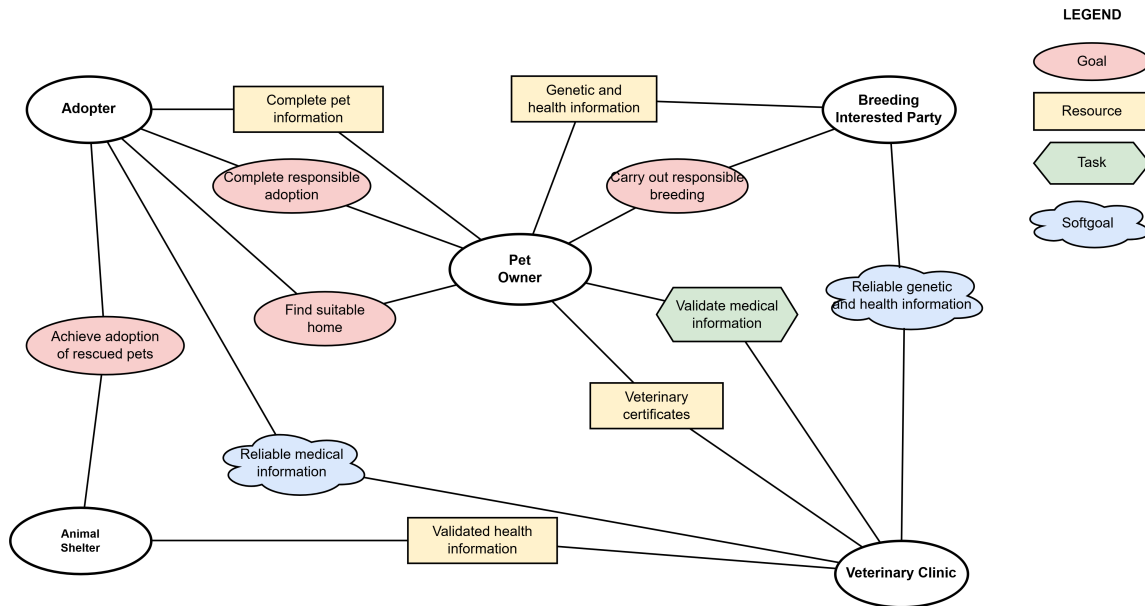


Figura 5: Diagrama de dependencia estratégica (SD) del sistema MundiPets.

En la Figura 5, que “Pet Owner” dependa de “Veterinary Clinic” para la tarea “Validate medical information” (una tarea, no un simple recurso) revela que la validación médica no es un intercambio de datos pasivo, sino un proceso activo que el dueño no puede realizar por sí mismo ni delegar a otro actor: la clínica es la única fuente legítima de verificación, lo que justifica por qué el ERS exige trazabilidad hacia un actor externo verificador y no solo un campo de texto autodeclarado.

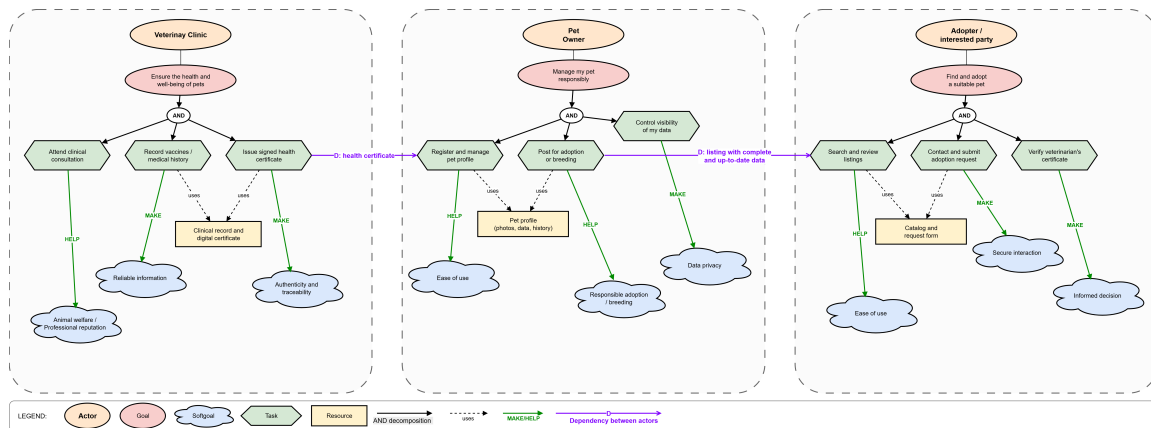


Figura 6: Diagrama de razón estratégica (SR) del sistema MundiPets.

Dentro del racional del “Pet Owner” en la Figura 6, que “Post for adoption or breeding” dependa del recurso “Pet profile (photos, data, history)” mediante *uses* (y no que ambas tareas generen su propio perfil por separado) refleja la decisión de que el perfil de la mascota es un artefacto único y compartido entre los flujos de adopción y de cruce, evitando duplicidad de datos y asegurando que cualquier corrección al perfil (por ejemplo, del historial médico) se propague automáticamente a ambos procesos.

4.6 Diagrama de Flujo de Datos (Nivel 1)

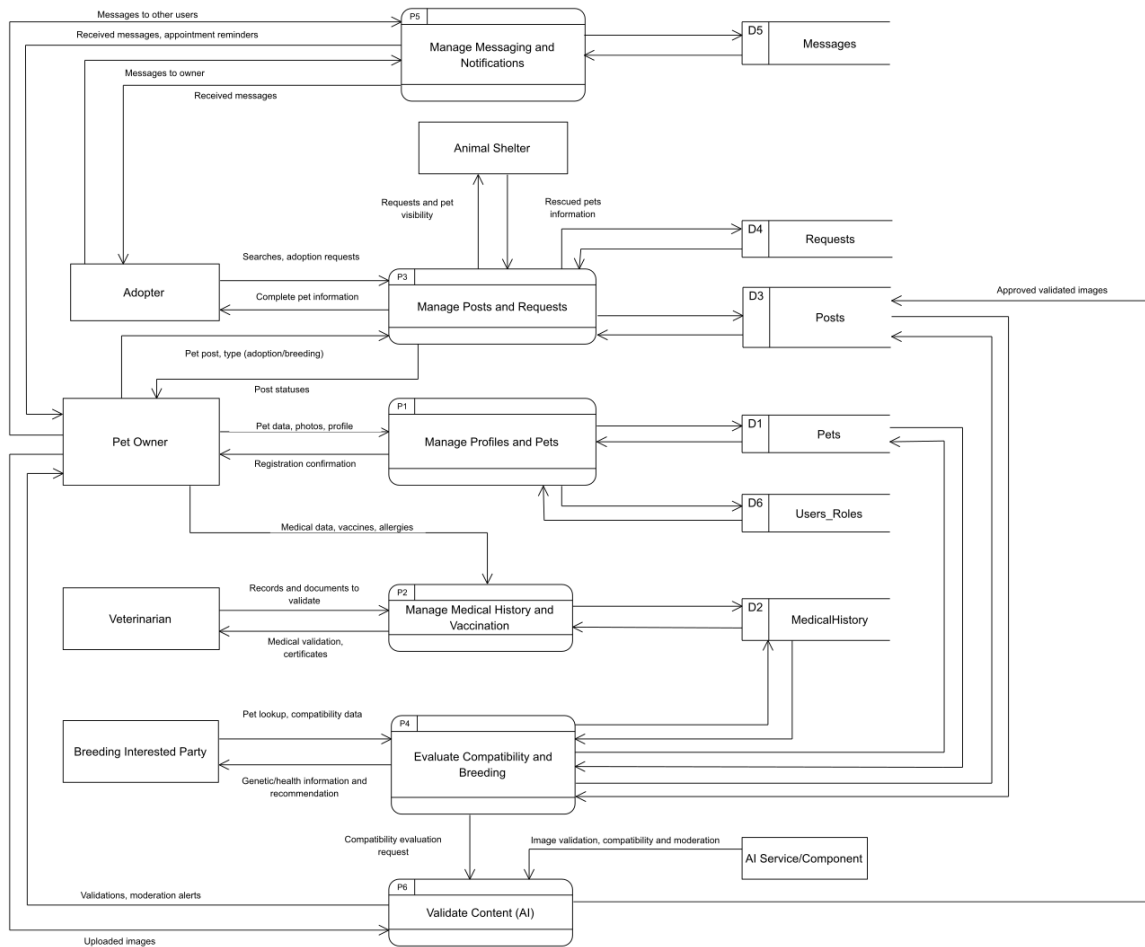


Figura 7: Diagrama de flujo de datos, Nivel 1, notación Yourdon-DeMarco.

Que el proceso único “0. MundiPets” del Diagrama de Contexto (Figura 2, que funciona como DFD Nivel 0) se descomponga en exactamente seis procesos (P1 Manage Profiles and Pets, P2 Manage Medical History and Vaccination, P3 Manage Posts and Requests, P4 Evaluate Compatibility and Breeding, P5 Manage Messaging and Notifications, P6 Validate Content) revela que el equipo agrupó la funcionalidad del sistema por dominio de datos compartido y no por actor o por caso de uso individual: P2, por ejemplo, concentra tanto el registro del historial médico (RF-02) como su validación profesional (RF-09) y la trazabilidad de vacunas (RF-14, RF-24), porque las tres operan sobre el mismo almacén D2 (MedicalHistory), aunque correspondan a actores distintos (propietario y veterinaria). Que P6 (“Validate Content”) se mantenga como proceso independiente, con su propio flujo de entrada y salida hacia el “AI Service/Component” ya visto en el Diagrama de Contexto, refuerza la misma decisión de aislamiento de la IA observada en el Diagrama de Componentes: el flujo de datos de la validación automática nunca se mezcla con el flujo de datos de negocio de los demás procesos, sino que pasa siempre por un almacén y un proceso propios.

4.7 Diagramas de Máquinas de Estado

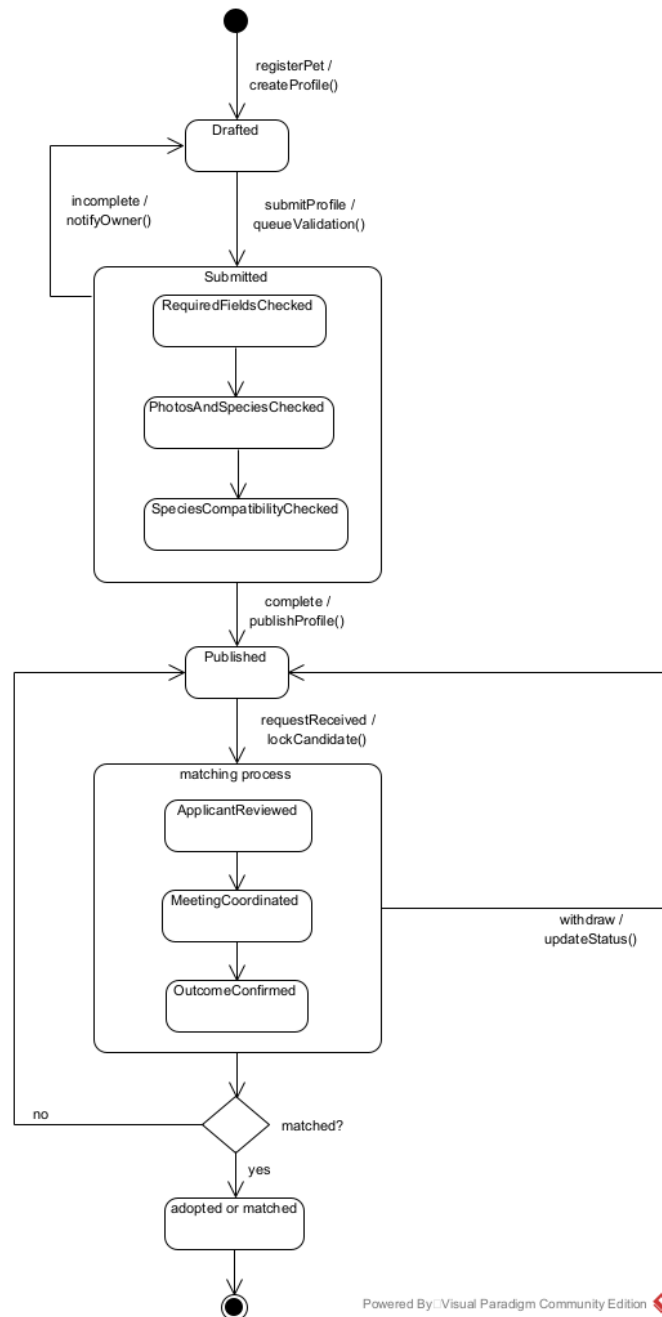


Figura 8: Diagrama de máquinas de estado – Perfil Mascota.

Que el estado “Submitted” sea un subestado compuesto con una secuencia interna obligatoria (*RequiredFieldsChecked* → *PhotosAndSpeciesChecked* → *SpeciesCompatibilityChecked*) antes de poder llegar a “Published”, como se ve en la Figura 8, revela que la publicación de una mascota nunca es un evento atómico, sino el resultado de una validación en capas donde ningún criterio puede saltarse: si falta un campo, el sistema regresa a “Drafted” y notifica al dueño en vez de dejar el perfil a medio camino. Asimismo, que “matched?” sea una decisión posterior a todo el subestado “matching process” (y no una simple bandera booleana temprana) muestra que el sistema exige agotar la revisión del solicitante, coordinación de encuentro y confirmación del resultado antes de decidir si hubo adopción o cruce exitosa, evitando cierres prematuros del caso.

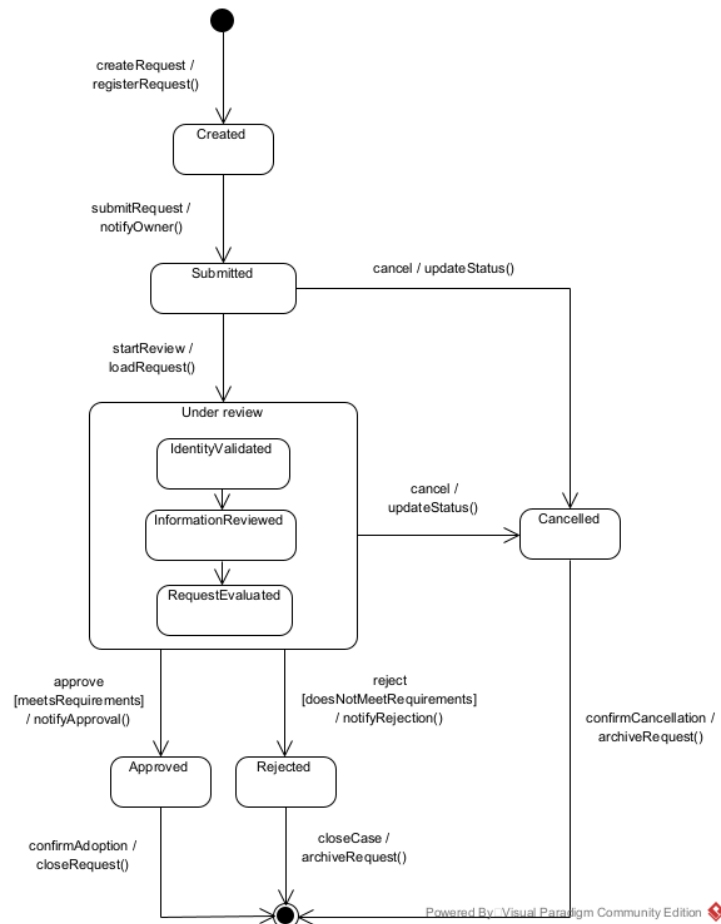


Figura 9: Diagrama de máquinas de estado – Solicitudes.

Que “cancel” esté habilitado tanto desde “Submitted” como desde dentro de “Under review” (dos transiciones distintas hacia “Cancelled”, no una sola desde el estado padre), tal como se muestra en la Figura 9, refleja que el diseño reconoce que una solicitud puede ser retirada por el interesado en cualquier punto del proceso, incluso mientras ya se está validando la identidad o evaluando la solicitud, y no únicamente antes de empezar la revisión. Además, que “Approved” y “Rejected” conduzcan a acciones finales distintas (*confirmAdoption* vs. *closeCase*, ambas con *archiveRequest()*) en lugar de una única salida genérica, indica que el cierre de una adopción concretada exige un paso adicional de confirmación que el cierre de un rechazo no necesita, correspondiendo a una responsabilidad distinta según el desenlace.

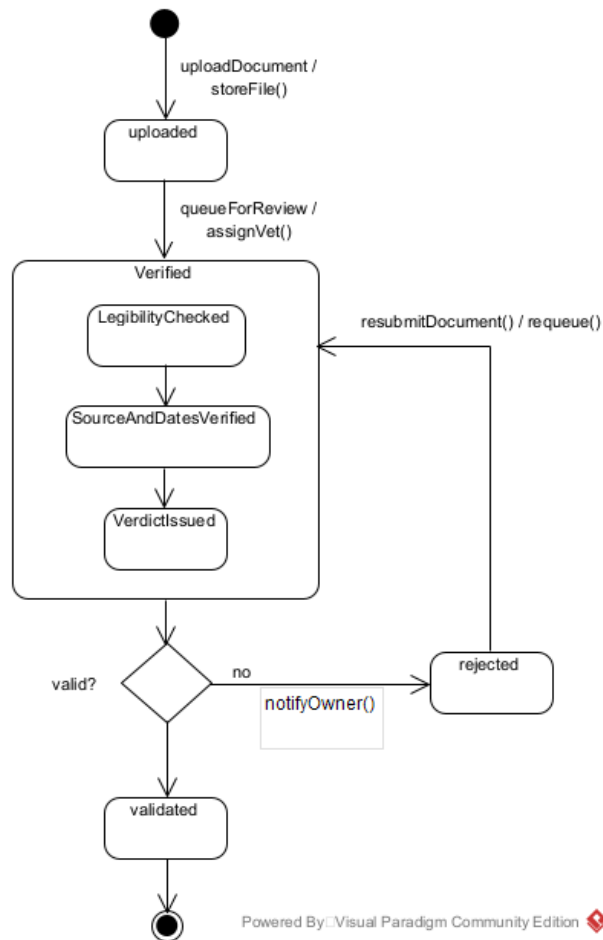


Figura 10: Diagrama de máquinas de estado – Verificación de Documentos.

Que *resubmitDocument()/requeue()* regrese exclusivamente desde “rejected” hacia el subestado compuesto “Verified” (y no hacia “uploaded”), como se aprecia en la Figura 10, revela que un documento rechazado no reinicia el ciclo completo de carga, sino que retorna directamente a la cola de revisión una vez corregido, evitando que el dueño tenga que subir el archivo desde cero. También, que la verificación interna se descomponga en tres pasos secuenciales (legibilidad → verificación de fuente y fechas → emisión de veredicto) antes de la decisión “valid?” muestra que el sistema separa expresamente la validez formal del documento (que se lea bien) de su validez de contenido (que la fuente y fechas sean correctas), permitiendo auditar en qué paso específico falló un documento rechazado.

4.8 Diagramas de Secuencia

4.8.1 Registrar mascota (CU-01)

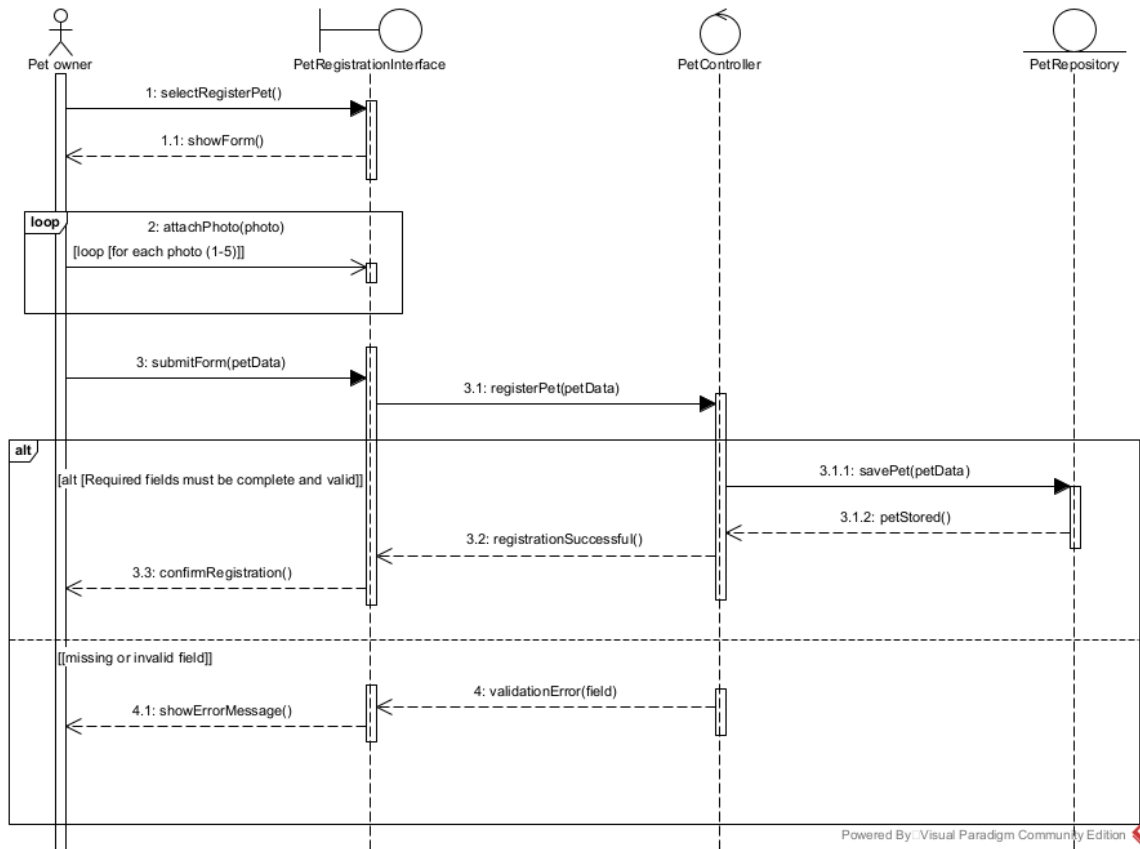


Figura 11: Diagrama de secuencia – Registrar mascota (CU-01).

En la Figura 11, que *attachPhoto(photo)* se ejecute en un *loop* previo al *submitForm(petData)* (y no como parte del mismo envío) revela que el diseño separa la carga incremental de fotos de la validación final del formulario, permitiendo que el dueño adjunte entre 1 y 5 fotos de forma progresiva antes de comprometerse a enviar el registro completo. Además, que la validación de campos ocurra en el *PetController* y no en la interfaz (el *alt* evalúa “Required fields must be complete and valid” a nivel de controlador) muestra que la lógica de negocio, no la capa de presentación, es la responsable de decidir si el registro es válido.

4.8.2 Registrar historial médico y antecedentes genéticos (CU-02)

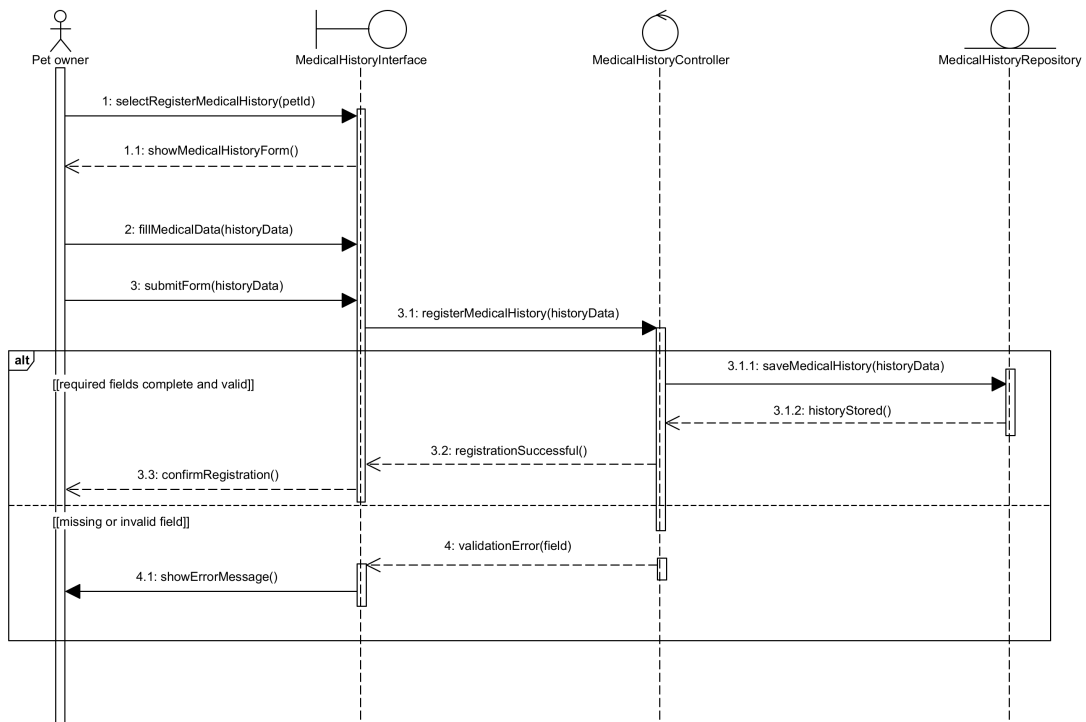


Figura 12: Diagrama de secuencia – Registrar historial médico y antecedentes genéticos (CU-02).

La estructura casi idéntica al CU-01 (mismo patrón de *alt* con validación en el controlador), visible en la Figura 12, refleja una decisión consciente de reutilizar el mismo patrón arquitectónico de registro para todo dato del sistema, de forma que el equipo pueda mantener consistencia entre módulos distintos (mascota vs. historial médico) sin duplicar convenciones. Esto evidencia que el historial médico se trata como una entidad de registro independiente y no como un campo más del perfil de la mascota.

4.8.3 Cargar documento veterinario y carnet de vacunación (CU-03)

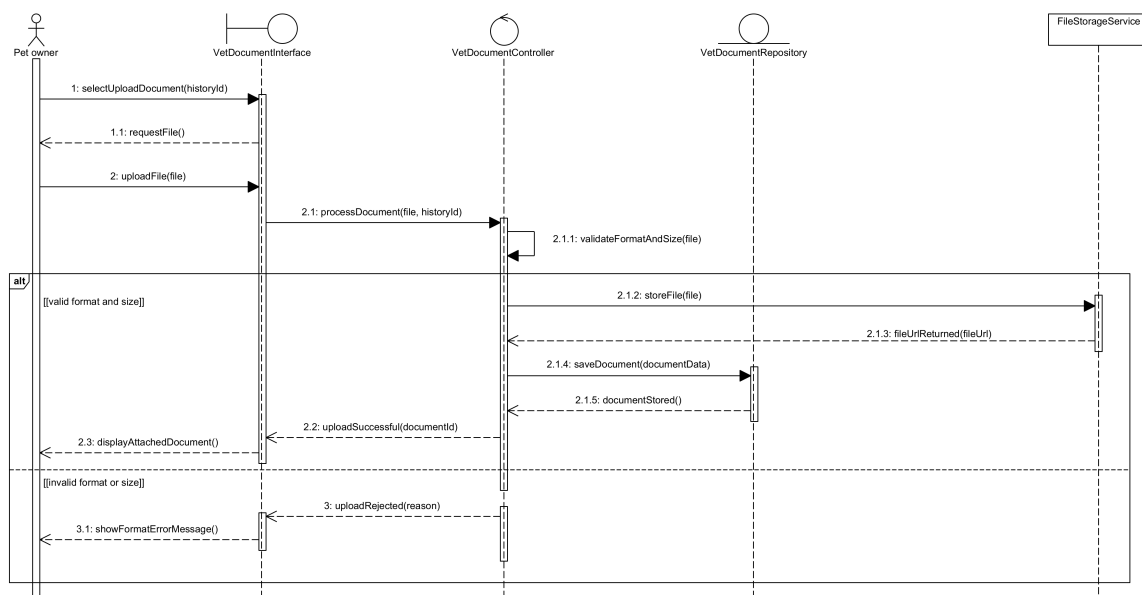


Figura 13: Diagrama de secuencia – Cargar documento veterinario y carnet de vacunación (CU-03).

Que *validateFormatAndSize(file)* sea una llamada reflexiva del propio *VetDocumentController* (mensaje a sí mismo) antes de invocar al *FileStorageService*, como se ve en la Figura 13, revela que el sistema nunca envía

un archivo a almacenamiento externo sin antes garantizar localmente que cumple formato y tamaño, evitando cargas costosas o inseguras a un servicio de terceros. También, que *storeFile* preceda a *saveDocument* (primero el archivo físico, luego el registro en base de datos) muestra que el diseño prioriza tener la evidencia física respaldada antes de dejar constancia lógica de que el documento existe.

4.8.4 Publicar mascota en adopción (CU-04)

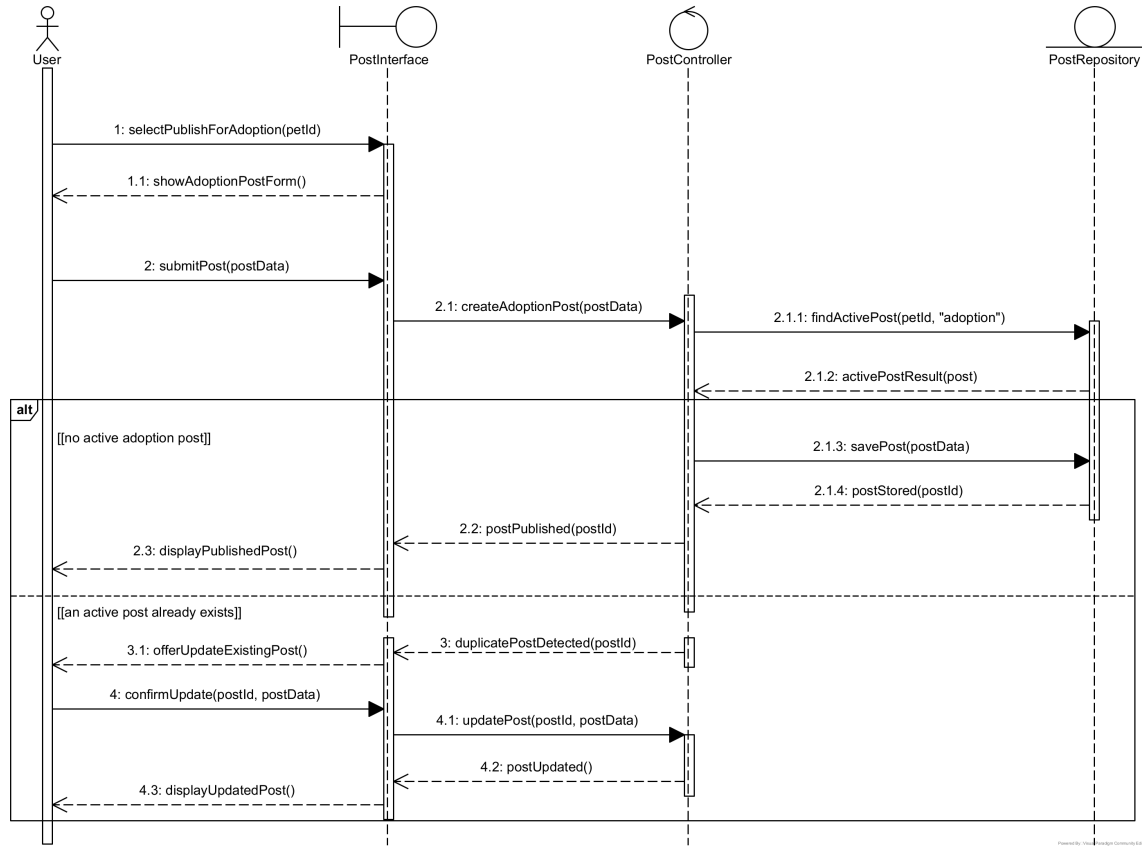


Figura 14: Diagrama de secuencia – Publicar mascota en adopción (CU-04).

La verificación *findActivePost(petId, "adoption")* antes de decidir entre crear o actualizar, presente en la Figura 14, revela que el sistema previene activamente publicaciones duplicadas de una misma mascota, tratando la unicidad de la publicación activa como una regla de negocio explícita y no como una validación de interfaz. Que el flujo de “an active post already exists” ofrezca actualizar en lugar de simplemente rechazar el nuevo intento (*offerUpdateExistingPost*) muestra que el diseño prioriza la conveniencia del usuario sobre el rechazo estricto.

4.8.5 Publicar mascota para cruce y gestionar solicitud (CU-05)

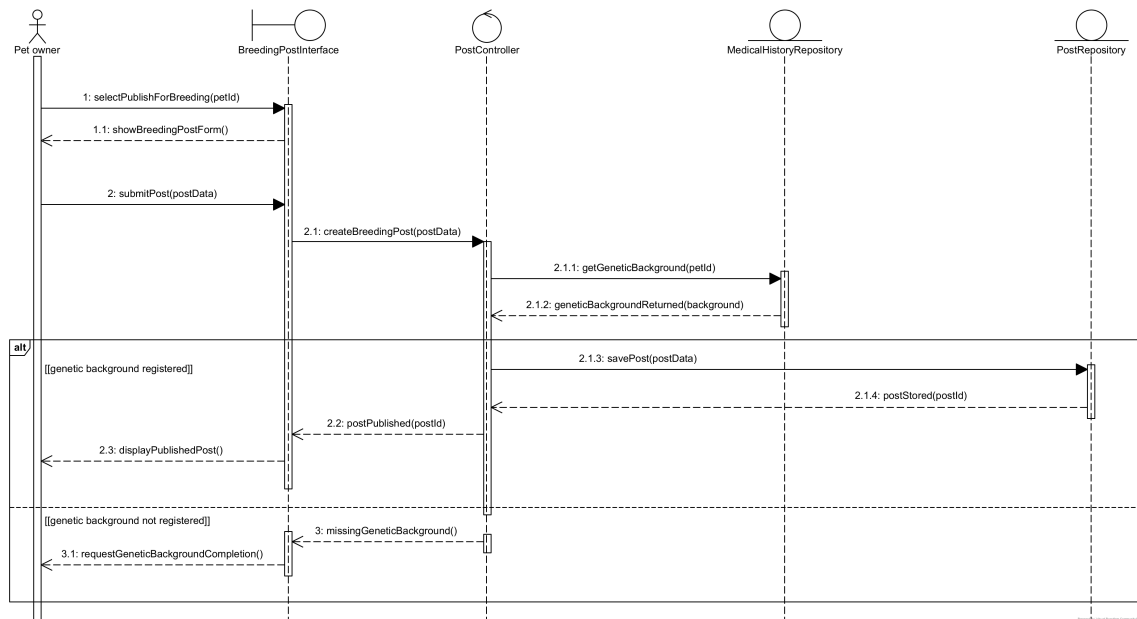


Figura 15: Diagrama de secuencia – Publicar mascota para cruce y gestionar solicitud (CU-05).

Que *getGeneticBackground(petId)* se consulte obligatoriamente antes de poder publicar (y bloquee la publicación en el camino “genetic background not registered”), como se observa en la Figura 15, revela que la trazabilidad genética no es un campo opcional del formulario, sino una precondition dura para la cruce, materializando directamente el requisito de “responsabilidad en la cruce” declarado en el alcance de MundiPets. Esta dependencia obligatoria contrasta con el CU-04 (adopción), donde no existe tal barrera, mostrando que ambos procesos de publicación tienen niveles de exigencia distintos según el riesgo asociado.

4.8.6 Configurar privacidad médica (CU-06)

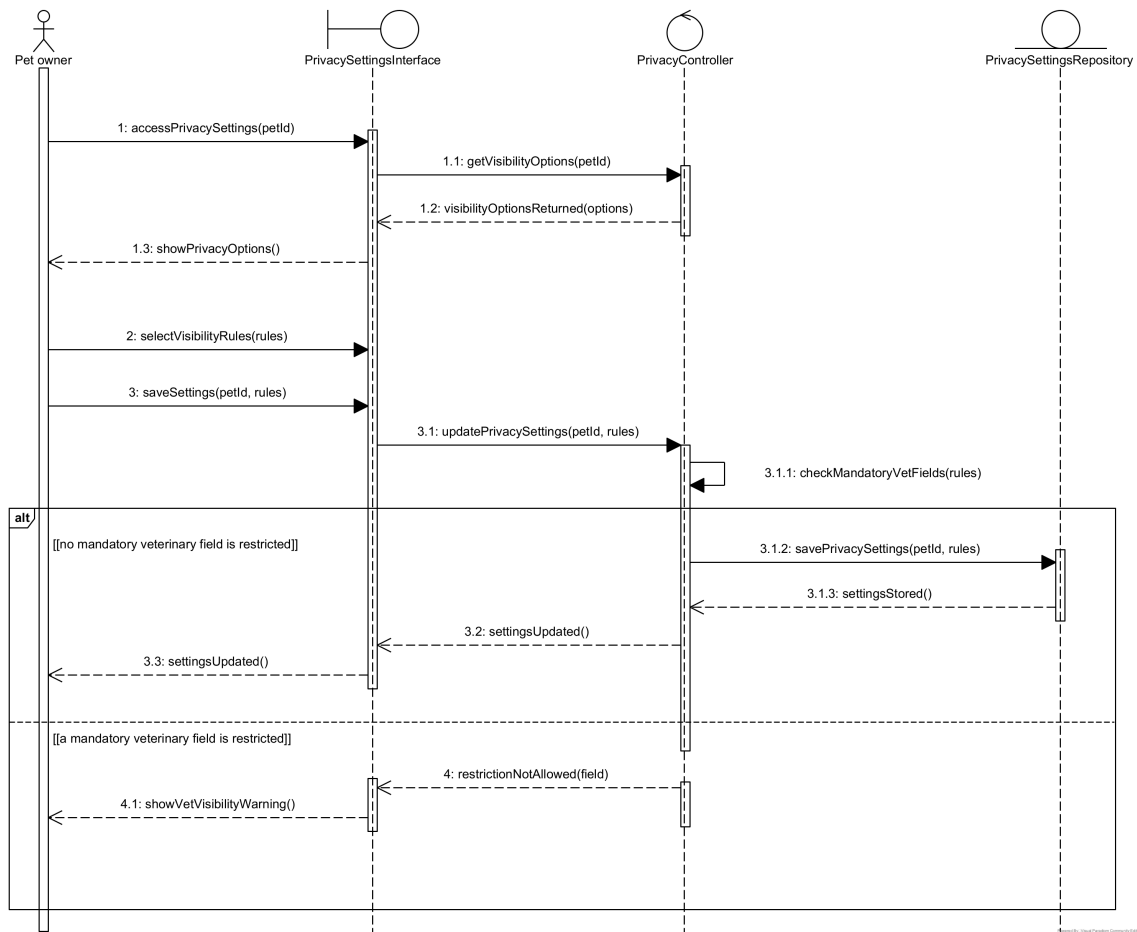


Figura 16: Diagrama de secuencia – Configurar privacidad médica (CU-06).

Que *checkMandatoryVetFields(rules)* sea una validación reflexiva del controlador que puede rechazar la solicitud del dueño (“a mandatory veterinary field is restricted”), tal como se ve en la Figura 16, revela que la privacidad configurable por el usuario tiene un límite impuesto por el sistema: el dueño puede ocultar información no crítica, pero no puede restringir campos que la clínica veterinaria necesita ver para validar. Esto demuestra que la privacidad del dato médico no es absoluta, sino que está subordinada a la seguridad clínica del proceso.

4.8.7 Gestionar proceso de adopción (CU-07)

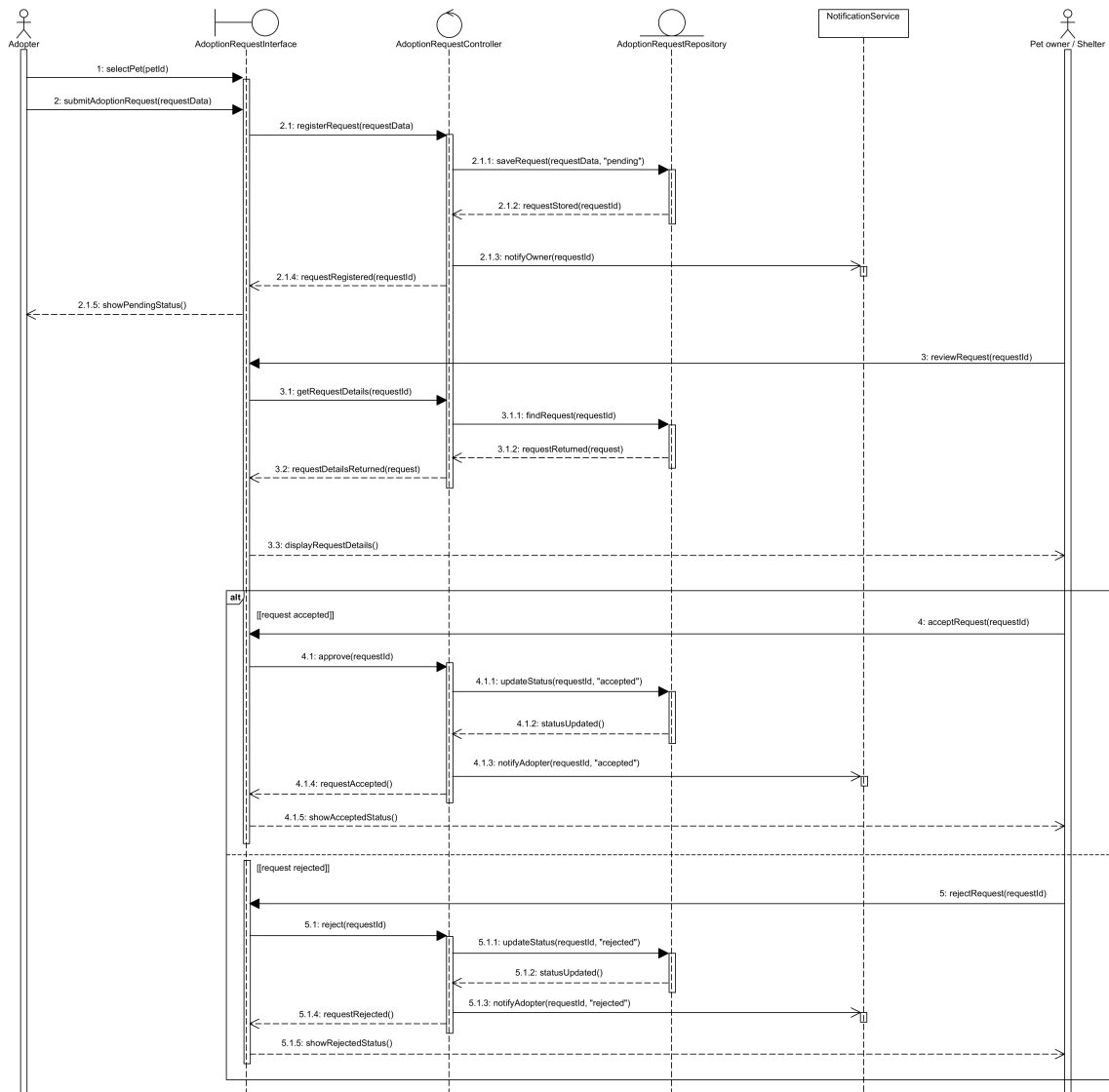


Figura 17: Diagrama de secuencia – Gestionar proceso de adopción (CU-07).

Que *notifyOwner/notifyAdopter* se disparen en cada transición de estado (registro, aceptación, rechazo) a través de un *NotificationService* externo y compartido, en lugar de que cada actor consulte manualmente el estado, según se ve en la Figura 17, revela que el proceso de adopción se diseñó como orientado a eventos: ningún cambio de estado ocurre en silencio. Que el *alt* solo contemple “accepted” o “rejected” como desenlaces (sin una rama de “cancelado” en este mismo diagrama) muestra que la cancelación se modela por separado (coherente con la máquina de estados de Solicitudes, Figura 9), evitando que este diagrama de secuencia intente cubrir cada excepción del ciclo de vida completo.

4.8.8 Gestionar comunicación entre usuarios (CU-08)

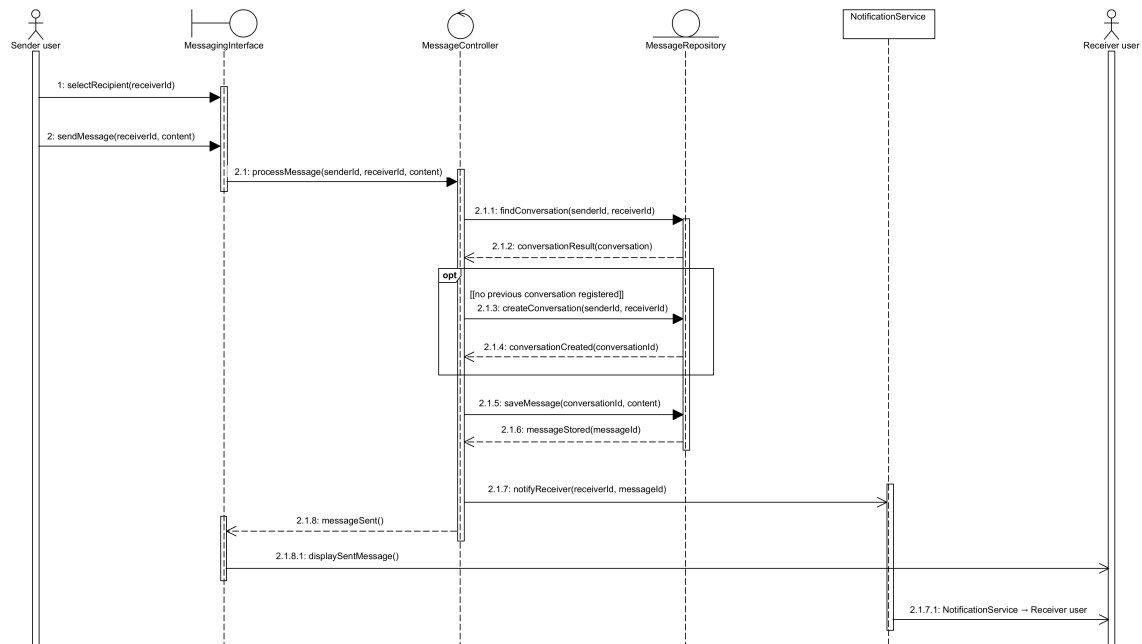


Figura 18: Diagrama de secuencia – Gestionar comunicación entre usuarios (CU-08).

Que *createConversation* esté envuelto en un bloque *opt* condicionado a “no previous conversation registered” (y no se ejecute siempre), como se observa en la Figura 18, revela que el sistema modela la conversación como una entidad persistente y reutilizable entre dos usuarios, evitando fragmentar el historial de mensajes en hilos nuevos cada vez que alguien escribe. Que *notifyReceiver* ocurra después de *messageStored* (nunca antes) muestra que el sistema garantiza la persistencia del mensaje como condición previa a cualquier notificación, evitando alertar de algo que aún podría fallar al guardarse.

4.8.9 Validar información médica (CU-09)

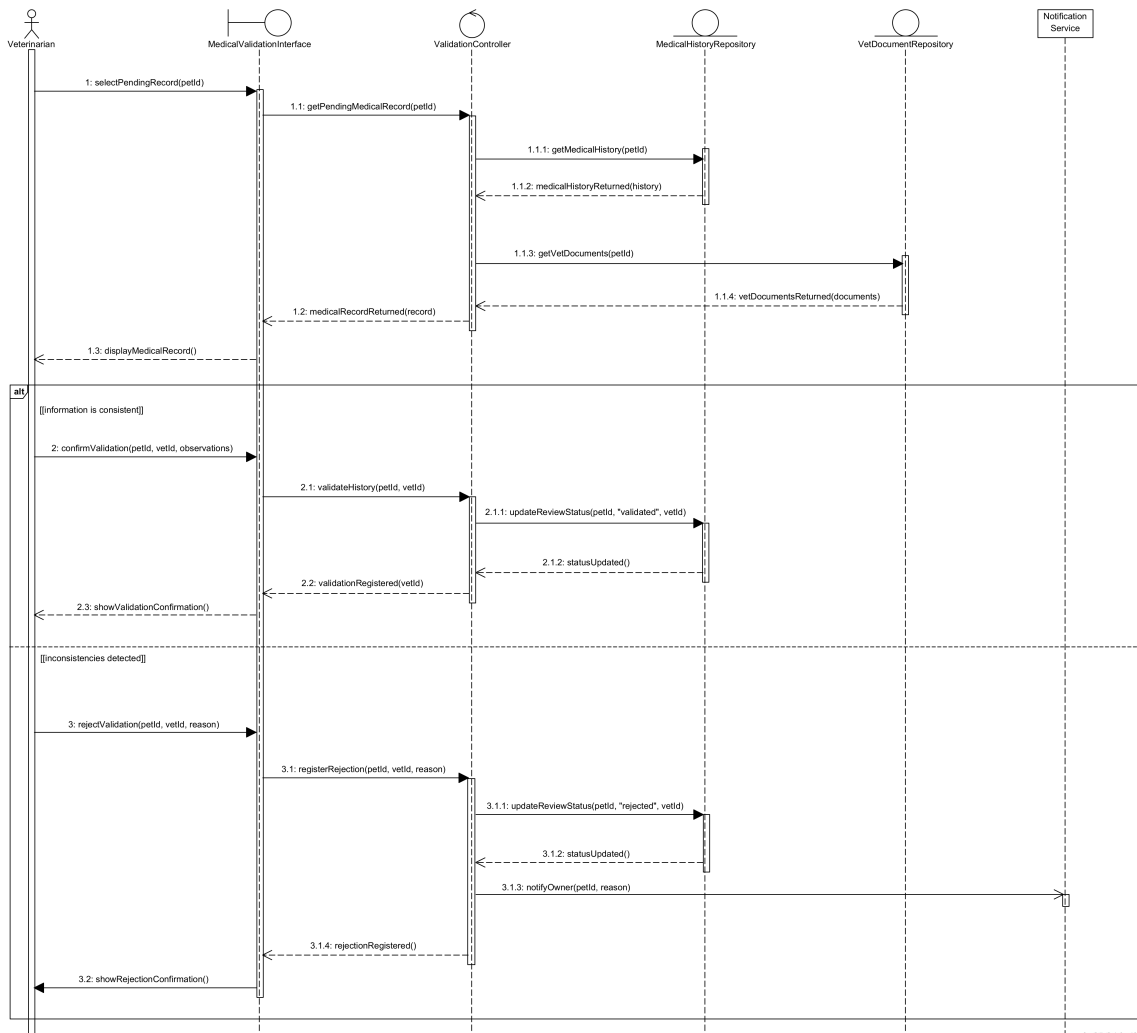


Figura 19: Diagrama de secuencia – Validar información médica (CU-09).

Que el veterinario reciba tanto *getMedicalHistory* como *getVetDocuments* en paralelo desde dos repositorios distintos antes de emitir cualquier veredicto, tal como se ve en la Figura 19, revela que la validación clínica exige cruzar la información declarada por el dueño con el documento formal adjunto, y no aceptar ninguna de las dos fuentes de forma aislada. Que el *alt* solo ofrezca “validated” o “rejected” (sin estado intermedio) y que el rechazo dispare *notifyOwner(petId, reason)* con motivo explícito, refleja que el sistema exige trazabilidad del porqué de un rechazo, no solo el hecho de que ocurrió, coherente con la máquina de estados de Verificación de Documentos (Figura 10).

4.8.10 Consultar perfil médico (CU-10)

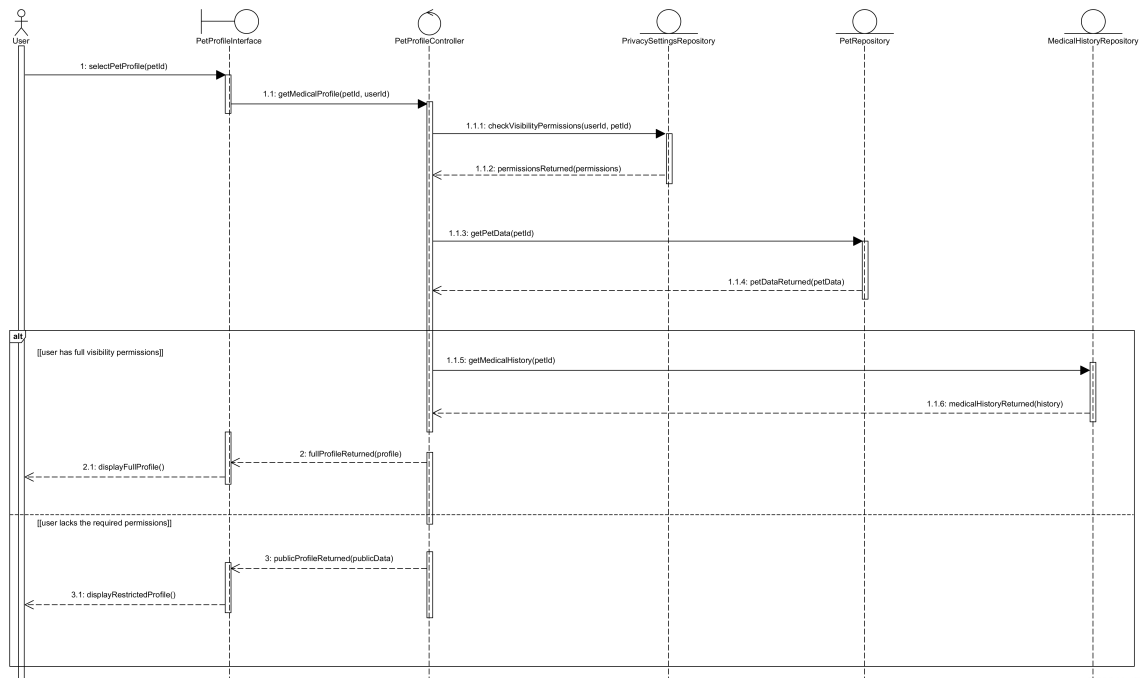


Figura 20: Diagrama de secuencia – Consultar perfil médico (CU-10).

Que *checkVisibilityPermissions(userId, petId)* se ejecute antes de decidir si se llama a *getMedicalHistory* (el *alt* separa “full visibility permissions” de “lacks the required permissions”), como se observa en la Figura 20, revela que el perfil médico no es un dato binario visible/oculto, sino que el propio controlador construye una respuesta distinta según el solicitante: la información sensible ni siquiera se consulta si el usuario no tiene permiso, en vez de consultarla y luego filtrarla. Esto materializa directamente el CU-06 (Figura 16): ambos diagramas comparten la misma fuente de reglas (*PrivacySettingsRepository*).

4.8.11 Buscar mascota disponible (CU-11)

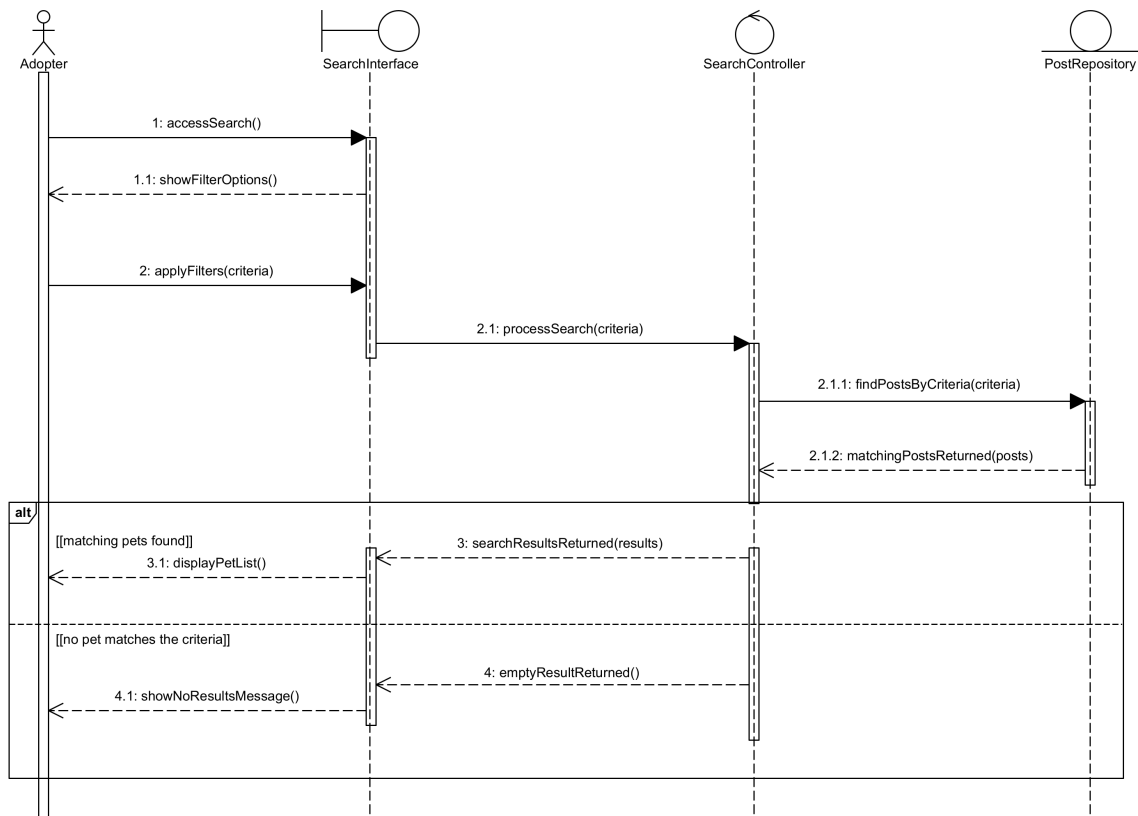


Figura 21: Diagrama de secuencia – Buscar mascota disponible (CU-11).

Que el *alt* distinga entre “matching pets found” y “no pet matches the criteria” como dos resultados igualmente válidos de una búsqueda (y no un error), tal como se ve en la Figura 21, revela que el sistema trata la ausencia de resultados como un estado de negocio esperado y comunicable (*showNoResultsMessage*), no como una excepción a capturar. La simplicidad del flujo (sin bloque de validación de errores de entrada) muestra que este caso de uso confía en que los filtros de búsqueda siempre producen criterios sintácticamente válidos, delegando esa validación a la capa de interfaz.

4.8.12 Consultar recomendaciones de cuidado (CU-12)

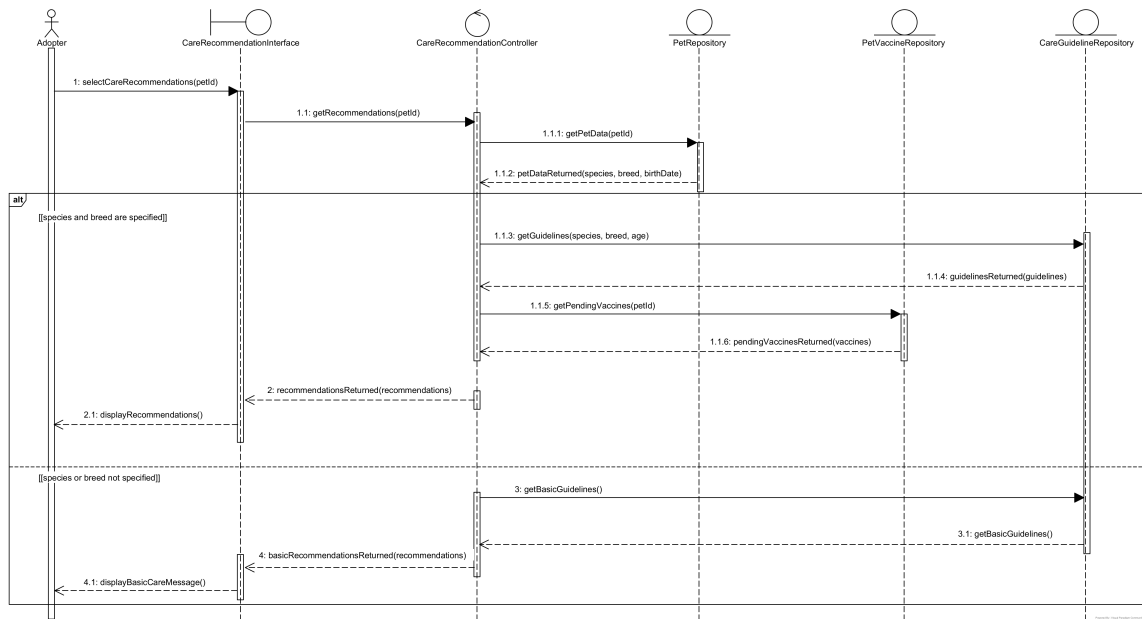


Figura 22: Diagrama de secuencia – Consultar recomendaciones de cuidado (CU-12).

Que el *alt* dependa de si “species and breed are specified” y no de si el usuario las proporcionó explícitamente en este momento (los datos ya vienen de *getPetData*), como se observa en la Figura 22, revela que las recomendaciones de cuidado dependen de la calidad de los datos previamente registrados del perfil, no de una nueva entrada del adoptante: si el perfil está incompleto, el sistema degrada a *getBasicGuidelines()* en lugar de bloquear la función. Esta decisión de tener un camino de *fallback* con recomendaciones genéricas, en vez de negar el servicio, refleja que la utilidad de la funcionalidad se prioriza sobre la exhaustividad de los datos.

4.8.13 Evaluar compatibilidad de cruce (CU-13)

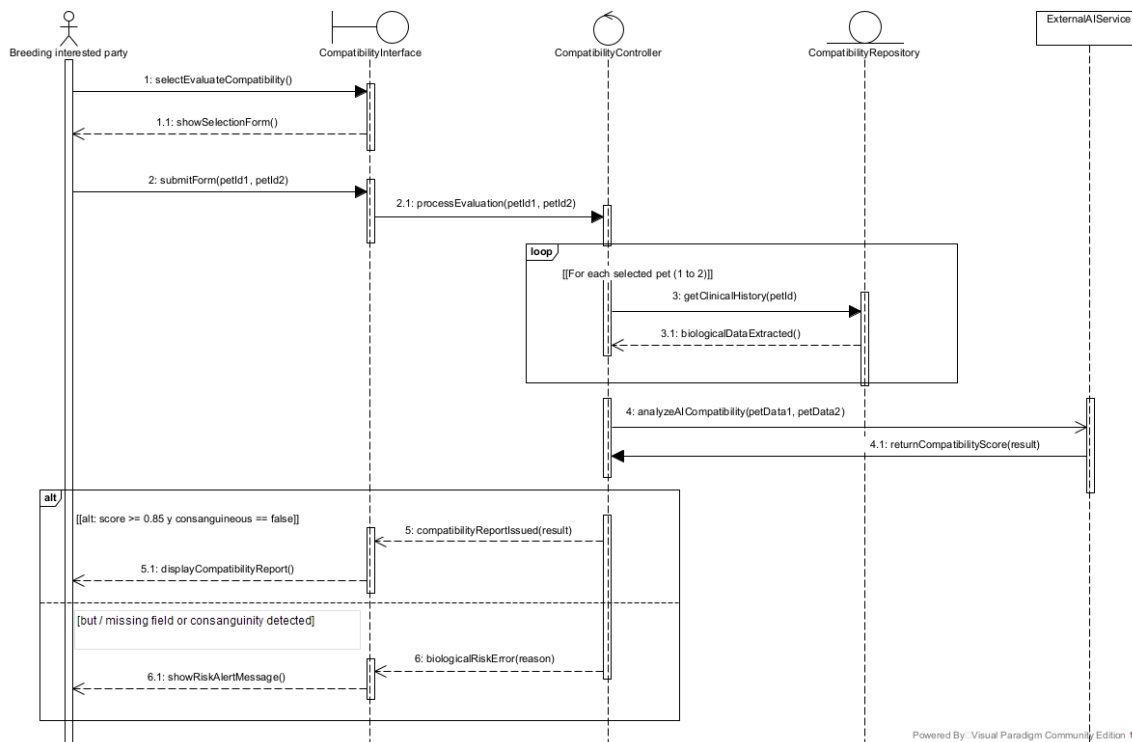


Figura 23: Diagrama de secuencia – Evaluar compatibilidad de cruce (CU-13).

Que el *loop* recorra los historiales clínicos de ambas mascotas (1 a 2) antes de invocar al *ExternalAIService*, y que la condición del *alt* combine dos criterios distintos (*score* ≥ 0.85 y *consanguineous* $== false$) en vez de basarse solo en el puntaje de la IA, tal como se ve en la Figura 23, revela que la recomendación de un componente probabilístico nunca es la única barrera de decisión: existe una regla determinista de consanguinidad que puede anular un buen puntaje de compatibilidad. Esto es la aplicación directa del límite declarado en la Sección ?? (“no reemplaza la evaluación veterinaria profesional”): la IA aporta una señal, pero una regla de negocio dura puede vetarla.

4.9 Diagramas de Actividad

4.9.1 Registrar mascota (CU-01)

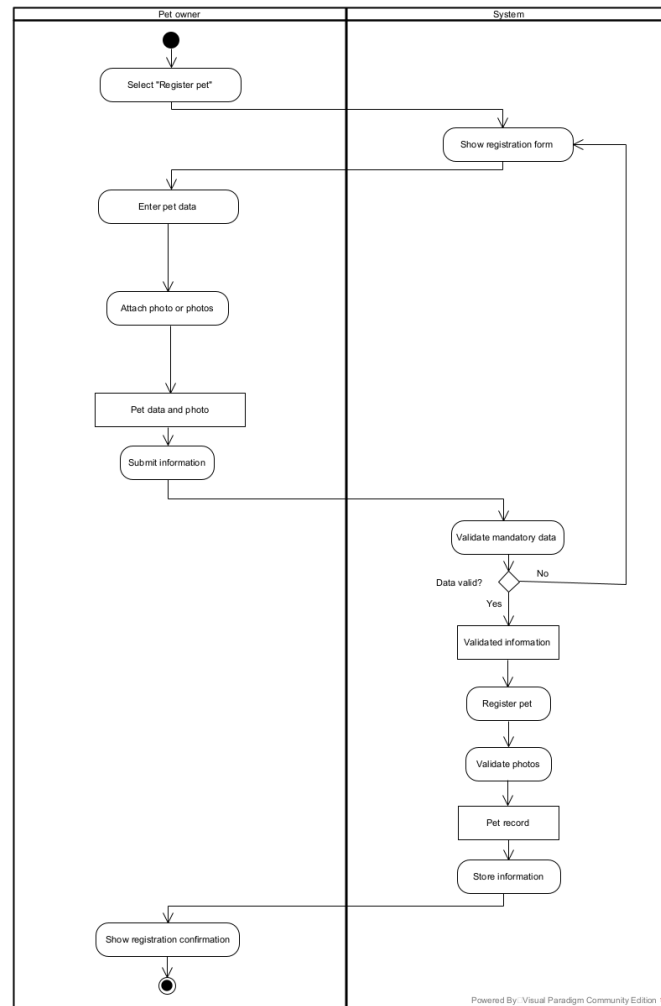


Figura 24: Diagrama de actividades – Registrar mascota (CU-01).

Que la validación “Data valid?” con salida “No” regrese al inicio del carril *System* (“Show registration form”) y no simplemente muestre un mensaje de error puntual, como se ve en la Figura 24, revela que el sistema fuerza al dueño a repetir el ciclo completo de llenado en lugar de corregir solo el campo fallido, priorizando la simplicidad de implementación sobre la fricción del usuario en esta primera versión del flujo.

4.9.2 Registrar historial médico y antecedentes genéticos (CU-02)

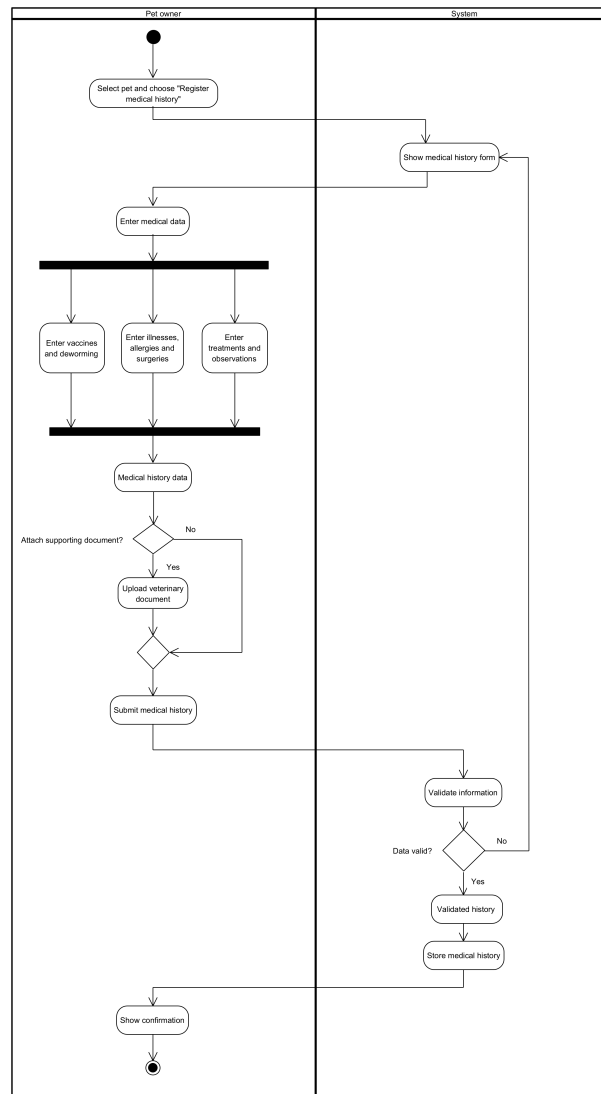


Figura 25: Diagrama de actividades – Registrar historial médico y antecedentes genéticos (CU-02).

Que “Enter vaccines and deworming”, “Enter illnesses...” y “Enter treatments...” se ejecuten en una bifurcación paralela (*fork/join*) y no en secuencia, según se observa en la Figura 25, revela que estas tres categorías de datos médicos son independientes entre sí y no existe un orden obligatorio de captura, permitiendo que el dueño complete primero la que tenga más a mano. Que “Attach supporting document?” sea una decisión opcional embebida dentro de este mismo caso de uso (y no delegada por completo al CU-03) muestra que subir el documento formal es una posibilidad conveniente durante el registro, aunque el CU-03 exista como flujo independiente para hacerlo después.

4.9.3 Cargar documento veterinario y carnet de vacunación (CU-03)

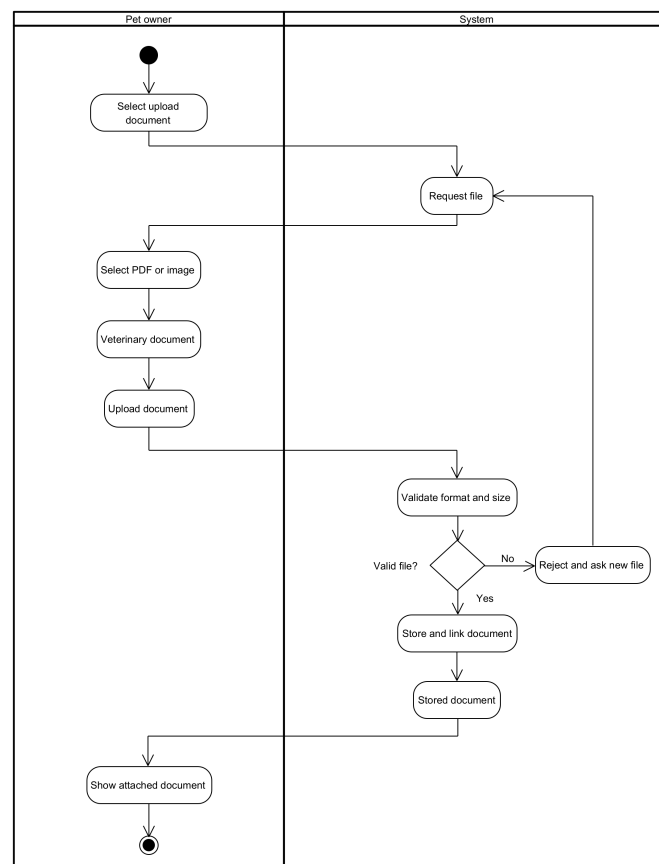


Figura 26: Diagrama de actividades – Cargar documento veterinario y carnet de vacunación (CU-03).

Que el rechazo del archivo (“Reject and ask new file”) regrese al nodo “Request file” (y no obligue a reiniciar todo el caso de uso desde “Select upload document”), tal como se ve en la Figura 26, revela que el sistema minimiza la fricción ante un archivo inválido, permitiendo reintentar solo el paso de selección del archivo sin perder el contexto de a qué historial médico pertenece.

4.9.4 Publicar mascota en adopción (CU-04)

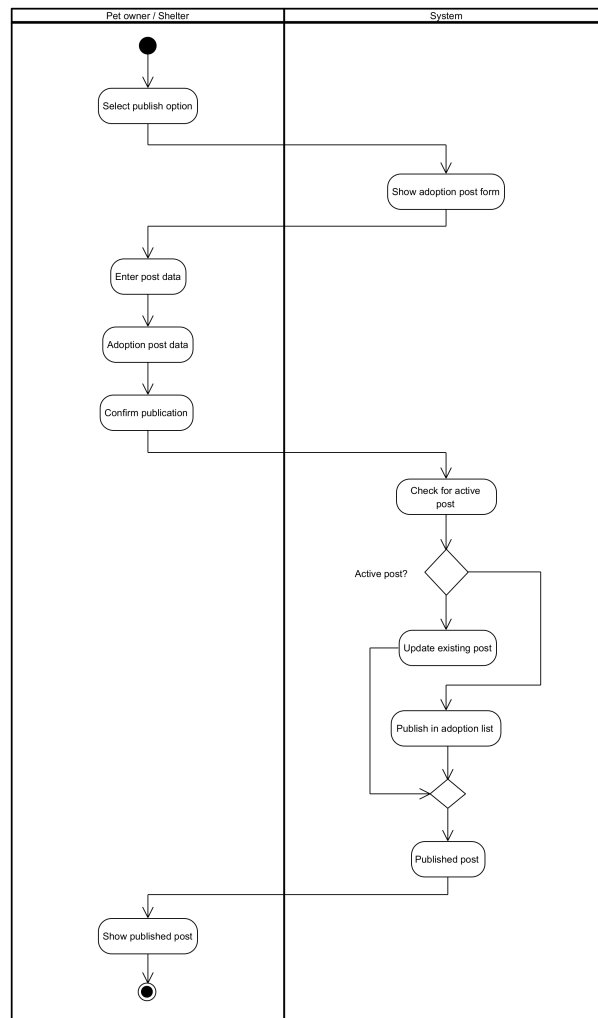


Figura 27: Diagrama de actividades – Publicar mascota en adopción (CU-04).

Que la decisión “Active post?” determine si se ejecuta “Update existing post” antes de fusionar el flujo hacia “Publish in adoption list” (ambas ramas convergen al mismo paso de publicación), como se observa en la Figura 27, revela que, exista o no una publicación previa, el resultado final es siempre una sola publicación activa vigente: el sistema absorbe la lógica de duplicados de forma transparente para el usuario, que solo ve el resultado publicado, sin pasos adicionales visibles según el caso.

4.9.5 Publicar mascota para cruce y gestionar solicitud (CU-05)

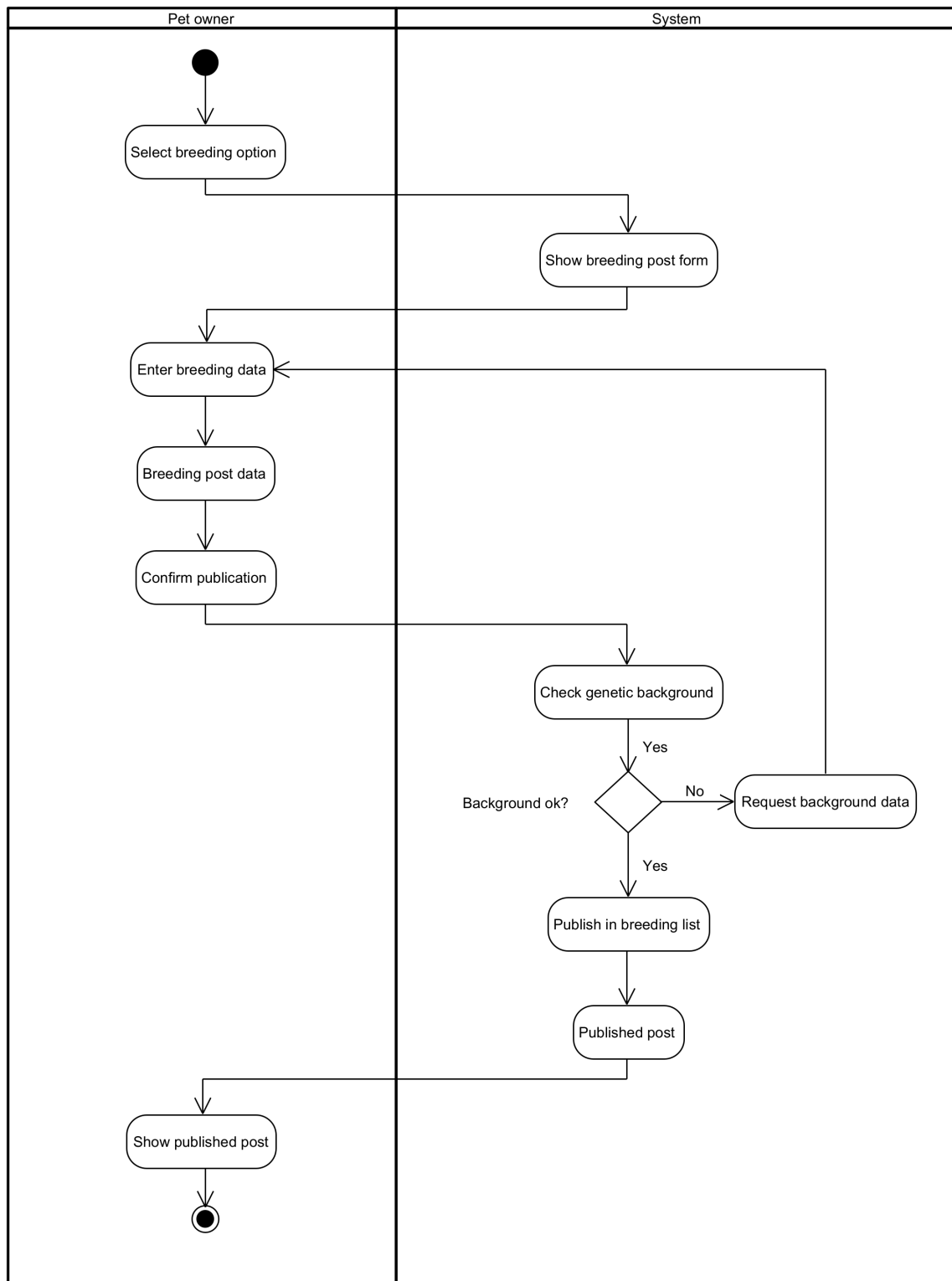


Figura 28: Diagrama de actividades – Publicar mascota para cruce y gestionar solicitud (CU-05).

Que “Background ok?” con salida “No” regrese específicamente a “Enter breeding data” (y no a “Show breeding post form” como en otros flujos), tal como se ve en la Figura 28, revela que el sistema no permite publicar sin importar cuántas veces se lo intente si falta el antecedente genético: el dueño queda atrapado en un ciclo de corrección hasta cumplir este requisito no negociable, reafirmando visualmente lo mismo que ya mostró el diagrama de secuencia CU-05 (Figura 15): la trazabilidad genética es condición dura, no sugerencia.

4.9.6 Configurar privacidad médica (CU-06)

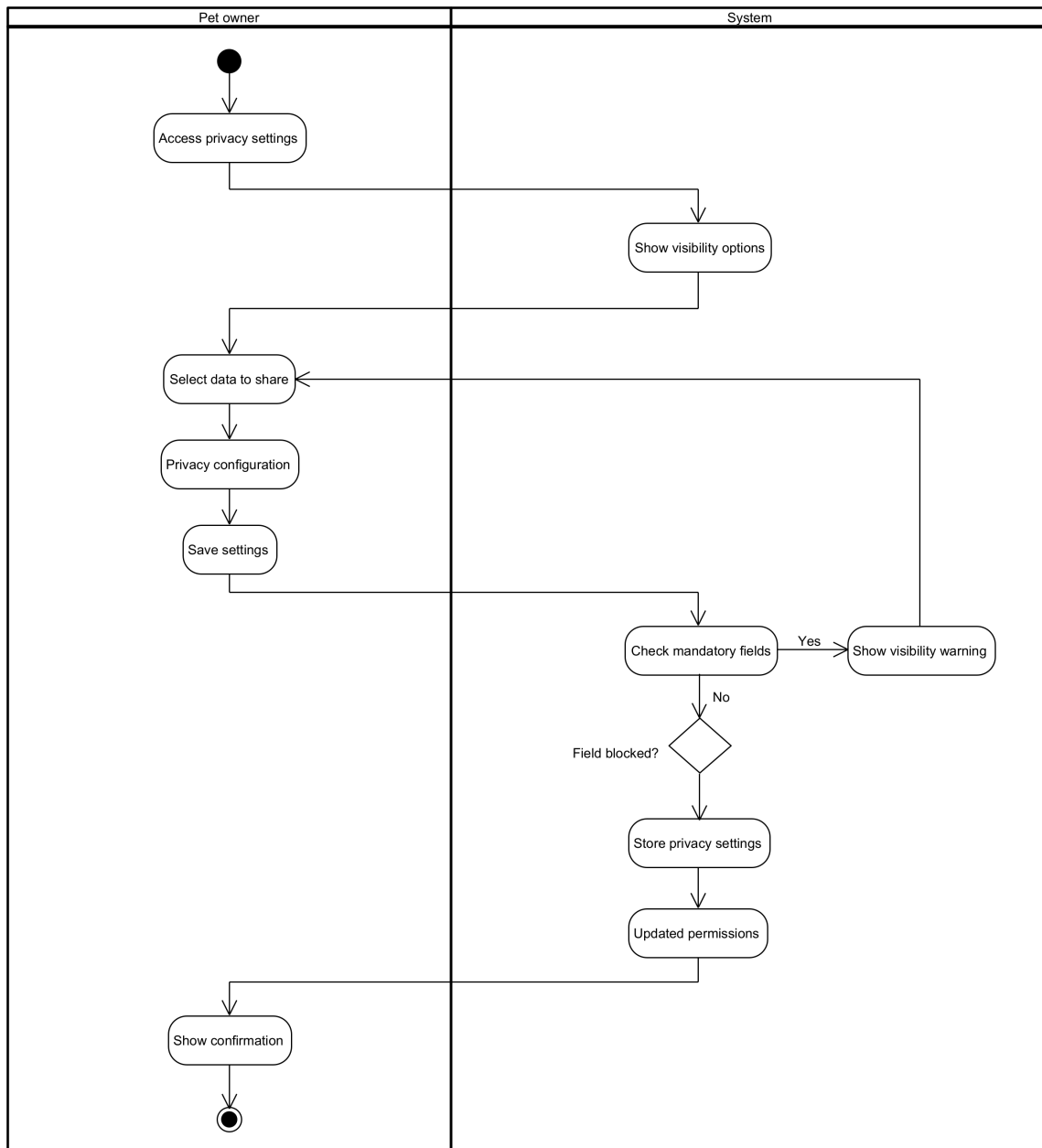


Figura 29: Diagrama de actividades – Configurar privacidad médica (CU-06).

Que “Check mandatory fields” con salida “Yes” (campo bloqueado indebidamente) regrese a “Select data to share” mediante “Show visibility warning” (en vez de simplemente no guardar el cambio en silencio), como se ve en la Figura 29, revela que el sistema educa activamente al dueño sobre por qué su configuración fue rechazada, en lugar de aplicar un filtro invisible que ignore la instrucción del usuario sin explicación.

4.9.7 Gestionar proceso de adopción (CU-07)

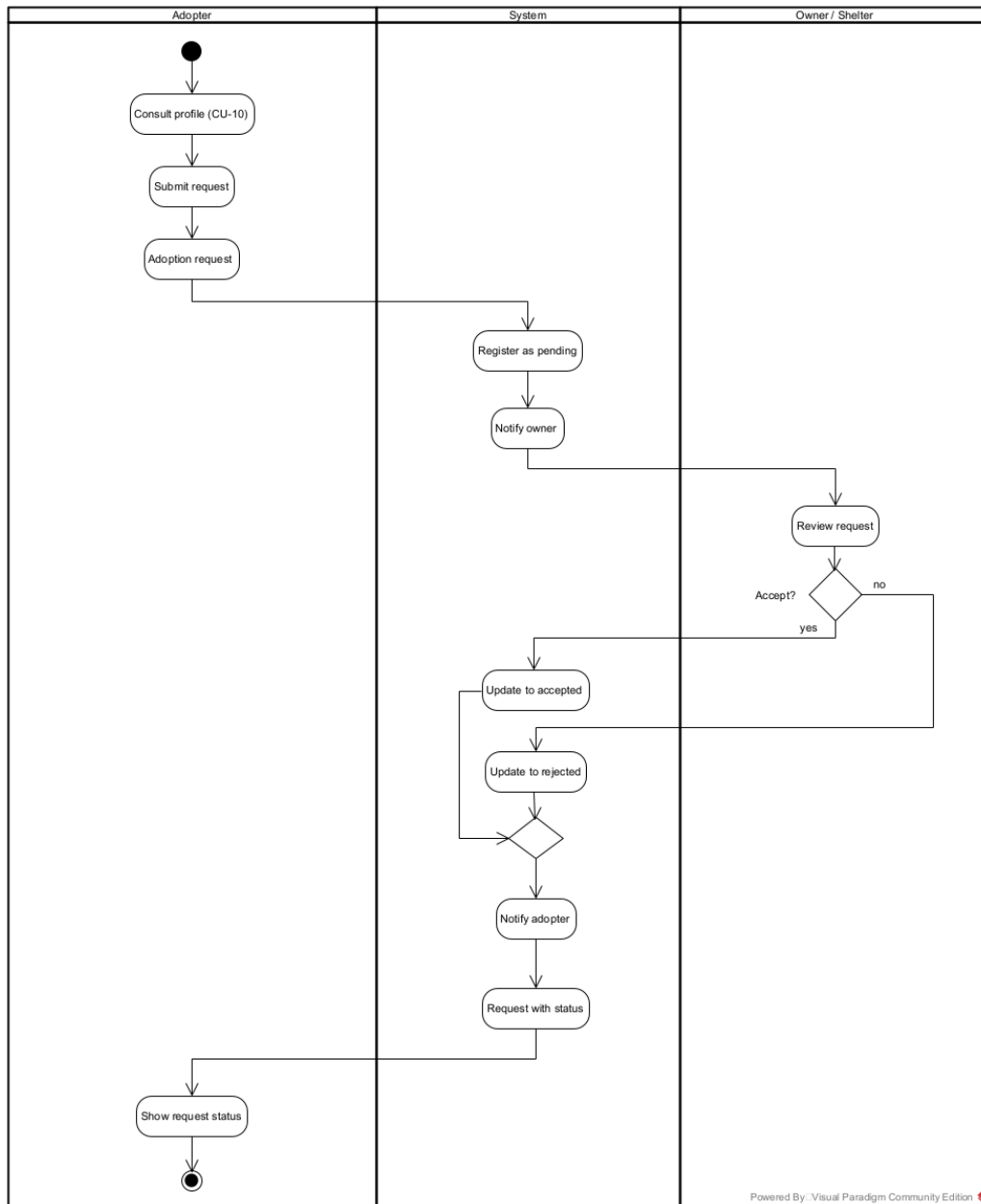


Figura 30: Diagrama de actividades – Gestionar proceso de adopción (CU-07).

Que exista un carril separado “Owner / Shelter” distinto del carril “System” para la actividad “Review request”, tal como se ve en la Figura 30, revela que la decisión de aceptar o rechazar una solicitud es exclusivamente humana y deliberada, sin ningún paso automatizado de aprobación: el sistema solo gestiona el registro y la notificación, nunca decide en nombre del dueño.

4.9.8 Gestionar comunicación entre usuarios (CU-08)

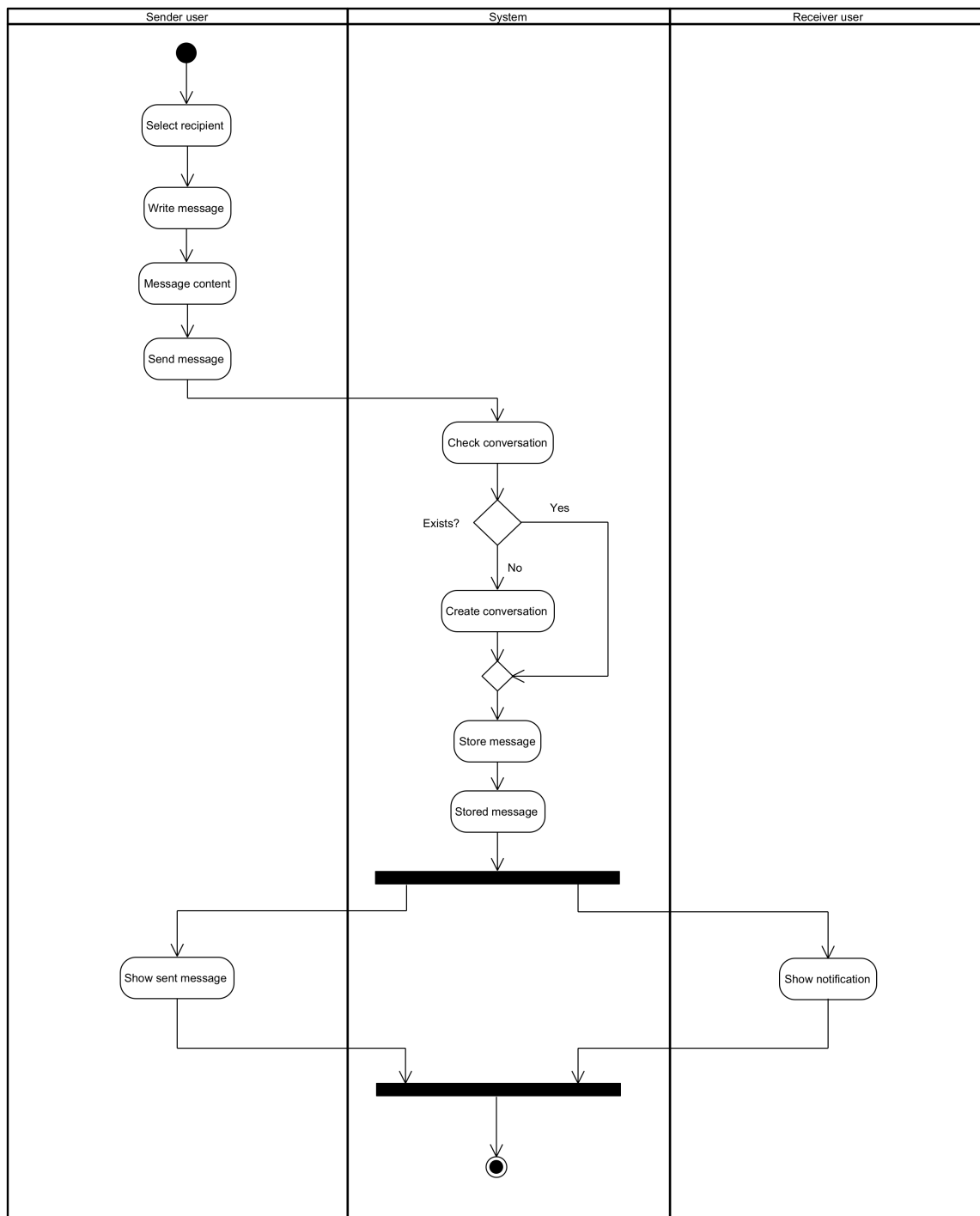


Figura 31: Diagrama de actividades – Gestionar comunicación entre usuarios (CU-08).

Que el diagrama cierre con una bifurcación (*fork*) que ejecuta en paralelo “Show sent message” (carril *Sender*) y “Show notification” (carril *Receiver*), como se observa en la Figura 31, revela que el sistema trata la confirmación del emisor y la alerta del receptor como dos efectos simultáneos e independientes de un mismo evento de guardado, y no como una cadena secuencial donde uno depende del otro.

4.9.9 Validar información médica (CU-09)

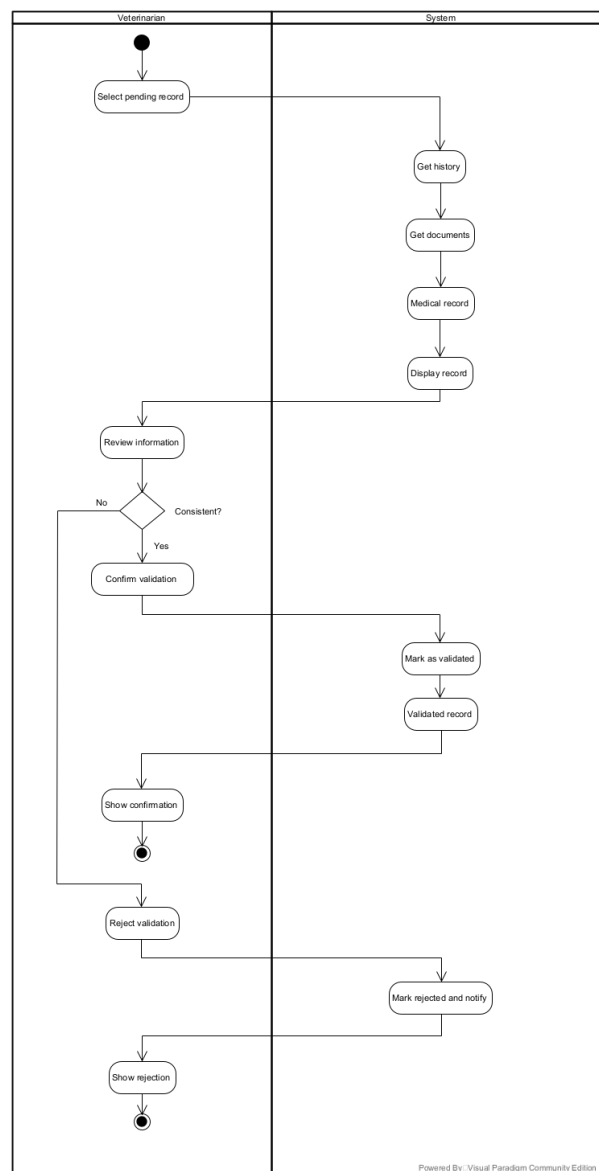


Figura 32: Diagrama de actividades – Validar información médica (CU-09).

Que el flujo de rechazo (“Reject validation”) no reingrese al ciclo de revisión sino que termine directamente en su propio nodo final (“Show rejection”), tal como se ve en la Figura 32, revela que, a diferencia del CU-05 o CU-06, esta actividad no fuerza una corrección inmediata: el veterinario simplemente cierra el caso como rechazado y es el dueño quien deberá iniciar un nuevo ciclo de validación por separado, coherente con la máquina de estados de Verificación de Documentos (Figura 10).

4.9.10 Consultar perfil médico (CU-10)

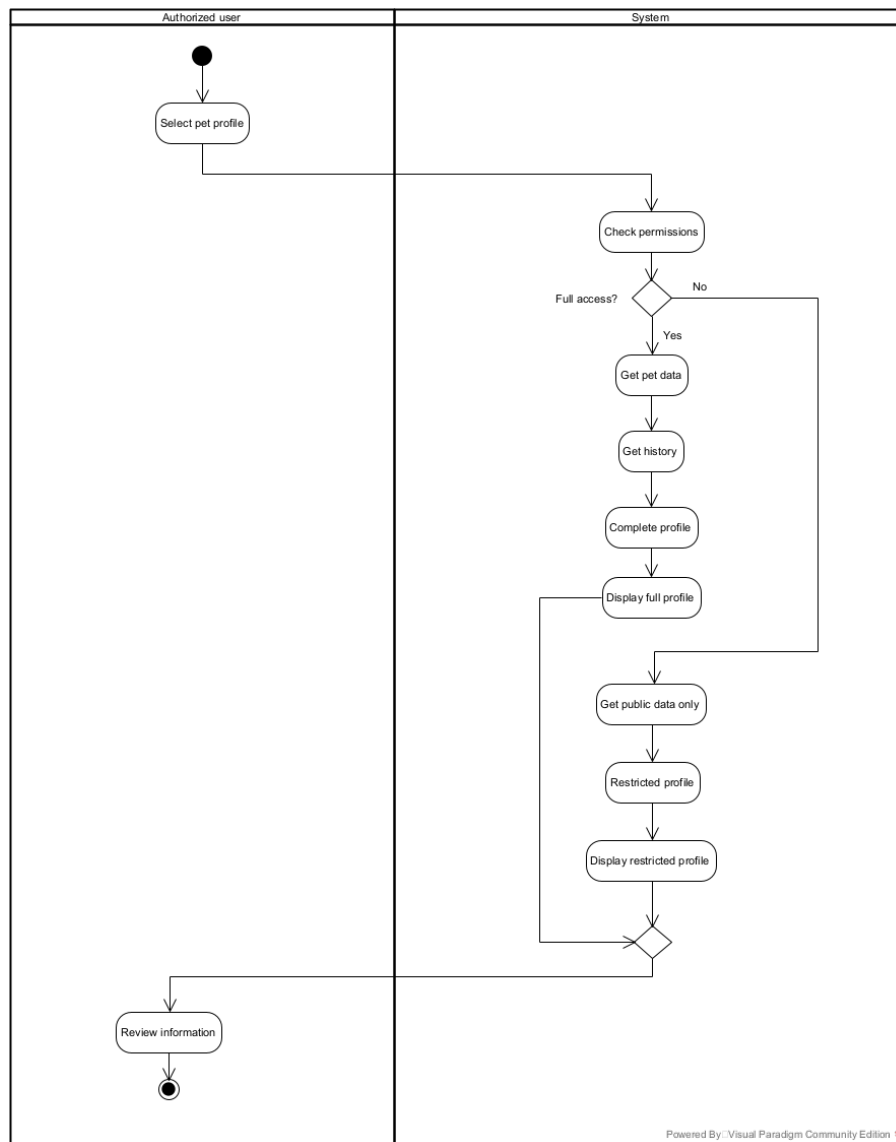


Figura 33: Diagrama de actividades – Consultar perfil médico (CU-10).

Que las dos ramas de la decisión “Full access?” (perfil completo vs. restringido) converjan en un mismo punto de unión antes de “Review information”, como se observa en la Figura 33, revela que, desde la perspectiva del usuario final, ambos caminos son indistinguibles en la interfaz: siempre recibe “un perfil”, solo que su contenido varía según el permiso, reforzando que la restricción ocurre completamente del lado del sistema y nunca se expone como un estado de error al usuario.

4.9.11 Buscar mascota disponible (CU-11)

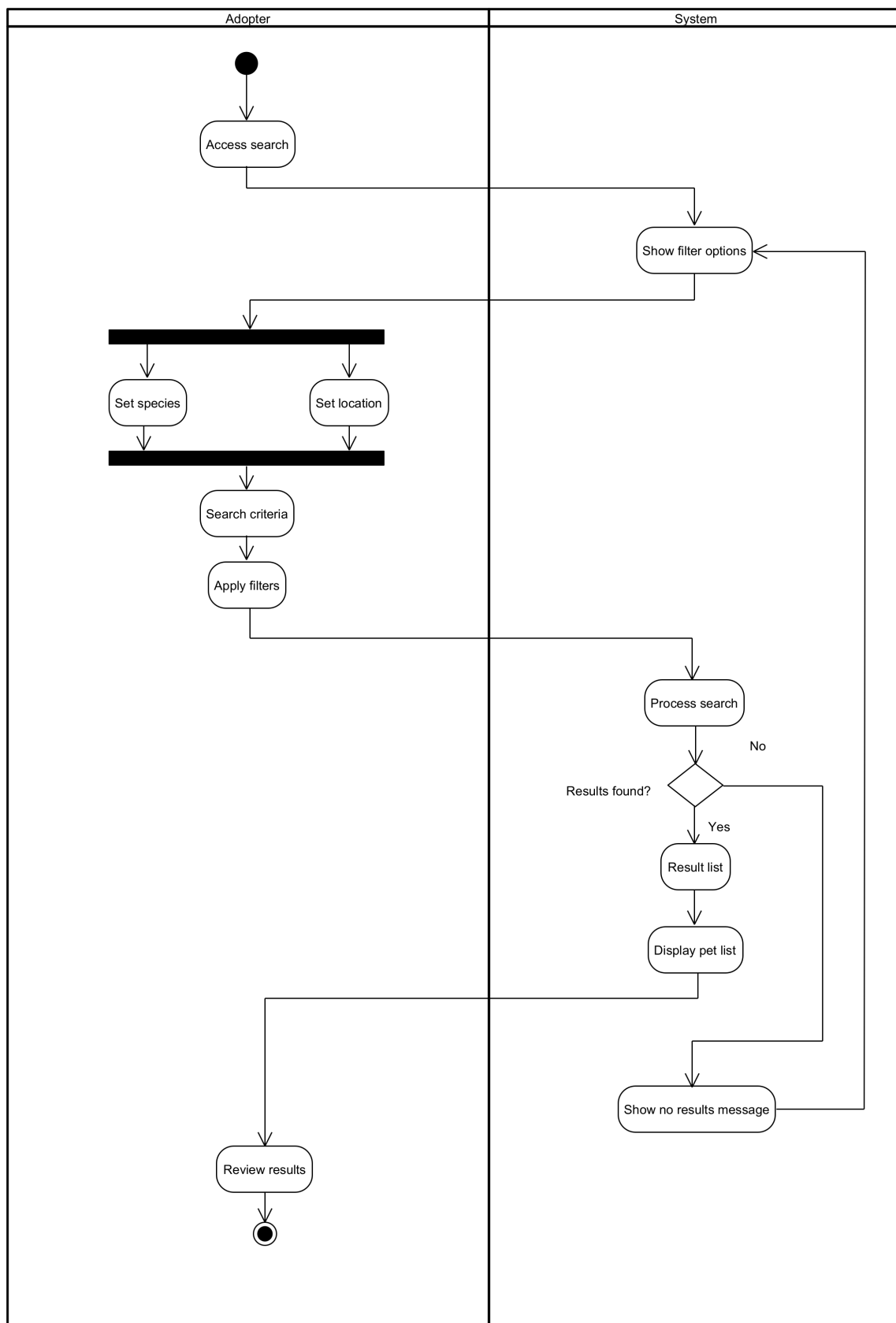


Figura 34: Diagrama de actividades – Buscar mascota disponible (CU-11).

Que “Set species” y “Set location” se definan como actividades paralelas (*fork/join*) dentro de “Apply filters”, tal como se ve en la Figura 34, revela que los criterios de búsqueda son independientes entre sí y combinables libremente, sin que uno dependa de haber definido antes el otro, permitiendo búsquedas parciales (solo especie, o solo ubicación).

4.9.12 Consultar recomendaciones de cuidado (CU-12)

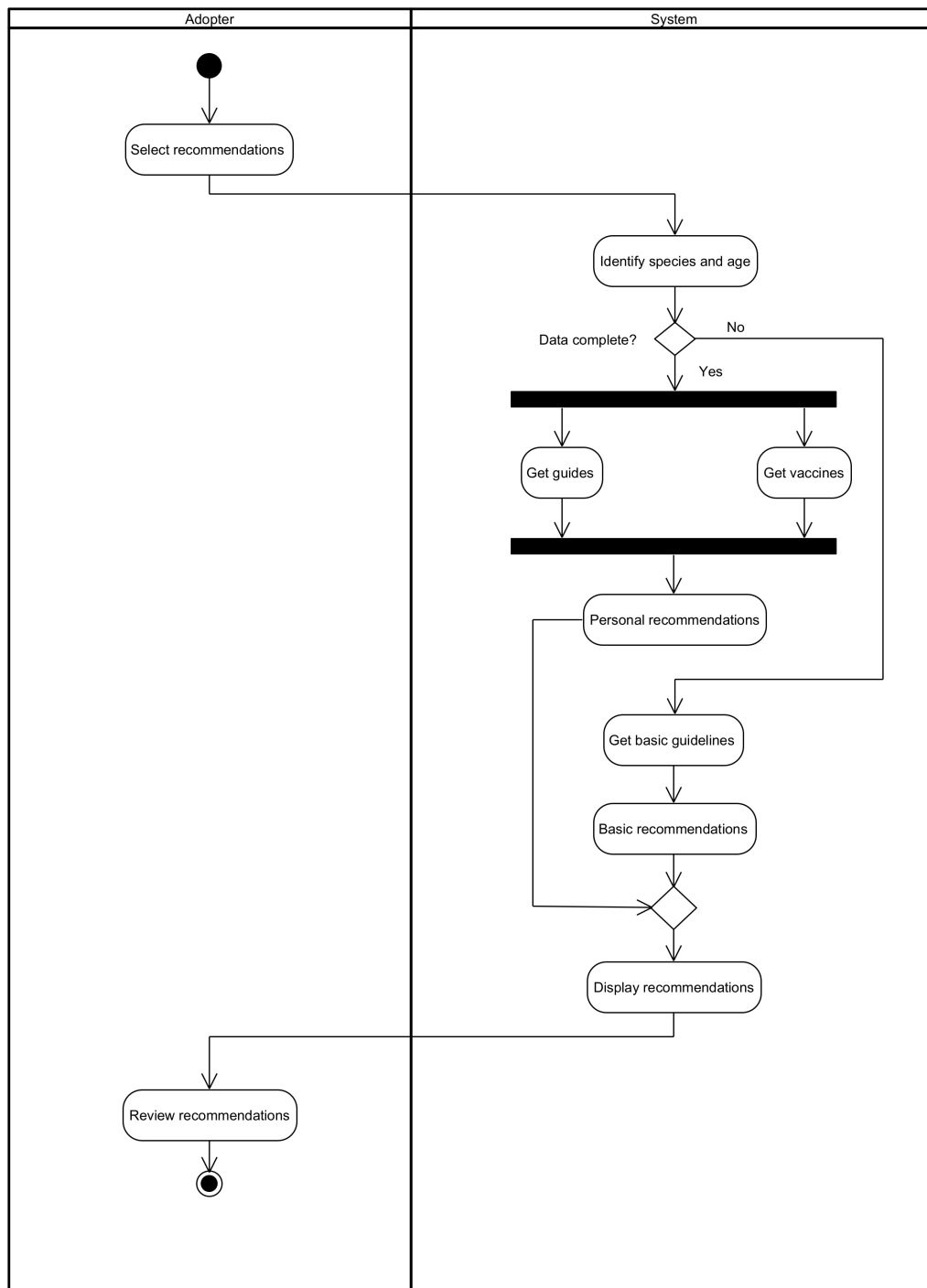


Figura 35: Diagrama de actividades – Consultar recomendaciones de cuidado (CU-12).

Que “Data complete?” con salida “No” desvíe directamente hacia “Get basic guidelines” (evitando por completo la rama paralela de “Get guides”/“Get vaccines”), como se ve en la Figura 35, revela que el sistema no intenta forzar al adoptante a completar el perfil antes de dar alguna recomendación: prefiere entregar un resultado genérico útil de inmediato en lugar de bloquear la funcionalidad por datos incompletos, igual que se observó en el diagrama de secuencia correspondiente (Figura 22).

4.9.13 Evaluar compatibilidad de cruce (CU-13)

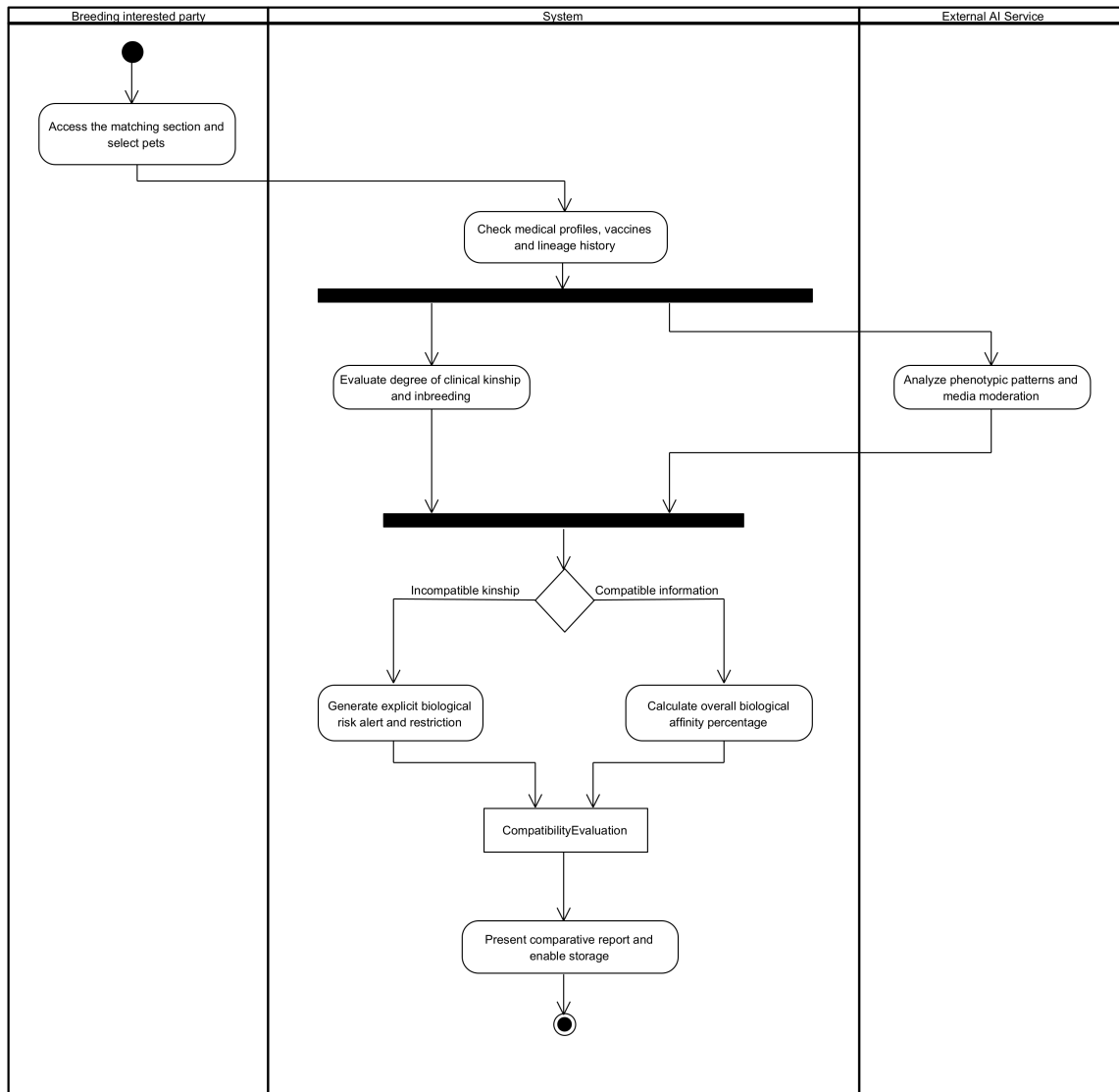


Figura 36: Diagrama de actividades – Evaluar compatibilidad de cruce (CU-13).

Que “Evaluate degree of clinical kinship and inbreeding” (carril *System*) y “Analyze phenotypic patterns and media moderation” (carril *External AI Service*) se ejecuten en paralelo, pero que la decisión final (“Incompatible kinship” vs. “Compatible information”) dependa de fusionar ambos resultados antes de bifurcarse hacia “Generate explicit biological risk alert” o “Calculate overall biological affinity percentage”, como se observa en la Figura 36, revela visualmente el mismo principio observado en el diagrama de secuencia CU-13 (Figura 23): el veredicto de riesgo determinista (parentesco) y el análisis probabilístico de la IA operan de forma independiente, pero la alerta de riesgo biológico tiene prioridad sobre el cálculo de afinidad cuando ambos resultados entran en conflicto.

4.10 Diagrama de Clases

4.10.1 Usuarios y Roles

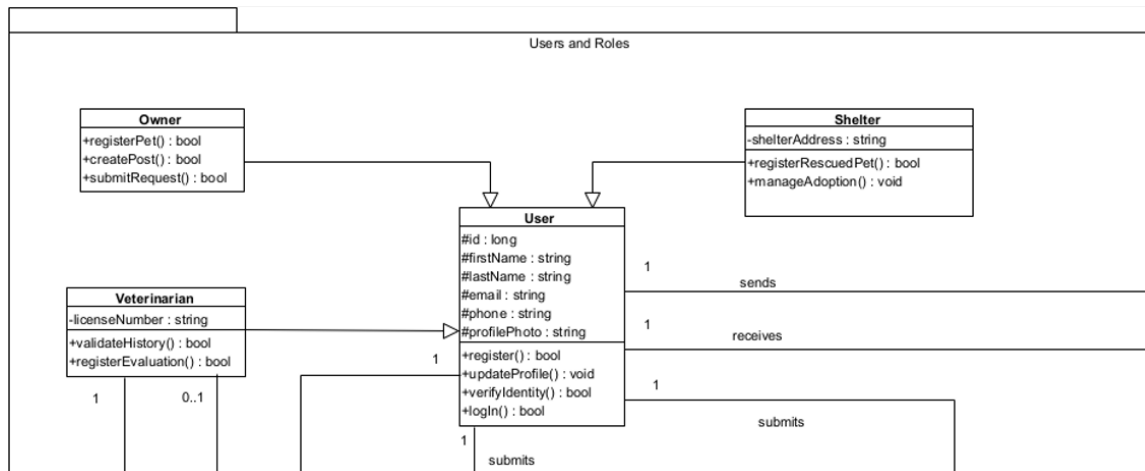


Figura 37: Diagrama de clases – Usuarios y Roles.

Que “Shelter”, “Owner” y “Veterinarian” hereden todos de una única clase base “User” (en lugar de modelarse como entidades independientes con su propia tabla de *login*), como se observa en la Figura 37, revela que el sistema trata la identidad y autenticación como algo unificado y transversal, mientras que el comportamiento específico de cada actor (*registerRescuedPet()*, *validateHistory()*, etc.) se resuelve por especialización. La ausencia deliberada de una clase “Administrator” (y de un mecanismo dinámico de roles tipo *Role/UserRole*) refleja que MundiPets opera como una plataforma entre pares sin un rol administrativo central: el tipo de usuario — y por tanto sus permisos — queda resuelto enteramente por la propia jerarquía de herencia, sin necesitar una capa de gestión de roles independiente que ningún requisito funcional respalda.

Corrección aplicada (diagnóstico de cobertura, Sección 6): las clases *Administrator*, *Role* y *UserRole* se eliminaron de esta partición del diagrama de clases por constituir una cadena rota — ningún RF ni CU las respaldaba—. El control de acceso por tipo de usuario queda completamente cubierto por la herencia *Shelter/Owner/Veterinarian* → *User*, ya utilizada de forma consistente en el campo “Actor” de los 27 requisitos funcionales del ERS.

4.10.2 Mascotas y Salud

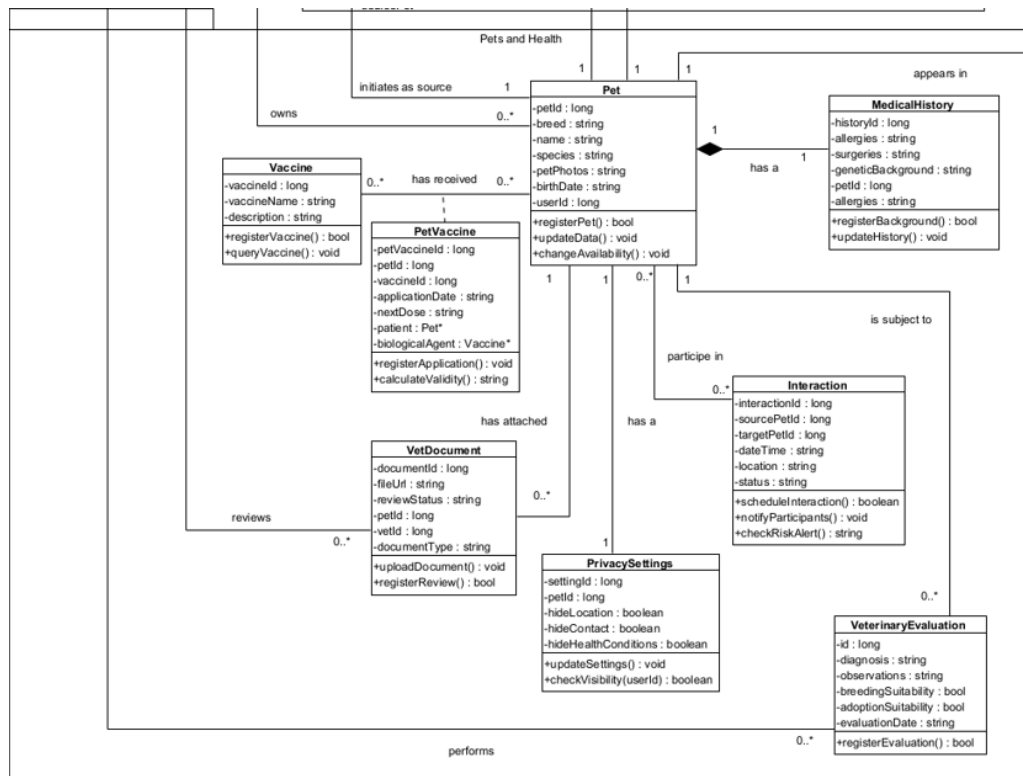


Figura 38: Diagrama de clases – Mascotas y Salud.

Que “MedicalHistory” se relacione con “Pet” mediante una composición (rombo relleno, “has a” 1 a 1) y no una simple asociación, como se aprecia en la Figura 38, revela que el historial médico no tiene existencia independiente de la mascota: si se elimina el perfil de la mascota, su historial desaparece con él, a diferencia de “VeterinaryEvaluation” o “VetDocument”, que se asocian de forma más débil (0..*) y podrían sobrevivir como registros históricos aun si la mascota deja de estar activa. Por otro lado, que “PetVaccine” sea una clase de asociación entre “Pet” y “Vaccine” (con atributos propios como *nextDose* y *biologicalAgent*) en lugar de una relación directa, muestra que el sistema necesita rastrear cada aplicación individual de una vacuna (con su propia fecha y próxima dosis) y no solo si la mascota “tiene” cierta vacuna en abstracto. Que “Interaction” reutilice el mismo patrón estructural que “BreedingRequest” (dos referencias a “Pet”, origen y destino, más un método *checkRiskAlert()*) revela que el sistema reconoce que cualquier vínculo entre dos mascotas concretas, sea un encuentro de socialización o una solicitud de cruce, comparte el mismo riesgo biológico de fondo y debe validarse con la misma lógica, en lugar de duplicar esa regla de negocio en dos lugares distintos del modelo. Que “PrivacySettings” se relacione con “Pet” mediante composición 1 a 1 (“has a”), igual que “MedicalHistory”, muestra que la configuración de privacidad no es un ajuste global de la cuenta del usuario, sino un dato que pertenece y viaja con cada mascota individual: dos mascotas del mismo propietario pueden tener reglas de visibilidad distintas entre sí.

4.10.3 Comunicación y Publicaciones

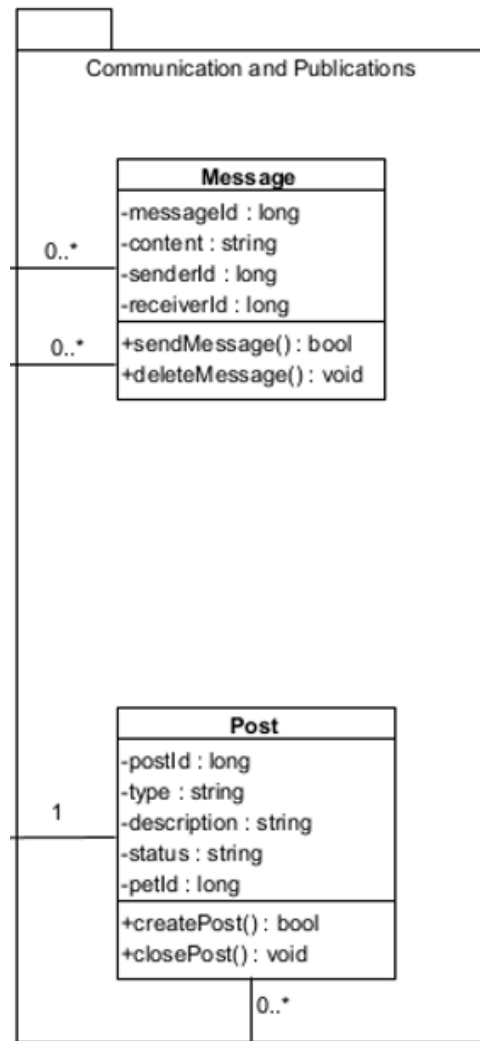


Figura 39: Diagrama de clases – Comunicación y Publicaciones.

Que “Message” siga sin ninguna relación directa con “Post”, ni siquiera ahora que existe “Interaction” para modelar encuentros presenciales (partición Mascotas y Salud), como se observa en la Figura 39, refuerza la misma lectura de diseño ya identificada: la mensajería es un canal transversal entre usuarios, desacoplado tanto del contexto de una publicación como del registro formal de un encuentro coordinado. Esto evita que “Message” termine sobrecargado con responsabilidades que no le corresponden (agendar, registrar ubicación, marcar estado de un encuentro), las cuales fueron correctamente delegadas a una clase propia (“Interaction”) en lugar de resolverse con campos adicionales dentro del propio mensaje.

4.10.4 Solicitudes y Evaluaciones

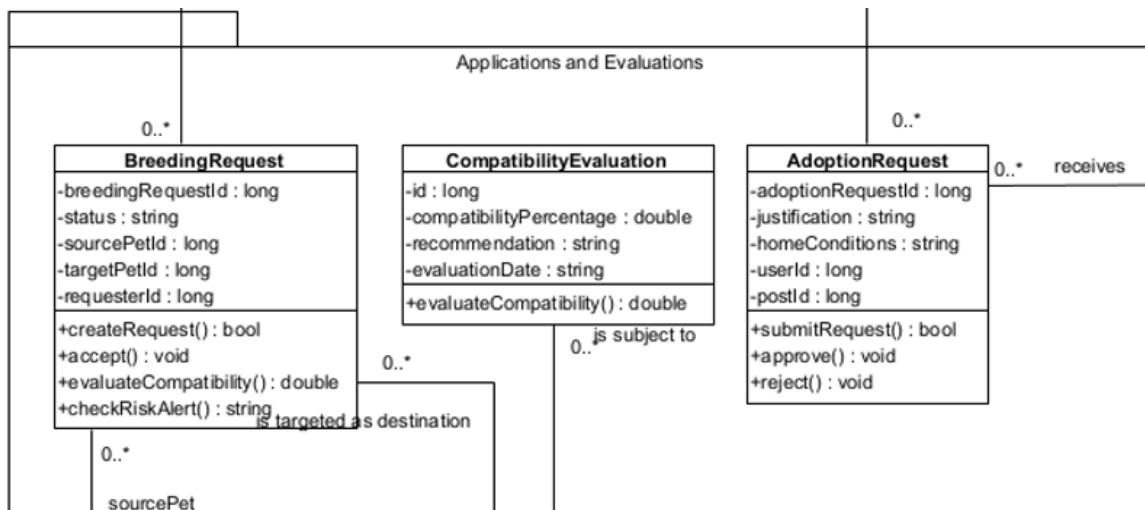


Figura 40: Diagrama de clases – Solicitudes y Evaluaciones.

Que *checkRiskAlert()* se agregue como método propio de “BreedingRequest” (y no como una clase de evaluación independiente, a diferencia de “CompatibilityEvaluation”), como se ve en la Figura 40, revela que la alerta de riesgo sanitario o físico se trató deliberadamente como una validación transitoria calculada al momento de crear la solicitud, no como un resultado que deba persistirse y auditarse después: mientras “CompatibilityEvaluation” sí se guarda como registro propio (porque su recomendación debe poder consultarse más adelante), la alerta de riesgo solo necesita bloquear o advertir en el instante de la confirmación, y desaparece una vez tomada la decisión.

4.10.5 Vista Refinada General

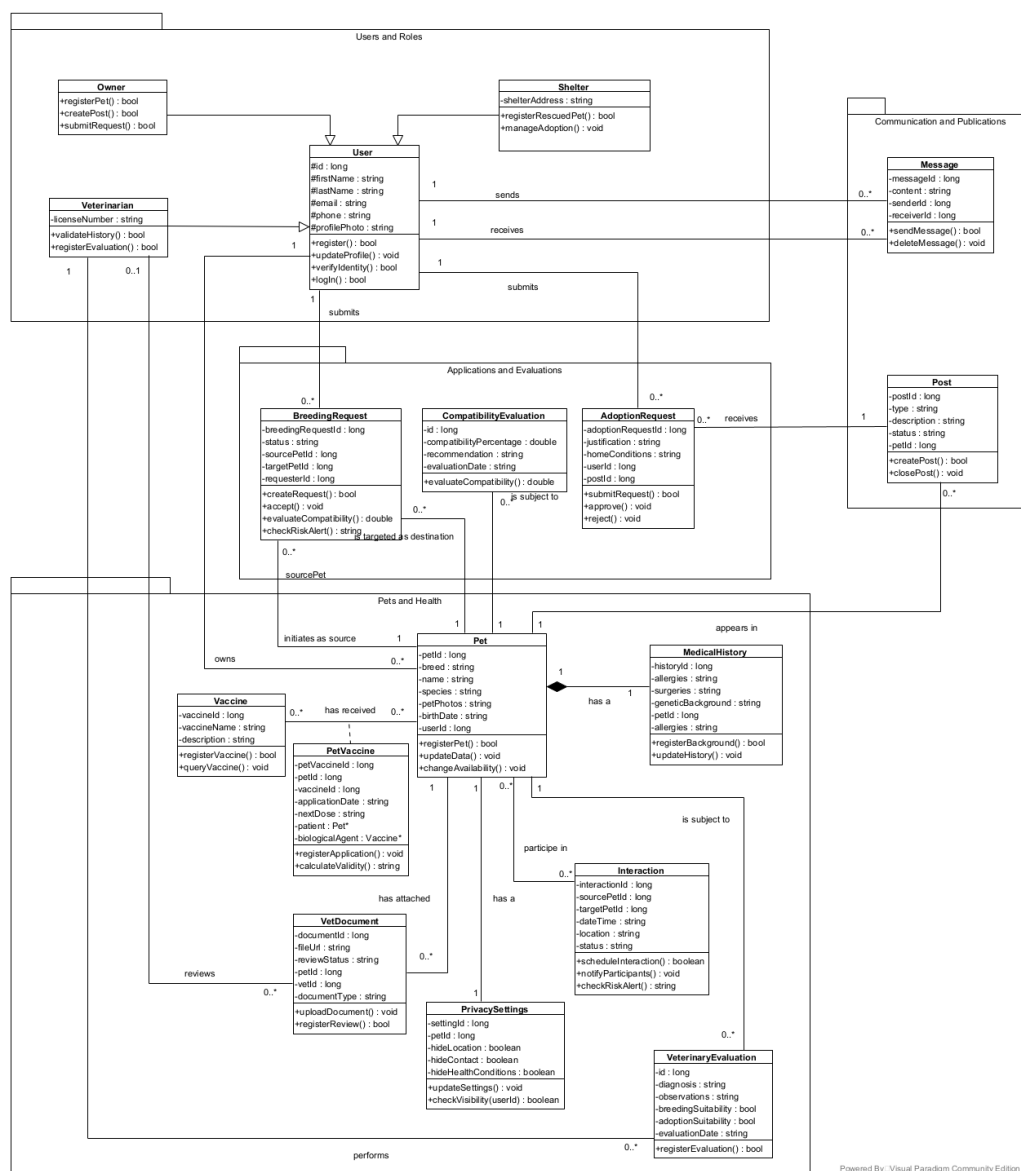


Figura 41: Diagrama de clases – Vista refinada general (integración de las cuatro particiones).

Que “Pet” sea la clase con más conexiones entrantes de todo el diagrama (origen de “AdoptionRequest”, “BreedingRequest” vía *sourcePet* / “is targeted as destination”, “MedicalHistory”, “PetVaccine”, “VetDocument”, “VeterinaryEvaluation” y “Post”), como se observa en la Figura 41, confirma visualmente lo que ya el diagrama de Contexto (Figura 2) y la matriz de trazabilidad (Sección 6) venían señalando: la mascota es el agregado central del dominio de MundiPets, y prácticamente ningún proceso (adopción, cruce, salud, comunicación) tiene sentido sin anclarse primero a un “Pet” concreto. Esto también explica por qué la Sección 8 (trazabilidad) probablemente encuentre más filas asociadas a RF relacionados con “Pet” que con cualquier otra entidad, y por qué un defecto en el modelo de “Pet” tendría el radio de impacto más amplio de todo el sistema (relevante para M5, modificabilidad).

Corrección aplicada (diagnóstico de cobertura, Sección 6): la clase *VeterinaryAppointment* (agendar/cancelar citas) se elimina de esta partición por constituir una cadena rota que además contradice dos exclusiones de alcance declaradas explícitamente en el ERS: MundiPets no reemplaza la consulta veterinaria presencial ni ofrece una agenda médica completa de clínicas. RF-10 (recordatorios de controles preventivos) cubre únicamente el aviso pasivo de vacunas y controles próximos, no la reserva activa de un turno dentro del sistema, por lo que ambas funciones no deben confundirse.

5 Validación: Inspección, Defectos y Re-inspección

5.1 Resultados de la inspección de la PE4

5.2 Correcciones aplicadas

5.3 Re-inspección de la PE5 y defectos residuales

6 Gestión y Trazabilidad

6.1 Línea base y control de cambios

6.2 Matriz de trazabilidad final

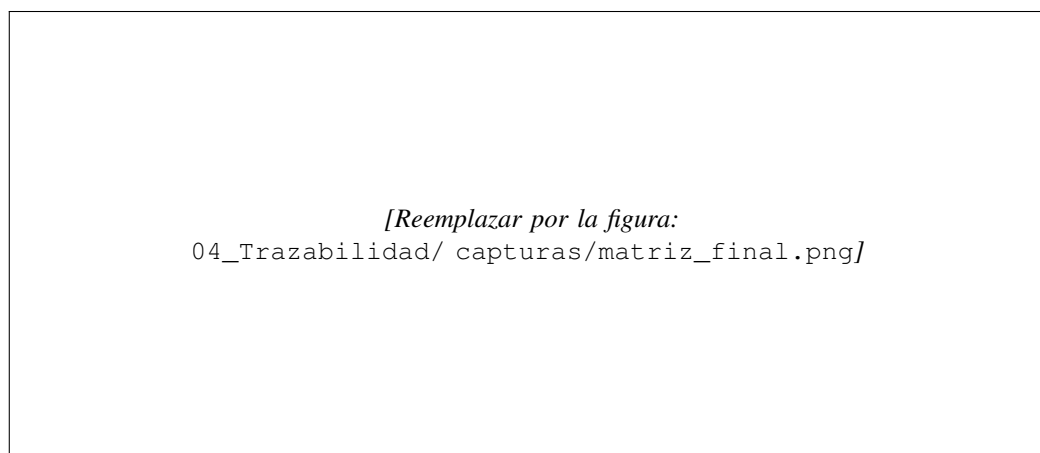


Figura 42: Vista general de la matriz de trazabilidad final de MundiPets.

6.3 Diagnóstico de cobertura: huérfanos y cadenas rotas

Tabla 6: Requisitos huérfanos, artefactos huérfanos y cadenas rotas.

Identificador	Patología	Causa	Acción tomada

6.4 Sincronización entre el *product backlog* y el ERS

Tabla 7: Sincronización entre el *product backlog* y el ERS.

Verificación	Cumple	Total	%
RF con al menos una historia de usuario asociada			
Historias del <i>backlog</i> con un RF de origen identificado			
Herramienta de tablero declarada: _____			

7 Requisitos de los Componentes de Inteligencia Artificial

7.1 Identificación y justificación de los componentes

7.2 Ficha del componente IA-01

Tabla 8: Ficha del componente de inteligencia artificial IA-01.

Campo	Contenido
Identificador y nombre	IA-01 —
Tarea y tipo	(recomendación / clasificación / detección de anomalías / conversación / visión)
Entradas y salidas	Variables de entrada, formato y semántica de la salida
Consumidor del resultado	Quién usa la salida y para qué decisión
Datos de entrenamiento	Origen, volumen estimado, variables, etiquetado, calidad mínima, sesgos conocidos, base legal del tratamiento y política de conservación
Métricas de éxito	Métrica principal con umbral, métricas secundarias, conjunto de evaluación
Equidad	Métrica de equidad, grupos comparados, brecha máxima tolerada
Explicabilidad	Qué se explica, a quién, en qué formato y en qué momento
Riesgos y <i>fallback</i>	Consecuencia del error y comportamiento del sistema si el modelo falla o no está disponible
Clasificación de riesgo	Nivel conforme al enfoque basado en riesgo del Reglamento (UE) 2024/1689 [16] y obligaciones derivadas
Plan de monitoreo	Indicadores en producción, frecuencia de medición, umbral de alerta por deriva, criterio de reentrenamiento o retirada
Trazas	RF y RNF del sistema con los que se relaciona

7.3 Requisitos funcionales y no funcionales de IA

Tabla 9: Requisitos funcionales y no funcionales de los componentes de IA.

ID	Tipo	Enunciado con umbral y unidad	Método de comprobación
RF-IA-01	RF		
RF-IA-02	RF		
RF-IA-03	RF		
RNF-IA-01	Rendimiento		
RNF-IA-02	Rendimiento		
RNF-IA-03	Rendimiento		
RNF-IA-04	Equidad		
RNF-IA-05	Equidad		
RNF-IA-06	Explicabilidad		

7.4 Ética, privacidad y clasificación de riesgo

7.5 Plan de monitoreo y trazado de los requisitos de IA

8 Métricas de Calidad del ERS

8.1 Instrumento de auditoría

8.2 Cálculo de las seis métricas con aritmética visible

$$M_{1a} = \frac{\text{req. con los 4 atributos completos}}{\text{req. totales}} \times 100 = \underline{\hspace{2cm}}$$

$$M_{1b} = \frac{\text{CU especificados}}{\text{CU identificados}} \times 100 = \underline{\hspace{2cm}}$$

$$M_{1c} = \frac{\text{actores con } \geq 1 \text{ RF}}{\text{actores}} \times 100 = \underline{\hspace{2cm}}$$

$$M_2 = 1 - \frac{\text{conflictos detectados}}{\text{pares de requisitos analizados}} = \underline{\hspace{2cm}}$$

$$M_3 = \frac{\text{req. con criterio BDD comprobable}}{\text{req. totales}} \times 100 = \underline{\hspace{2cm}}$$

$$M_{4ade} = \frac{\text{RF con cadena adelante completa}}{\text{RF}} \times 100 = \underline{\hspace{2cm}}$$

$$M_{4atr} = \frac{\text{RF con fuente o stakeholder identificado}}{\text{RF}} \times 100 = \underline{\hspace{2cm}}$$

$$M_5 = \frac{\sum_{i=1}^n \text{req. afectados al modificar } r_i}{n} = \underline{\hspace{2cm}}$$

$$M_6 = \frac{\text{defectos hallados después de la inspección}}{\text{req. totales}} = \underline{\hspace{2cm}}$$

8.3 Tabla de auditoría de calidad

Tabla 10: Tabla de auditoría de calidad del ERS de MundiPets.

Métrica	Valor obtenido	Valor de referencia	Referencia normativa	Cumple	Acción de mejora
M1 Compleitud		$\geq 95\%$	ISO 25010; 29148	S/N	
M2 Consistencia		$\geq 0,98$	ISO 25010	S/N	
M3 Verificabilidad		$\geq 90\%$	ISO 29148	S/N	
M4 Trazabilidad		90 / 100 %	ISO 29148	S/N	
M5 Modificabilidad		$\leq 3,0$	ISO 25010	S/N	
M6 Corrección		$\leq 0,05$	ISO 25010	S/N	

8.4 Comparación antes/después de las mejoras aplicadas

Tabla 11: Comparación antes/después de las mejoras aplicadas al ERS.

Métrica	Antes	Después	Mejora aplicada al ERS
M1			
M2			
M3			
M4			
M5			
M6			

9 Retrospectiva del Equipo

Tabla 12: Retrospectiva Start–Stop–Continue del equipo.

Categoría	Elemento (mínimo tres por categoría)	Responsable / acción
Start		
Stop		
Continue		

10 Conclusiones

Conclusión 1 — Unidad 1: Fundamentos y elicitación

Conclusión 2 — Unidad 2: Especificación del ERS

Conclusión 3 — Unidad 3: Modelado UML y trazabilidad

Conclusión 4 — Unidad 4: Validación y gestión de cambios

Conclusión 5 — Unidad 5: Integración, métricas e IA

Preparación de la Defensa Técnica

Reparto de la exposición

Tabla 13: Reparto de la exposición y los tiempos de la defensa.

#	Bloque de diapositivas	Expone	Tiempo
1–2	Portada; problema y contexto	NIEVES SÁNCHEZ, Jimmy Samuel	3 min
3–5	Sistema y <i>stakeholders</i> ; proceso de IR (PE1–PE5); ERS: estructura y requisitos clave	GUTIÉRREZ ORTEGA, Génesis Adriana	5 min
6	Modelos UML principales	MORALES SÁNCHEZ, Gary Alejandro	2 min
7	Validación: resultados de la inspección	FUERTES ARRAES, Edson Daniel	2 min
8	Trazabilidad y métricas	MORALES SÁNCHEZ, Gary Alejandro, FUERTES ARRAES, Edson Daniel	4 min
9	Componentes de inteligencia artificial	MENDOZA PÁRRAGA, Andy Johel	3 min
10–12	Lecciones aprendidas; conclusiones; preguntas	NIEVES SÁNCHEZ, Jimmy Samuel	1 min + cierre

Reglas de la defensa

Referencias

- [1] ISO/IEC, “ISO/IEC 25010:2023 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model,” International Organization for Standardization, International Standard, 2023.
- [2] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell y A. Wesslén, *Experimentation in Software Engineering*, Berlin, Heidelberg, Germany: Springer, 2012. DOI: 10.1007/978-3-642-29044-2
- [3] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering,” International Organization for Standardization, International Standard, 2018.
- [4] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2.^a ed. Berlin, Heidelberg: Springer, 2025, ISBN: 978-3-662-69204-2. dirección: <https://link.springer.com/book/9783662692042>
- [5] H. Washizaki, “Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0,” IEEE Computer Society, Los Alamitos, CA, USA, inf. téc., 2024. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [6] S. Bühne y M. Glinz, “CPRE Foundation Level Syllabus, Version 3.3.0,” International Requirements Engineering Board (IREB) e.V., Karlsruhe, Germany, inf. téc., 2026. dirección: <https://cpre.ireb.org/en/downloads-and-resources/downloads>
- [7] ISO/IEC/IEEE, *ISO/IEC/IEEE 15288:2023 – Systems and Software Engineering – System Life Cycle Processes*, Geneva, Switzerland, 2023. DOI: 10.1109/IEEESTD.2023.10123367 dirección: <https://www.iso.org/standard/81702.html>
- [8] ISO/IEC/IEEE, *ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes*, Geneva, Switzerland, 2017. DOI: 10.1109/IEEESTD.2017.8100771 dirección: <https://www.iso.org/standard/63712.html>
- [9] O. C. Z. Gotel y A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” *Proceedings of IEEE International Conference on Requirements Engineering*, págs. 94-101, 1994. DOI: 10.1109/ICRE.1994.292398
- [10] J. Mucha, A. Kaufmann y D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requirements Engineering*, vol. 29, n.º 2, págs. 119-141, 2024. DOI: 10.1007/s00766-023-00412-z
- [11] A. Vogelsang y M. Borg, “Requirements Engineering for Machine Learning: Perspectives from Data Scientists,” en *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*, IEEE, 2019, págs. 245-251. DOI: 10.1109/REW.2019.00050
- [12] K. Ahmad, M. Abdelrazek, C. Arora, M. Bano y J. Grundy, “Requirements Engineering for Artificial Intelligence Systems: A Systematic Mapping Study,” *Information and Software Technology*, vol. 158, pág. 107 176, 2023. DOI: 10.1016/j.infsof.2023.107176
- [13] U.-e. Habiba, M. Haug, J. Bogner y S. Wagner, “How mature is requirements engineering for AI-based systems? A systematic mapping study on practices, challenges, and future research directions,” *Requirements Engineering*, vol. 29, págs. 567-600, 4 dic. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-024-00432-3
- [14] S. Amershi et al., “Software Engineering for Machine Learning: A Case Study,” en *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, IEEE, 2019, págs. 291-300. DOI: 10.1109/ICSE-SEIP.2019.00042
- [15] K. M. Habibullah, G. Gay y J. Horkoff, “Non-Functional Requirements for Machine Learning: Understanding Current Use and Challenges Among Practitioners,” *Requirements Engineering*, vol. 28, n.º 2, págs. 283-316, 2023. DOI: 10.1007/s00766-022-00395-3
- [16] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial)*, Diario Oficial de la Unión Europea, L 2024/1689, 12/07/2024, 2024. dirección: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [17] Asamblea Nacional del Ecuador, *Ley Orgánica de Protección de Datos Personales*, Registro Oficial Suplemento N.º 459, 26 de mayo de 2021, 2021.
- [18] Presidencia de la República del Ecuador, *Reglamento General a la Ley Orgánica de Protección de Datos Personales*, Decreto Ejecutivo N.º 904, Registro Oficial N.º 435, 13 de noviembre de 2023, 2023.
- [19] Project Management Institute, “A Guide to the Project Management Body of Knowledge (PMBOK Guide),” Project Management Institute, Newtown Square, PA, USA, inf. téc., 2021.

- [20] Z. Hoy y M. Xu, “Agile Software Requirements Engineering Challenges-Solutions – A Conceptual Framework from Systematic Literature Review,” *Information*, vol. 14, n.º 6, pág. 322, 2023. DOI: 10.3390/info14060322
- [21] A. Mavin, P. Wilkinson, A. Harwood y M. Novak, “Easy Approach to Requirements Syntax (EARS),” en *2009 17th IEEE International Requirements Engineering Conference*, 2009, págs. 317-322. DOI: 10.1109/RE.2009.9
- [22] M. E. Fagan, “Design and Code Inspections to Reduce Errors in Program Development,” *IBM Systems Journal*, vol. 15, n.º 3, págs. 182-211, 1976. DOI: 10.1147/sj.153.0182

Apéndice

Tabla 14: Preguntas del tribunal, respuesta preparada y artefacto de respaldo.

#	Pregunta	Respuesta preparada	Artefacto
1	¿Cuál es la frontera del sistema y qué queda deliberadamente fuera?		
6	Muéstrenme un requisito y díganme de qué evidencia concreta nació.		
13	Tomen un RF y díganme qué se rompería si mañana cambia.		
16	¿Qué decide el modelo y qué pasa cuando se equivoca?		
19	¿Qué métrica salió peor y qué hicieron al respecto?		
20	Enséñenme los conteos con los que calcularon la verificabilidad.		
22	¿Qué partes del informe fueron asistidas por IA y cómo las validaron?		

Tabla 15: Declaración individual de aporte al proyecto integrador.

Integrante	Aportes concretos (artefactos y secciones)	Evidencia en Git (<i>commits</i> , archivos, fechas)
FUERTES ARRAES, Edson Daniel		
MORALES SÁNCHEZ, Gary Alejandro		
MENDOZA PÁRRAGA, Andy Johel		
GUTIÉRREZ ORTEGA, Génesis Adriana		
NIEVES SÁNCHEZ, Jimmy Samuel		

[Insertar captura del historial de commits con autoría y fecha por integrante]

Figura 43: Historial de *commits* del repositorio al corte de la PE5.

Tabla 16: Declaración de uso de inteligencia artificial por sección.

Sección	Herramienta	Tipo de asistencia	Método de validación

El equipo declara ser responsable de la veracidad, coherencia y originalidad del contenido final entregado.

Tabla 17: Matriz de trazabilidad final del sistema MundiPets.

RF/RNF	Fuente	CU	Clase	DFD	Estado	Criterio BDD	CP	Historia	Estado traza
RF-01									
RF-02									
RF-03									
<i>[Continuar hasta RF-27, RNF-01–RNF-16 y los requisitos RF-IA / RNF-IA]</i>									